

FrontendEdu

База знаний фронтенд-разработчика

React Middle+

Составил: Sergei Krk & Hermes AI

Дата сборки: 2026-07-23

Содержание

План обучения React Middle+

- Рoadmap
- Сводка компетенций Middle+
- Как отслеживать прогресс
- Связанное

TypeScript продвинутый

- 1. Generics (дженерики)
- 2. Utility Types — швейцарский нож
- 3. Discriminated Unions — вместо ``status: string``
- 4. Conditional Types и ``infer``
- 5. Template Literal Types
- 6. Mapped Types — трансформация типов полей
- 7. ``as const`` и ``satisfies``
- 8. Распространённые паттерны в React

Event Loop: макротаски и микротаски

- Почему это важно
- Как устроен Event Loop
- Макротаски (MacroTasks)
- Микротаски (MicroTasks)
- Примеры с разбором
- Node.js: отличия
- ``requestAnimationFrame`` (браузер)
- Типичные вопросы на собеседовании

Алгоритмические задачи

- 1. Debounce
- 2. Throttle
- 3. Promise Pool (ограничение параллельности)
- 4. Flatten (разворачивание массива)
- 5. Deep Equal
- 6. Promise.all (своя реализация)
- 7. EventEmitter (Observer / PubSub)
- Связанное

Браузер и HTTP

- 1. Critical Rendering Path (критический путь рендеринга)
- 2. Reflow vs Repaint
- 3. CORS (Cross-Origin Resource Sharing)
- 4. Cookie / localStorage / sessionStorage

- 5. HTTP-кэширование
- 6. HTTP/2 и HTTP/3
- Связанное

Web API

- 1. Service Workers и PWA
- 2. WebSocket vs SSE vs Long Polling
- 3. History API (SPA-роутинг)
- 4. Web Workers
- 5. Intersection Observer и Resize Observer
- Fetch API и AbortController
- Связанное

CSS и стилизация

- Объяснение для ребёнка
- Junior-уровень
- Middle-уровень
- Senior-уровень
- Связанное

HTML и Accessibility (a11y)

- Уровень 1: Объясни ребёнку (10 лет)
- Уровень 2: Junior-разработчик
- Уровень 3: Middle-разработчик
- Уровень 4: Senior-разработчик
- Связанное

React: рендеринг и производительность

- 1. Reconciliation — как React понимает, что менять
- 2. Fiber — переписанный движок (React 16+)
- 3. Что вызывает ререндер
- 4. Инструменты предотвращения ререндеров
- 5. Три закона оптимизации React
- 6. Инструменты профилирования
- 7. `useTransition` и `useDeferredValue` (React 18+)
- 8. Распространённые ошибки

Управление состоянием

- 1. Серверное vs клиентское состояние
- 2. Серверное состояние: TanStack Query (React Query)
- 3. Клиентское состояние: Zustand
- 4. Когда что использовать — дерево решений
- 5. URL как состояние (React Router)
- 6. Антипаттерны

- Связанное

Архитектура React компонентов

- 1. Composition over Configuration
- 2. Compound Components
- 3. Headless Components
- 4. Slots Pattern (подход Next.js / shadcn/ui)
- 5. Custom Hooks — вынос логики из компонентов
- 6. Принцип единственной ответственности в компонентах
- 7. Render Props vs Hooks — эволюция
- 8. Server Components (React 19 / Next.js 14+)

Архитектурные паттерны

- 1. MVC → Flux → Redux → Zustand/Query
- 2. Observer / PubSub (EventEmitter)
- 3. Singleton
- 4. Factory и Strategy в React
- 5. Module Pattern и Revealing Module Pattern
- Связанное

Тестирование React

- 1. Пирамида тестирования
- 2. Unit-тесты: Vitest
- 3. Интеграционные тесты: React Testing Library
- 4. Моки API: MSW (Mock Service Worker)
- 5. E2E-тесты: Playwright
- 6. Что тестировать — приоритеты
- 7. Антипаттерны в тестах
- Связанное

Сборщики и инструменты

- 1. Webpack vs Vite
- 2. Tree-shaking
- 3. Babel и полифилы
- 4. HMR (Hot Module Replacement)
- 5. Code Splitting
- 6. Source Maps
- Связанное

Безопасность фронтенда

- 1. XSS (Cross-Site Scripting)
- 2. CSRF (Cross-Site Request Forgery)
- 3. CSP (Content Security Policy)
- 4. Другие заголовки безопасности

- 5. Чек-лист безопасности для фронтендера
- Связанное

DevOps для фронтендера

- 1. Docker
- 2. CI/CD Pipeline
- 3. Nginx для SPA
- 4. Переменные окружения
- 5. Мониторинг и алертинг
- Связанное

Micro Frontends и Module Federation

- Что это и зачем
- Module Federation (Webpack 5)
- Shared-зависимости: ключевой момент
- Module Federation 2 (Rspack / ByteDance)
- Роутинг между микрофронтендами
- Коммуникация между микрофронтендами
- Версионирование и деплой
- Антипаттерны

Feature-sliced design и Atomic Design

- Feature-Sliced Design (FSD)
- Atomic Design
- Что отвечать на собеседовании
- Антипаттерны
- Связанное

Виртуализация рендеринга

- Проблема
- Решение: виртуализация (windowing)
- react-window (react-window)
- TanStack Virtual
- Бесконечный скролл + виртуализация
- Что отвечать на собеседовании
- Антипаттерны
- Связанное

Интернационализация и локализация i18n

- i18n vs l10n
- Основные задачи i18n
- react-intl (FormatJS)
- i18next + react-i18next
- react-intl vs i18next

- Загрузка переводов: стратегии
- RTL (Right-to-Left)
- Что отвечать на собеседовании

Практика написания хуков и usehooks-ts

- Почему именно хуки
- usehooks-ts: библиотека и учебник
- Методика практики
- Что именно проверяет написание хуков
- Связанное

Soft skills собеседование

- 1. Расскажи про конфликт в команде
- 2. Самый сложный баг
- 3. Почему уходишь / ищешь работу
- 4. Как оцениваешь задачу по времени
- 5. Кем ты видишь себя через 3 года
- 6. Расскажи про проект, которым гордишься
- 7. Вопросы, которые Ты задаёшь компании
- Связанное

Вопросы на собеседовании FullStack

- 1. Самый сложный фуллстэк-проект: технологии и почему
- 2. Оптимизация при наплыве пользователей
- 3. Архитектура БД: пользователи и события со множеством связей
- 4. Безопасность фронтенд ↔ бэкенд
- 5. Отладка багов на стороне клиента
- Связанное

План обучения React Middle+

План обучения React Middle+

Цель: переход с Junior+ на Middle+ React-разработчика. Срок: 10 недель.
Нагрузка: 10-15 часов в неделю.

Рoadmap

Неделя 1-2: TypeScript продвинутый

Цель: перестать использовать any и уверенно владеть системой типов.

День	Тема	Практика
1-2	Generics: constraints, несколько параметров, хуки, компоненты	Написать дженерик-хук <code>useLocalStorage<T></code>
3-4	Utility Types: Partial, Pick, Omit, Record, ReturnType, Parameters	Переписать проект без any
5-6	Discriminated Unions: type narrowing, exhaustive check	Заменить <code>status: string</code> на <code>discriminated union</code>
7-8	Conditional Types + infer: ComponentProps, AsyncReturnType, DeepPartial	Написать тип <code>PropsOf<T></code>
9-10	Template Literal Types, Mapped Types, Branded Types	Типизировать API-роуты
11-12	Type Guards (is, asserts), unknown vs any, keyof/typeof	Рефакторинг кода с проверками типов
13-14	Повторение + практический проект	Полная типизация небольшого приложения

Ресурс: TypeScript продвинутый

Неделя 3-4: React рендеринг и производительность

Цель: понимать, КОГДА и ПОЧЕМУ компонент ререндерится.

День	Тема	Практика
1-2	Reconciliation, Virtual DOM, key	Открыть Profiler, посмотреть на своём проекте
3-4	Fiber: две фазы, приоритеты, почему нельзя side-effect в render	Найти и исправить side-effect'ы в своих компонентах
5-6	React.memo, useMemo, useCallback	Добавить мемо ТОЛЬКО где реально нужно
7-8	Законы оптимизации: двигать состояние вниз, children-паттерн	Рефакторинг медленной страницы
9-10	useTransition, useDeferredValue	Добавить отзывчивый поиск/фильтрацию
11-14	Профилирование: React DevTools Profiler, Performance API	Профилировать реальный проект, найти узкие места

Ресурс: React рендеринг и производительность

Неделя 5-6: Управление состоянием

Цель: разделить серверное и клиентское состояние, выбрать правильный инструмент.

День	Тема	Практика
1-3	Серверное vs клиентское состояние	Разобрать свой проект: что где хранится?
4-6	TanStack Query: useQuery, кэш, ключи, staleTime/gcTime	Перенести fetch на useQuery
7-8	TanStack Query: useMutation, инвалидация, оптимистичные обновления	Добавить мутации с оптимистичным UI
9-10	Zustand: store, селекторы, persist	Вынести UI-состояние в Zustand
11-12	URL как состояние: useSearchParams	Перенести фильтры/пагинацию в URL
13-14	Связка Query + Zustand + Router	Полный рефакторинг страницы каталога

Ресурс: Управление состоянием

Неделя 7: Архитектура компонентов

Цель: проектировать API компонентов, а не «накидывать пропсы».

День	Тема	Практика
1-2	Composition over Configuration	Переписать Modal/Form на композицию
3-4	Compound Components	Сделать свои Tabs/Accordion
5-6	Headless Components, Slot Pattern, Custom Hooks	Вынести логику в хуки
7	Server Components (React 19)	Переписать страницу на Server Component

Ресурс: Архитектура React компонентов

Неделя 8: Тестирование

Цель: писать поддерживаемые тесты на трёх уровнях.

День	Тема	Практика
1-2	Unit-тесты: Vitest, чистые функции, хуки	Покрыть utils и кастомные хуки
3-5	Интеграционные: React Testing Library, userEvent	Протестировать 3-5 ключевых компонентов
6-7	MSW: моки API	Настроить MSW для проекта
8	E2E: Playwright (один критический сценарий)	Тест флоу покупки/регистрации

Ресурс: Тестирование React

- Soft skills собеседование — поведенческие вопросы, STAR, конфликты

Неделя 9-10: Проект и собеседование

Цель: закрепить всё на практике и подготовиться к собеседованию.

День	Тема
1-7	Проект: Event-платформа с нуля (или рефакторинг существующего). Next.js 14 + TypeScript strict + TanStack Query + Zustand + Vitest + Playwright
8-9	Code review своего проекта: проверить по всем чек-листам из базы
10-14	Подготовка к собеседованию: разобрать Вопросы на собеседовании FullStack, мок-собеседование

Сводка компетенций Middle+

Знать (теория)

- [] Как работает Reconciliation и Fiber
- [] Разница между серверным и клиентским состоянием
- [] Когда useMemo/useCallback нужны, а когда вредны

- [] Принципы композиции (Composition, Compound, Headless)
- [] Пирамида тестирования

Уметь (практика)

- [] Написать дженерик-компонент и дженерик-хук
- [] Спроектировать API компонента с нуля
- [] Найти узкое место через React Profiler и исправить
- [] Написать интеграционный тест с RTL + MSW
- [] Объяснить архитектурное решение («почему выбрал X, а не Y»)

Не делать

- [] any в TypeScript (без крайней необходимости)
- [] useEffect для производного состояния
- [] useMemo / useCallback на каждый чих
- [] index как key
- [] Класть серверные данные в Zustand/Redux

Как отслеживать прогресс

1. **GitHub:** 1-2 коммита в день минимум
2. **Код-ревью:** раз в неделю просматривать свой код недельной давности
3. **Пет-проект:** event-платформа, один репозиторий, последовательное улучшение
4. **Дневник:** короткая запись после каждой сессии — что узнал, что непонятно

Связанное

- TypeScript продвинутый
- React рендеринг и производительность
- Управление состоянием
- Архитектура React компонентов
- Тестирование React
- **Ресурс:** Вопросы на собеседовании FullStack
- Интернационализация и локализация i18n — react-intl, i18next, RTL

TypeScript продвинутый

TypeScript продвинутый

Уровень Middle требует не просто «типизировать пропсы», а уверенно владеть системой типов TypeScript. Эта страница — углублённый разбор

ключевых концепций с пояснениями «зачем» и «когда», а не только «как».

1. Generics (дженерики)

Зачем

Дженерики позволяют писать **переиспользуемый код**, который работает с разными типами, не теряя типовую безопасность. Без дженериков приходится либо дублировать код для каждого типа, либо использовать `any` (теряя проверки).

Проблема без дженериков:

```
// any — теряем информацию о типе
function getFirst(arr: any[]): any {
    return arr[0];
}
const num = getFirst([1, 2, 3]); // тип any — бесполезно
num.toUpperCase();                // ошибки нет, но будет runtime crash
```

Решение с дженериком:

```
// Дженерик сохраняет конкретный тип
function getFirst<T>(arr: T[]): T {
    return arr[0];
}
const num = getFirst([1, 2, 3]); // тип number
num.toUpperCase();                // Ошибка компиляции — и отлично!
```

Как читать дженерики (не бойся синтаксиса)

Когда видишь запись `type Something<T> = ...` — не пугайся. Читай её **буквально**:

«Создай шаблон типа с именем *Something*, в котором *T* — это место, куда потом подставится конкретный тип.»

```
// Сначала это шаблон:
type Box<T> = {
    value: T
}

// А потом мы его используем:
type NumberBox = Box<number>
// После подстановки получается:
// type NumberBox = { value: number }

type UserBox = Box<User>
// После подстановки:
// type UserBox = { value: User }
```

Тот же принцип с функциями:

```
// «Создай шаблон функции getFirst, в котором T – место для конкретного
типа»
function getFirst<T>(arr: T[]): T {
    return arr[0];
}

// getFirst<number> → (arr: number[]) => number
// getFirst<string> → (arr: string[]) => string
```

Мнемоника: <T> читай как <ТипСюдаВставишь>. Это не магия, это просто placeholder.

Ограничения дженериков (constraints)

Часто мы хотим, чтобы дженерик был не «любым», а имел определённые свойства:

```
// T обязан иметь поле length
function getLength<T extends { length: number }>(item: T): number {
    return item.length;
}

getLength("hello");           // 5 (строка имеет length)
getLength([1, 2, 3]);         // 3 (массив имеет length)
getLength(42);                 // Ошибка: у number нет length
```

Реальный пример — дженерик для API-ответа:

```
interface ApiResponse<T> {
    data: T;
    status: number;
    error?: string;
}

// Использование с конкретным типом
interface User { id: number; name: string; }

async function fetchUser(id: number): Promise<ApiResponse<User>> {
    const res = await fetch(`/api/users/${id}`);
    return res.json();
}

const result = await fetchUser(1);
// result.data.name – TypeScript знает, что это string
```

Дженерик-хук в React

```
// Универсальный хук для любого GET-запроса
function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<Error | null>(null);

  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then((json: T) => {
        setData(json);
        setLoading(false);
      })
      .catch(err => {
        setError(err);
        setLoading(false);
      });
  }, [url]);

  return { data, loading, error } as const;
}

// Использование:
interface Post { id: number; title: string; body: string; }
const { data, loading } = useFetch<Post[]>('/api/posts');
// data[0].title – TypeScript знает тип
```

Дженерик-компонент

```
interface ListProps<T> {
  items: T[];
  renderItem: (item: T, index: number) => React.ReactNode;
  keyExtractor: (item: T) => string | number;
}

function List<T>({ items, renderItem, keyExtractor }: ListProps<T>) {
  return (
    <ul>
      {items.map((item, i) => (
        <li key={keyExtractor(item)}>{renderItem(item, i)}</li>
      ))}
    </ul>
  );
}

// Использование:
interface Product { id: number; name: string; price: number; }

<List<Product>
  items={products}
  renderItem={(product) => `${product.name} – ${product.price}₽`}
  keyExtractor={(p) => p.id}
/>
```

Несколько дженерик-параметров

```
// Типичная ситуация: маппинг одного типа в другой
function map<T, U>(arr: T[], fn: (item: T, index: number) => U): U[] {
  return arr.map(fn);
}

const nums = [1, 2, 3];
const strings = map(nums, n => `Число ${n}`); // string[]
```

2. Utility Types — швейцарский нож

TypeScript поставляет встроенные типы-утилиты, которые трансформируют существующие типы. Знать их — обязательно.

Partial<T> — все поля опциональны

```
interface User { id: number; name: string; email: string; }

// Частичное обновление: передаём только то, что меняем
function updateUser(id: number, changes: Partial<User>) {
  // changes может содержать любое подмножество полей
}

updateUser(1, { name: "Новое имя" });      //
updateUser(1, { email: "a@b.com" });      //
```

Pick<T, K> — выбрать только нужные поля

```
interface User { id: number; name: string; email: string; password: string; }

// Для ответа API убираем пароль
type SafeUser = Pick<User, 'id' | 'name' | 'email'>;
// { id: number; name: string; email: string; }

// Альтернативный подход через Omit:
type SafeUserAlt = Omit<User, 'password'>;
```

Record<K, V> — словарь с фиксированным типом значений

```
// Словарь ролей пользователя
type Role = 'admin' | 'editor' | 'viewer';
type Permissions = Record<Role, string[]>;
// { admin: string[]; editor: string[]; viewer: string[] }

const permissions: Permissions = {
  admin: ['read', 'write', 'delete'],
  editor: ['read', 'write'],
  viewer: ['read'],
};
```

Extract<T, U> / Exclude<T, U> — работа с union

```
type Event = 'click' | 'focus' | 'blur' | 'keydown' | 'keyup';

// Извлечь только клавиатурные события
type KeyboardEvent = Extract<Event, `key${string}`>;
// 'keydown' | 'keyup'

// Исключить клавиатурные
type MouseEvent = Exclude<Event, `key${string}`>;
// 'click' | 'focus' | 'blur'
```

ReturnType<T> и Parameters<T> — мета-информация о функциях

```
function createUser(name: string, age: number) {
    return { name, age, createdAt: new Date() };
}

type UserFromFn = ReturnType<typeof createUser>;
// { name: string; age: number; createdAt: Date }

type CreateUserArgs = Parameters<typeof createUser>;
// [name: string, age: number]

// Полезно, когда тип функции определён где-то в библиотеке
type onClickParams = Parameters<React.MouseEventHandler>;
// [event: MouseEvent<Element, MouseEvent>]
```

Awaited<T> — развернуть Promise

```
async function fetchData(): Promise<{ id: number }> {
    return { id: 1 };
}

type Data = Awaited<ReturnType<typeof fetchData>>;
// { id: number } — без Promise-обёртки
```

Композиция utility types

```
interface Config {
    apiUrl: string;
    timeout: number;
    retries: number;
    debug: boolean;
}

// Частичная конфигурация: все поля опциональны, кроме apiUrl
type PartialConfig = Partial<Omit<Config, 'apiUrl'>> & Pick<Config, 'apiUrl'>;

const config: PartialConfig = {
    apiUrl: 'https://api.example.com',
    timeout: 5000,
    // retries и debug — опциональны
};
```

3. Discriminated Unions — вместо status: string

Проблема: тип `{ status: string; data?: Data; error?: Error }` не говорит TypeScript'у, что `data` есть только когда `status === 'success'`.

Решение — discriminated union (размеченное объединение):


```
// . Junior: нет связи между status и остальными полями
type BadState = {
  status: 'idle' | 'loading' | 'success' | 'error';
  data?: Data;
  error?: Error;
};

// Middle: TypeScript понимает связь
type State =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: Data }
  | { status: 'error'; error: Error };
```

Почему это важно: type narrowing (сужение типа)

Как читать discriminated union: `type State = A | B | C` читай буквально: «State — это **ЛИБО** объект A, **ЛИБО** объект B, **ЛИБО** объект C». Вертикальная черта `|` = «или». TypeScript смотрит на поле-дискриминатор (`status`) и понимает, какой именно вариант сейчас активен.

```
function render(state: State) {
  switch (state.status) {
    case 'idle':
      return <p>Ожидание...</p>;
    case 'loading':
      return <Spinner />;
    case 'success':
      // TypeScript ЗНАЕТ, что здесь есть state.data
      return <DataView data={state.data} />;
    case 'error':
      // TypeScript ЗНАЕТ, что здесь есть state.error
      return <ErrorView message={state.error.message} />;
    default:
      // Исчерпывающая проверка: если добавили новый status —
      // TypeScript выдаст ошибку, что не все варианты обработаны
      const _exhaustive: never = state;
      return null;
  }
}
```

Реальный пример — хук useQuery с discriminated union

```
type QueryState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: Error }
  | { status: 'refetching'; data: T }; // есть старые данные, но идёт
обновление

function useQuery<T>(url: string): QueryState<T> {
  // ... реализация
}

// Использование в компоненте:
const users = useQuery<User[]>('/api/users');

if (users.status === 'success') {
  users.data.map(u => u.name); // TypeScript знает тип
}
if (users.status === 'refetching') {
  users.data.map(u => u.name); // Тоже знает (data есть!)
}
```

4. Conditional Types и infer

Conditional types — это «if/else» на уровне типов. Ключевое слово `infer` позволяет «вытащить» часть типа.

Базовый синтаксис

Как читать conditional type: $T \text{ extends } X ? Y : Z$ читай как тернарный оператор, только на уровне типов: «если T является подтипом X , то верни тип Y , иначе — тип Z ». Ничего сверхъестественного.

```
type IsString<T> = T extends string ? 'да' : 'нет';

type A = IsString<'hello'>; // 'да'
type B = IsString<42>;     // 'нет'
```

`infer` — вытаскиваем вложенный тип

Как читать `infer`: $T \text{ extends } \text{SomePattern}<\text{infer } U> ? U : \text{never}$ читай как: «если T соответствует шаблону, **вытащи** внутренний тип и **назови** его U , затем верни U , иначе `never`». `infer` = «извлеки и дай имя».

```
// Извлечь тип элемента массива
type ArrayElement<T> = T extends (infer U)[] ? U : never;

type E1 = ArrayElement<string[]>; // string
type E2 = ArrayElement<number[]>; // number
type E3 = ArrayElement<{ id: number }[]>; // { id: number }
type E4 = ArrayElement<string>; // never (не массив)
```

Практические применения infer

1. Тип пропсов React-компонента:

```
type ComponentProps<T> = T extends React.ComponentType<infer P> ? P : never;

import { Button } from './Button'; // допустим, Button принимает
{ variant: 'primary' | 'secondary' }
type ButtonProps = ComponentProps<typeof Button>;
// { variant: 'primary' | 'secondary' }
```

2. Тип возврата асинхронной функции:

```
type AsyncReturnType<T> = T extends (...args: any[]) => Promise<infer R> ? R : never;

async function getUser() { return { id: 1, name: 'Alice' }; }
type User = AsyncReturnType<typeof getUser>; // { id: number; name: string }
```

3. Глубокий Partial (рекурсивный):

```
type DeepPartial<T> = T extends object
  ? { [K in keyof T]?: DeepPartial<T[K]> }
  : T;

interface Config { server: { host: string; port: number }; debug: boolean; }
type PartialConfig = DeepPartial<Config>;
// Все поля на всех уровнях – опциональны
```

5. Template Literal Types

Позволяют конструировать строковые типы на основе других типов — как template literals в JavaScript, но на уровне типов.

Базовый синтаксис

```
type Greeting = `Hello, ${string}!`;
const a: Greeting = 'Hello, World!'; //
const b: Greeting = 'Hi!';           //
```

Capitalize, Uncapitalize, Uppercase, Lowercase

```
type EventName = 'click' | 'focus' | 'blur';
type Handler = `on${Capitalize<EventName>}`;
// 'onClick' | 'onFocus' | 'onBlur'
```

Реальный пример — типизированный роутинг

```
type Route = 'users' | 'posts' | 'comments';
type ApiRoute = `/${Route}`;
// '/api/users' | '/api/posts' | '/api/comments'

type RouteWithId = `/${Route}/${number}`;
// '/api/users/1' | '/api/posts/42' | ...
```

CSS-in-JS с типобезопасностью

```
type Spacing = 0 | 4 | 8 | 12 | 16 | 20 | 24;
type Margin = `m-${Spacing}`; // 'm-0' | 'm-4' | 'm-8' | ...
type Padding = `p-${Spacing}`;

type SpacingClass = Margin | Padding;
// Всего 12 валидных значений вместо string
```

Извлечение частей строкового литерала через infer

```
// Извлечь ID из строки '/users/123'
type ExtractId<T> = T extends `${string}/${infer Id}` ? Id : never;

type Id = ExtractId<'/users/123'>; // '123'
type Id2 = ExtractId<'/posts/456/comments/789'>; // '789' (последний
сегмент)
```

6. Mapped Types — трансформация типов полей

Как читать mapped type: $\{ [K \text{ in } \textit{keyof } T]: T[K] \}$ читай буквально: «для **каждого** ключа K в типе T — создай поле с таким же типом, как у $T[K]$ ». Это цикл *for...in*, только на уровне типов. Меняй $T[K]$ на что угодно, чтобы трансформировать тип каждого поля.

```

interface User {
    name: string;
    age: number;
    email: string;
}

// Сделать все поля readonly
type ReadonlyUser = { readonly [K in keyof User]: User[K] };
// То же самое: type ReadonlyUser = Readonly<User>;

// Сделать все поля nullable
type Nullable<T> = { [K in keyof T]: T[K] | null };
type NullableUser = Nullable<User>;
// { name: string | null; age: number | null; email: string | null; }

// Поменять типы через as (key remapping)
type Getters<T> = {
    [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};
type UserGetters = Getters<User>;
// { getName: () => string; getAge: () => number; getEmail: () =>
string; }

```

7. as const и satisfies

as const — точные литеральные типы

```

// Без as const — тип string[]
const roles = ['admin', 'editor', 'viewer'];
// С as const — точный кортеж
const rolesConst = ['admin', 'editor', 'viewer'] as const;
// тип: readonly ['admin', 'editor', 'viewer']

// Полезно для discriminated unions:
const STATUSES = ['idle', 'loading', 'success', 'error'] as const;
type Status = (typeof STATUSES)[number]; // 'idle' | 'loading' |
'success' | 'error'

```

satisfies — проверка без сужения типа (TS 4.9+)

```

// satisfies проверяет, что значение соответствует типу, но сохраняет
точный тип
const config = {
    api: 'https://api.example.com',
    timeout: 5000,
    retries: 3,
} satisfies Record<string, string | number>;

// config.api — тип 'https://api.example.com' (точный литерал, не string)
// config.retries — тип 3 (числовой литерал, не number)

```

8. Распространённые паттерны в React

Типизация useReducer

```
type Action =
  | { type: 'INCREMENT' }
  | { type: 'DECREMENT' }
  | { type: 'SET'; payload: number };

function reducer(state: number, action: Action): number {
  switch (action.type) {
    case 'INCREMENT': return state + 1;
    case 'DECREMENT': return state - 1;
    case 'SET':       return action.payload; // TypeScript знает,
    что payload есть
  }
}
```

Типизация useContext

```
interface ThemeContextType {
  theme: 'light' | 'dark';
  toggleTheme: () => void;
}

const ThemeContext = createContext<ThemeContextType | null>(null);

function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme must be used within
ThemeProvider');
  return ctx;
}
```

Типизация событий

```
// Не надо any или угадывать тип
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  console.log(e.target.value); // string
};

const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  const formData = new FormData(e.currentTarget);
};

const handleKeyDown = (e: React.KeyboardEvent<HTMLInputElement>) => {
  if (e.key === 'Enter') submit();
};
```

9. Практические задания

Неделя 1: дженерики и utility types

- [] Найти в проекте any и заменить на строгие типы
- [] Написать дженерик-хук `useLocalStorage<T>(key: string, initial: T)`
- [] Создать тип `Nullable<T>` и `DeepReadonly<T>`

Неделя 2: discriminated unions и conditional types

- [] Заменить `status: string` на discriminated union в существующем коде
- [] Типизировать `useReducer` с экшенами
- [] Написать тип `PropsOf<T>` для извлечения пропсов компонента

Проверка понимания (ответь себе)

1. Когда использовать `extends` в дженерике?
2. Чем `Pick` отличается от `Omit`? Когда что выбрать?
3. Почему discriminated union лучше, чем `status: string` + опциональные поля?
4. Что делает `infer` и где он реально нужен?
5. Чем `as const` отличается от обычного `const`?

10. Type Guards — пользовательская проверка типов

TypeScript сам сужает типы внутри `if (typeof x === 'string')`, но для сложных объектов этого недостаточно. Type Guard — функция, возвращающая `x is Type`.

is — пользовательский type guard

```
interface User { id: number; name: string; email: string; }
interface Admin { id: number; name: string; role: 'admin'; permissions: string[]; }

// Type guard: говорит TypeScript'у, КАКОЙ именно тип
function isAdmin(person: User | Admin): person is Admin {
    return 'role' in person && person.role === 'admin';
}

function handlePerson(person: User | Admin) {
    if (isAdmin(person)) {
        // TypeScript знает: person = Admin
        console.log(person.permissions);
    } else {
        // TypeScript знает: person = User
        console.log(person.email);
    }
}
```

Type guard для API-ответов

```
// Частый кейс: проверка успешного ответа
type ApiResult<T> =
  | { success: true; data: T }
  | { success: false; error: string };

function isSuccess<T>(result: ApiResult<T>): result is { success: true; data: T } {
  return result.success === true;
}

// Использование:
const res = await fetchUser();
if (isSuccess(res)) {
  console.log(res.data.name); // без as / !
}
```

asserts — assertion function (TS 3.7+)

```
// Вместо return true/false – бросает ошибку, если условие не выполнено
function assert(condition: unknown, message: string): asserts condition {
  if (!condition) throw new Error(message);
}

function processValue(value: unknown) {
  assert(typeof value === 'string', 'Expected string');
  // После asserts TypeScript знает: value is string
  console.log(value.toUpperCase());
}
```

11. unknown vs any — критическая разница

```
// any: отключает ВСЕ проверки
let x: any = 'hello';
x.toUpperCase(); // ошибки нет, даже если x – не строка в runtime
x.nonExistent(); // ошибки нет – any разрешает всё

// unknown: «я не знаю тип, но заставлю тебя проверить»
let y: unknown = 'hello';
y.toUpperCase(); // Ошибка: Object is of type 'unknown'
// Нужно СНАЧАЛА сузить тип:
if (typeof y === 'string') {
  y.toUpperCase(); // теперь TypeScript знает
}
```

Правило: unknown — для входящих данных (API, localStorage, params). any — только для постепенной миграции с JS на TS.


```
// Реальный пример: парсинг JSON (возвращает any!)
const raw = localStorage.getItem('user');
const user: unknown = raw ? JSON.parse(raw) : null;
// unknown заставляет проверить перед использованием:
if (user && typeof user === 'object' && 'name' in user) { ... }
```

12. keyof и typeof — операторы типов

```
// keyof: получить union ключей объекта
interface User { id: number; name: string; email: string; }
type UserKey = keyof User; // 'id' | 'name' | 'email'

// Практика: типобезопасный getter
function get<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const user: User = { id: 1, name: 'Alice', email: 'a@b.com' };
get(user, 'name'); // string
get(user, 'xxx'); // Ошибка: 'xxx' не ключ User
```

```
// typeof: получить тип ЗНАЧЕНИЯ (не путать с runtime typeof!)
const config = { apiUrl: 'https://api.example.com', timeout: 5000 };
type Config = typeof config; // { apiUrl: string; timeout: number; }

// Часто вместе с ReturnType:
function createStore() { return { getState, dispatch, subscribe }; }
type Store = ReturnType<typeof createStore>;
```

13. Branded Types — типобезопасные идентификаторы

Проблема: `userId: number` и `postId: number` — TypeScript считает их одинаковыми.

```
function getUser(id: number) { ... }
function getPost(id: number) { ... }
getUser(postId); // Ошибки нет, хотя передали не тот ID!
```

Решение — брендинг:

```
type Brand<T, B> = T & { __brand: B };

type UserId = Brand<number, 'UserId'>;
type PostId = Brand<number, 'PostId'>;

function createUserId(id: number): UserId { return id as UserId; }
function createPostId(id: number): PostId { return id as PostId; }

function getUser(id: UserId) { ... }
function getPost(id: PostId) { ... }

getUser(createPostId(5)); // Ошибка: UserId !== PostId
```

14. Function Overloads — разные сигнатуры

```
// Разное поведение в зависимости от аргументов
function formatDate(date: Date): string;
function formatDate(date: Date, locale: string): string;
function formatDate(date: Date, format: 'short' | 'long'): string;
// Реализация — одна, принимает всё:
function formatDate(date: Date, arg?: string): string {
    if (arg === 'short') return date.toLocaleDateString();
    if (arg === 'long') return date.toLocaleString();
    return date.toISOString();
}

formatDate(new Date());           // string
formatDate(new Date(), 'ru-RU');  // string
formatDate(new Date(), 'short');  // string
formatDate(new Date(), 'invalid'); // Ошибка
```

15. const Type Parameters (TS 5.0+)

```
// Без const: теряем точные литералы
function useState<T>(initial: T): [T, (v: T) => void] { ... }
const [name, setName] = useState('Alice');
// name: string (не литерал)

// С const type parameter:
function useState<const T>(initial: T): [T, (v: T) => void] { ... }
const [name, setName] = useState('Alice');
// name: 'Alice' (точный литерал)

// Особенно полезно с массивами:
declare function useList<const T extends readonly string[]>(items: T):
T[number];
const status = useList(['idle', 'loading', 'success']);
// status: 'idle' | 'loading' | 'success' — без as const!
```

16. Enums vs as const — что выбрать

```
// Enum: генерирует JS-код, не всегда tree-shakeable
enum Status { Idle, Loading, Success, Error }

// Предпочтительный вариант: const object + тип
const STATUS = {
  Idle: 'idle',
  Loading: 'loading',
  Success: 'success',
  Error: 'error',
} as const;

type Status = (typeof STATUS)[keyof typeof STATUS];
// 'idle' | 'loading' | 'success' | 'error'
```

Почему as const лучше enum:

- Не генерирует лишний код (tree-shakeable)
- Предсказуемое работает с --isolatedModules
- Проще использовать как runtime-значение И тип одновременно

17. Module Augmentation — расширение сторонних типов

```
// Расширяем Window
declare global {
  interface Window {
    __APP_VERSION__: string;
    analytics: { track: (event: string) => void };
  }
}
window.__APP_VERSION__; // TypeScript знает

// Расширяем Express Request (для бэкенда)
declare namespace Express {
  interface Request {
    user?: { id: number; role: string };
  }
}
```

Шпаргалка: самые частые utility types

Тип	Что делает	Пример
Partial<T>	Все поля опциональны	Partial<User>
Required<T>	Все обязательны	Required<Config>
Pick<T, K>	Выбрать поля	Pick<User, 'id' \ 'name'>
Omit<T, K>	Исключить поля	Omit<User, 'password'>
Record<K, V>	Словарь	Record<string, User>
ReturnType<T>	Тип возврата функции	ReturnType<typeof fn>
Parameters<T>	Кортеж параметров	Parameters<typeof fn>

Связанное

- План обучения React Middle
- React рендеринг и производительность
- Управление состоянием
- Архитектура React компонентов

Event Loop: макротаски и микротаски

Event Loop: макротаски, микротаски и асинхронность

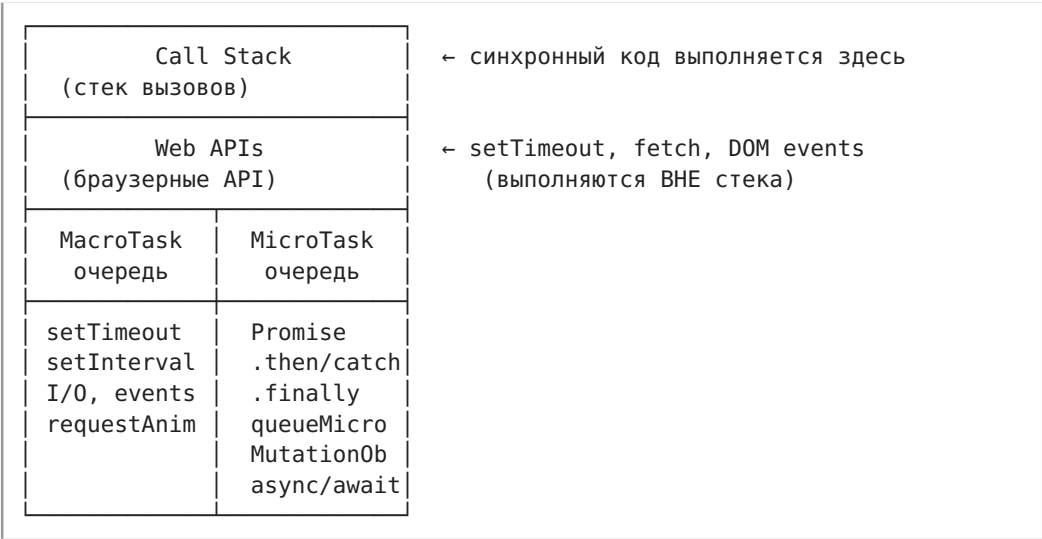
Event Loop — механизм, который позволяет JavaScript (однопоточному!) выполнять асинхронные операции: таймеры, промисы, запросы к серверу, события.

Почему это важно

Без понимания Event Loop невозможно предсказать порядок выполнения кода. На собеседовании Middle+/Senior это один из первых вопросов — проверяют фундамент.

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
// Что выведет и почему?
```

Как устроен Event Loop



Алгоритм (один тик Event Loop)

1. Взять самую старую **макротаску** из очереди → выполнить
2. Выполнить **ВСЕ** микротаски (включая те, что добавились во время выполнения)
3. Если есть свободное время — выполнить requestAnimationFrame
4. Обновить DOM (рендеринг), если нужно
5. Перейти к шагу 1

Макротаски (MacroTasks)

Источники:

- setTimeout(fn, delay)
- setInterval(fn, delay)
- setImmediate() (Node.js)
- I/O операции (fetch, чтение файлов)
- UI-события (click, keydown)
- requestAnimationFrame() (особая очередь перед рендерингом)

Правило: на каждый тик Event Loop — **одна** макротаска.

Микротаски (MicroTasks)

Источники:

- Promise.then(), .catch(), .finally()
- queueMicrotask(fn)
- async/await (после await — код становится микротаской)
- MutationObserver

Правило: микротаски выполняются **до тех пор, пока очередь не опустеет**. Если микротаска добавляет новую микротаску — она выполнится в этом же тике.

Примеры с разбором

Пример 1: классика собеседования

Как читать цепочку `.then()`: читай `.then(() => ...)` как «когда выполнится — положи в очередь микротаску, который сделает вот это». Каждый `.then()` не выполняется сразу — он становится в очередь микротасок и ждёт, пока Call Stack опустеет. Мнемоника: «then — не сейчас, а когда стек свободен».

```
console.log('1');

setTimeout(() => console.log('2'), 0);

Promise.resolve()
  .then(() => console.log('3'))
  .then(() => console.log('4'));

console.log('5');
```

Разбор по шагам:

- | | | |
|--|---------------------|----------|
| 1. console.log('1') | → Call Stack | → "1" |
| 2. setTimeout(...) | → MacroTask очередь | → (ждёт) |
| 3. Promise.resolve() | → MicroTask очередь | → (ждёт) |
| 4. console.log('5') | → Call Stack | → "5" |
| 5. Call Stack пуст → выполняем BCE микротаски: | | |
| - .then(() => log('3')) → "3" | | |
| - .then(() => log('4')) → "4" | | |
| 6. Микротаски пусты → выполняем макротаску: | | |
| - setTimeout → "2" | | |

Вывод: 1, 5, 3, 4, 2

Пример 2: микротаска внутри микротаски

```
Promise.resolve()
  .then(() => {
    console.log('A');
    Promise.resolve().then(() => console.log('B'));
  })
  .then(() => console.log('C'));
```

Разбор:

Микротаски до опустошения:

```
.then-1 → "A" → добавляет .then-B в очередь  
.then-B → "B" (добавлен во время этого же тика!)  
.then-2 → "C"
```

Вывод: A, B, C

Пример 3: `async/await` — синтаксический сахар над `Promise`

Как читать `await`: читай `await bar()` как «выполни `bar()` прямо сейчас, а всё что после меня — оберни в `.then()` и положи в очередь микротасок». Код ДО `await` — синхронный, код ПОСЛЕ — микротаска. Мнемоника: «`await` режет функцию пополам: верх — сейчас, низ — потом».

```
async function foo() {  
  console.log('X');  
  await bar();  
  console.log('Y'); // ← микротаска после await  
}  
  
async function bar() {  
  console.log('Z');  
}  
  
console.log('1');  
foo();  
console.log('2');
```

Разбор:

1. '1' → синхронно
2. `foo()` → 'X' (синхронно) → `await bar()` → 'Z' (синхронно)
Код ПОСЛЕ `await` = микротаска
3. '2' → синхронно
4. Микротаски → 'Y'

Вывод: 1, X, Z, 2, Y

Важно: `await` НЕ блокирует Call Stack. Функция «засыпает» на `await`, управление возвращается Event Loop.

Пример 4: `setTimeout 0` — НЕ ноль

```
console.time('timer');

setTimeout(() => {
  console.timeEnd('timer');
}, 0);

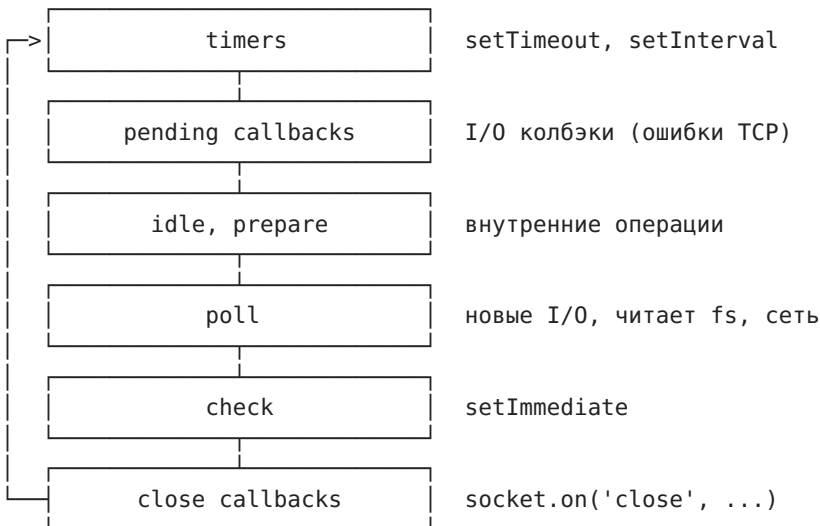
// Заблокируем стек на 100 мс
const start = Date.now();
while (Date.now() - start < 100) {}
```

Вывод: timer: ~104ms

`setTimeout(fn, 0)` означает «поставить в очередь макротаск прямо сейчас», но выполнится он только когда стек опустеет + все микротаски. Минимальная задержка в браузере — 4 мс для вложенных таймеров.

Node.js: отличия

В Node.js Event Loop устроен сложнее — есть фазы:



`process.nextTick()` — отдельная очередь, выполняется **перед** микротасками в Node.js.

Как читать `process.nextTick()`: читай как «положи в очередь, которая важнее всех — даже важнее промисов». Это не микротаска и не макротаска, а третья, самая приоритетная очередь. Мнемоника: «nextTick — вне очереди, всегда первый».


```
// Node.js:
Promise.resolve().then(() => console.log('promise'));
process.nextTick(() => console.log('nextTick'));
// → nextTick, promise (nextTick приоритетнее!)
```

requestAnimationFrame (браузер)

Особая очередь — выполняется **перед рендерингом**, 60 раз в секунду (16.6 мс на кадр):

```
setTimeout(() => console.log('timeout'), 0);
requestAnimationFrame(() => console.log('rAF'));

// Порядок: rAF → рендеринг → timeout (в следующем тике)
```

Типичные вопросы на собеседовании

1. Порядок вывода

```
setTimeout(() => console.log('timeout'), 0);
Promise.resolve().then(() => console.log('promise'));
console.log('sync');
```

Ответ: sync, promise, timeout

- Синхронный код → микротаски (promise) → макротаски (timeout)

2. Что быстрее: setTimeout 0 или Promise?

Promise (микротаска) — выполнится в этом же тике, до макротасок и до рендеринга. **setTimeout 0** — минимум 4 мс + ждёт следующего тика.

3. Может ли микротаска заблокировать рендеринг?

Да. Пока очередь микротасок не опустеет, рендеринг не произойдёт. Если микротаска рекурсивно добавляет себя — страница «зависает».

```
function blockRendering() {
  Promise.resolve().then(() => {
    blockRendering(); // ← бесконечные микротаски → рендеринга не будет
  });
}
```

4. Где использовать queueMicrotask?

Когда нужно выполнить код **асинхронно, но ДО следующего рендеринга**:

```
// Обновили DOM → нужно измерить до следующего кадра
element.textContent = 'new text';
queueMicrotask(() => {
  const height = element.getBoundingClientRect().height;
  // используем height...
});
```

Шпаргалка: порядок выполнения

1. Синхронный код (Call Stack)
2. Микротаски (Promise, queueMicrotask, await)
↓ (VCE до опустошения очереди!)
3. requestAnimationFrame (если браузер готов рендерить)
4. Рендеринг (если нужно)
5. Макротаска (setTimeout, setInterval, I/O, события)
6. → Вернуться к шагу 2

Антипаттерны

- Полагать, что setTimeout(fn, 0) выполнится через 0 мс
- Рекурсивные микротаски без выхода → блокировка Event Loop
- Тяжёлые вычисления в основном потоке → «замораживают» UI
- new Promise с синхронным executor'ом для оборачивания синхронного кода

Связанное

- Алгоритмические задачи (debounce, throttle используют setTimeout)
- React рендеринг и производительность (React использует микротаски для батчинга)
- Web API

Алгоритмические задачи

Алгоритмические задачи

На собеседовании Frontend Middle+ почти всегда просят написать руками одну из этих функций. Это не LeetCode — это реальные паттерны, которые используются в работе.

1. Debounce

Задача: отложить вызов функции до тех пор, пока не пройдет N миллисекунд с последнего вызова.

Где используется: поисковые подсказки (ждём, пока пользователь перестанет печатать), автосохранение, resize.

```
function debounce(fn, delay) {
  let timer = null;

  return function(...args) {
    // Сбрасываем предыдущий таймер
    clearTimeout(timer);
    // Ставим новый
    timer = setTimeout(() => {
      fn.apply(this, args); // сохраняем this и аргументы
    }, delay);
  };
}

// Использование:
const search = debounce((query) => {
  console.log('Ищем:', query);
  fetch(`/api/search?q=${query}`);
}, 300);

input.addEventListener('input', (e) => search(e.target.value));
```

Ключевые моменты:

- `fn.apply(this, args)` — сохраняем контекст `this`
- `clearTimeout` перед `setTimeout` — сбрасываем предыдущий
- Возвращаем НОВУЮ функцию (не вызываем исходную)

Как читать `fn.apply(this, args)`: «вызови функцию `fn` с тем же `this`, что у текущей функции, и передай все аргументы как есть через `args`». `apply` принимает `this` первым аргументом и массив аргументов вторым — в отличие от `call`, где аргументы перечисляются через запятую: `fn.call(this, arg1, arg2)`.

Вариант с немедленным первым вызовом (leading edge):

```
function debounce(fn, delay, leading = false) {
  let timer = null;

  return function(...args) {
    if (leading && !timer) {
      fn.apply(this, args); // немедленный вызов
    }
    clearTimeout(timer);
    timer = setTimeout(() => {
      if (!leading) fn.apply(this, args);
      timer = null;
    }, delay);
  };
}
```

2. Throttle

Задача: вызывать функцию не чаще одного раза в N миллисекунд.

Где используется: scroll, mousemove, ресайз окна.

```
function throttle(fn, delay) {
  let lastCall = 0;

  return function(...args) {
    const now = Date.now();
    if (now - lastCall >= delay) {
      lastCall = now;
      fn.apply(this, args);
    }
  };
}

// Использование:
const onScroll = throttle(() => {
  console.log('Scroll position:', window.scrollY);
}, 100);

window.addEventListener('scroll', onScroll);
```

Вариант с замыкающим вызовом (trailing edge):

```
function throttle(fn, delay) {
  let lastCall = 0;
  let timer = null;

  return function(...args) {
    const now = Date.now();
    const remaining = delay - (now - lastCall);

    if (remaining <= 0) {
      // Можно вызывать сейчас
      lastCall = now;
      fn.apply(this, args);
    } else {
      // Запланировать замыкающий вызов
      clearTimeout(timer);
      timer = setTimeout(() => {
        lastCall = Date.now();
        fn.apply(this, args);
      }, remaining);
    }
  };
}
```

Debounce vs Throttle:

- **Debounce:** «подожди, пока закончатся события» — 1 вызов в конце

серии

- **Throttle:** «вызывай не чаще чем» — регулярные вызовы во время серии

3. Promise Pool (ограничение параллельности)

Задача: выполнить N асинхронных задач, но не более K одновременно.

Где используется: загрузка файлов (макс 3 одновременно), API-запросы с rate limit.

Как читать *Promise.race(executing)* где *executing* — это *Set*: «подожди, пока завершится любая из выполняющихся сейчас задач — как только одна закончилась, запускай следующую». *Promise.race* обычно принимает массив, но *Set* тоже итерируемый. Хитрость в том, что через *.finally(() => executing.delete(p))* задача сама удаляет себя из *Set'a* при завершении, освобождая слот.

```
async function promisePool(tasks, concurrency) {
  const results = [];
  const executing = new Set();

  for (const task of tasks) {
    const promise = Promise.resolve().then(() => task());
    results.push(promise);
    executing.add(promise);

    promise.finally(() => executing.delete(promise));

    if (executing.size >= concurrency) {
      // Ждём, пока завершится хотя бы одна
      await Promise.race(executing);
    }
  }

  return Promise.all(results);
}

// Использование:
const urls = ['url1', 'url2', 'url3', 'url4', 'url5'];
const tasks = urls.map(url => () => fetch(url).then(r => r.json()));

const results = await promisePool(tasks, 2);
// Выполняется максимум 2 запроса одновременно
```

Более простая версия (с семафором):

```

async function promisePool(tasks, concurrency) {
  const results = new Array(tasks.length);
  let index = 0;

  async function worker() {
    while (index < tasks.length) {
      const i = index++;
      results[i] = await tasks[i]();
    }
  }

  // Запускаем concurrency «воркеров»
  const workers = Array.from({ length: concurrency }, () => worker());
  await Promise.all(workers);
  return results;
}

```

4. Flatten (разворачивание массива)

Задача: превратить вложенный массив в плоский.

```

// Рекурсивная версия (произвольная глубина)
function flatten(arr, depth = Infinity) {
  const result = [];

  for (const item of arr) {
    if (Array.isArray(item) && depth > 0) {
      result.push(...flatten(item, depth - 1));
    } else {
      result.push(item);
    }
  }

  return result;
}

flatten([1, [2, [3, [4]]]], 1); // [1, 2, [3, [4]]]
flatten([1, [2, [3, [4]]]]);   // [1, 2, 3, 4]

```

Итеративная версия (без рекурсии, не ломает стек):

```
function flatten(arr) {  
  const stack = [...arr];  
  const result = [];  
  
  while (stack.length) {  
    const item = stack.pop();  
    if (Array.isArray(item)) {  
      stack.push(...item); // разворачиваем массив в стек  
    } else {  
      result.push(item);  
    }  
  }  
  
  return result.reverse(); // восстанавливаем порядок  
}
```

5. Deep Equal

Задача: сравнить два объекта/массива на глубокое равенство.

```
function deepEqual(a, b) {
  // Примитивы и одинаковые ссылки
  if (a === b) return true;

  // null / undefined / разные типы
  if (a == null || b == null || typeof a !== typeof b) return false;

  // Даты
  if (a instanceof Date && b instanceof Date) {
    return a.getTime() === b.getTime();
  }

  // Массивы
  if (Array.isArray(a) && Array.isArray(b)) {
    if (a.length !== b.length) return false;
    return a.every((item, i) => deepEqual(item, b[i]));
  }

  // Объекты
  if (typeof a === 'object' && typeof b === 'object') {
    const keysA = Object.keys(a);
    const keysB = Object.keys(b);
    if (keysA.length !== keysB.length) return false;
    return keysA.every(key =>
      Object.prototype.hasOwnProperty.call(b, key) &&
      deepEqual(a[key], b[key])
    );
  }

  return false;
}

deepEqual({ a: 1, b: { c: 2 } }, { a: 1, b: { c: 2 } }); // true
deepEqual({ a: 1 }, { a: 1, b: 2 }); // false
```

Важные детали:

- `a === b` — быстрое сравнение для примитивов и одинаковых ссылок
- Проверка `null` до `typeof` (`typeof null === 'object'`)
- Рекурсия, но на реальных объектах глубина редко >10
- `hasOwnProperty` — игнорируем наследуемые свойства

Как читать `Object.prototype.hasOwnProperty.call(b, key)`: «проверь, что свойство `key` принадлежит САМОМУ объекту `b`, а не унаследовано от прототипа». Пишут через `Object.prototype.hasOwnProperty.call`, а не `b.hasOwnProperty(key)`, потому что `b` мог переопределить метод `hasOwnProperty` или быть созданным через `Object.create(null)` без прототипа.

6. Promise.all (своя реализация)

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    if (!promises.length) return resolve([]);

    const results = new Array(promises.length);
    let completed = 0;

    promises.forEach((promise, index) => {
      Promise.resolve(promise).then(value => {
        results[index] = value;
        completed++;
        if (completed === promises.length) {
          resolve(results);
        }
      }).catch(reject); // Первая ошибка → reject всего
    });
  });
}
```

7. EventEmitter (Observer / PubSub)

```
class EventEmitter {
  constructor() {
    this.events = new Map();
  }

  on(event, callback) {
    if (!this.events.has(event)) {
      this.events.set(event, []);
    }
    this.events.get(event).push(callback);
    return () => this.off(event, callback); // возвращает unsubscribe
  }

  off(event, callback) {
    const callbacks = this.events.get(event);
    if (callbacks) {
      this.events.set(event, callbacks.filter(cb => cb !== callback));
    }
  }

  emit(event, ...args) {
    const callbacks = this.events.get(event);
    if (callbacks) {
      callbacks.forEach(cb => cb(...args));
    }
  }

  once(event, callback) {
    const wrapper = (...args) => {
      callback(...args);
      this.off(event, wrapper);
    };
    this.on(event, wrapper);
  }
}
```

Связанное

- Вопросы на собеседовании FullStack
- Архитектурные паттерны
- TypeScript продвинутый

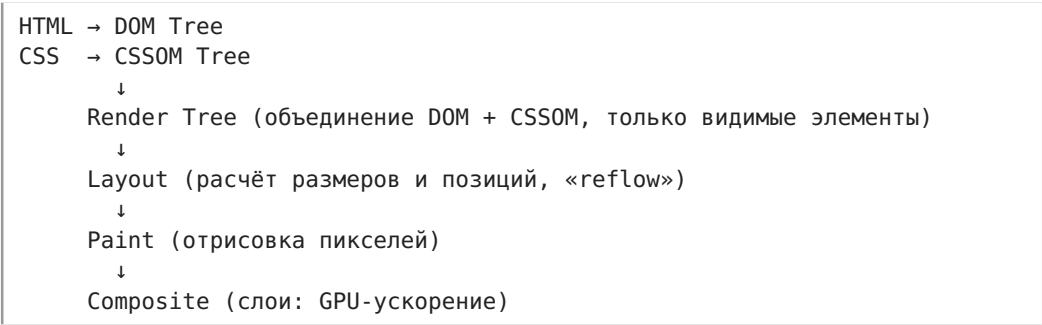
Браузер и HTTP

Браузер и HTTP

Базовые вопросы, которые проверяют понимание платформы, а не фреймворка. Спрашивают на 90% собеседований Middle+.

1. Critical Rendering Path (критический путь рендеринга)

Что делает браузер от получения HTML до первого пикселя:



Что блокирует рендеринг:

- **Синхронный <script> без defer/async** — блокирует DOM-построение, пока не загрузится и выполнится
- **CSS** — блокирует Render Tree, но не DOM (браузер может строить DOM параллельно)
- **<script> с defer** — скачивается параллельно, выполняется ПОСЛЕ DOM, до DOMContentLoaded
- **<script> с async** — скачивается параллельно, выполняется СРАЗУ как загрузился, порядок не гарантирован

Как ускорить First Contentful Paint:

```
<!-- Критический CSS — инлайн в <head> -->
<style>/* только стили для первого экрана */</style>
<!-- Некритический CSS — отложенная загрузка -->
<link rel="preload" href="styles.css" as="style" onload="this.rel='style sheet'">
<!-- Скрипты — defer чтобы не блокировали парсинг -->
<script src="app.js" defer></script>
```

2. Reflow vs Repaint

	Reflow (Layout)	Repaint
Что делает	Пересчитывает размеры и позиции	Перерисовывает пиксели без изменения геометрии
Стоимость	Дорого (каскадный эффект)	Дешевле
Триггеры	offsetWidth, getBoundingClientRect(), изменение DOM, resize, изменение шрифта	color, background, visibility, box-shadow

Layout Trashing — частая ошибка: чтение геометрии → запись стилей → снова чтение в одном кадре:

```
// Layout thrashing: чтение → запись → чтение → запись
elements.forEach(el => {
  const height = el.offsetHeight; // чтение → вызывает reflow
  el.style.height = height + 10 + 'px'; // запись → делает layout dirty
});

// Пакетное чтение, потом пакетная запись
const heights = elements.map(el => el.offsetHeight); // все чтения
elements.forEach((el, i) => {
  el.style.height = heights[i] + 10 + 'px'; // все записи
});

// Или requestAnimationFrame
requestAnimationFrame(() => {
  elements.forEach(el => {
    el.style.transform = 'translateY(10px)'; // composite-only, без
    reflow!
  });
});
```

Свойства, которые не вызывают reflow (composite-only): transform, opacity, filter (на отдельном слое).

3. CORS (Cross-Origin Resource Sharing)

Когда срабатывает: браузер блокирует запрос из <https://mysite.com> к <https://api.other.com>, если сервер не разрешил.

Механика:

- **Простые запросы** (GET, POST с определёнными Content-Type): браузер добавляет Origin и проверяет Access-Control-Allow-Origin в ответе
- **Preflight** (OPTIONS-запрос перед основным): для «сложных» запросов (PUT, DELETE, кастомные заголовки, Content-Type: application/json)

Как читать preflight-диалог: «браузер спрашивает OPTIONS: "Можно мне DELETE с заголовком X-Custom-Header с источника mysite.com?" — сервер отвечает: "Разрешаю DELETE, заголовок X-Custom-Header, кэшируй ответ на 24 часа"». Это невидимый для JS диалог: `fetch()` делает OPTIONS автоматически, и только если сервер разрешил — sends основной запрос.

```
# Preflight-запрос от браузера
OPTIONS /api/users HTTP/1.1
Origin: https://mysite.com
Access-Control-Request-Method: DELETE
Access-Control-Request-Headers: X-Custom-Header
```

```
# Ответ сервера
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://mysite.com
Access-Control-Allow-Methods: GET, POST, DELETE
Access-Control-Allow-Headers: X-Custom-Header
Access-Control-Max-Age: 86400 # кэшировать preflight на 24ч
```

Почему Access-Control-Allow-Origin: * не работает с credentials: 'include'?

Спецификация запрещает: если запрос с куками (credentials), * невалиден. Сервер должен явно указать конкретный origin.

Решение на практике:

```
// Фронт: fetch с credentials (если нужны куки)
fetch('https://api.example.com/data', {
  credentials: 'include', // отправлять куки
});

// Бэк (NestJS):
app.enableCors({
  origin: ['https://mysite.com'], // не *!
  credentials: true,
});
```

4. Cookie / localStorage / sessionStorage

	Cookie	localStorage	sessionStorage
Объём	4 KB	~5 MB	~5 MB
Срок жизни	Устанавливается (Expires/Max-Age)	Пока не удалят	До закрытия вкладки
Доступ	JS (document.cookie) + сервер (автоматически в каждом запросе)	Только JS	Только JS
Область	Domain + Path	Origin	Origin + вкладка

Почему JWT не хранят в localStorage:

- Любой XSS-скрипт получает доступ к localStorage → токен украден
- **Правильно:** Refresh Token в httpOnly Secure SameSite Cookie (недоступен из JS), Access Token в памяти JS-переменной (уязвим только пока открыта вкладка)

Access Token → в памяти (переменная JS, не localStorage)
Refresh Token → httpOnly Secure SameSite=Strict cookie

Атрибуты cookie:

- **HttpOnly** — JS не может прочитать (защита от XSS)
- **Secure** — только по HTTPS
- **SameSite=Strict** — не отправляется при переходе с другого сайта (защита от CSRF)
- **SameSite=Lax** — отправляется только для безопасных методов (GET) при переходе

5. HTTP-кэширование

Заголовки ответа (сервер → браузер):

Cache-Control: max-age=3600 # кэшировать 1 час
Cache-Control: no-cache # кэшировать, но всегда перепроверять (ETag)
Cache-Control: no-store # НЕ кэшировать вообще (платёжные данные)
Cache-Control: public, max-age=86400 # можно кэшировать даже CDN
Cache-Control: private, max-age=60 # только браузеру, не CDN

ETag / If-None-Match:

Первый запрос
GET /api/user HTTP/1.1
Ответ:
HTTP/1.1 200 OK
ETag: "abc123"

Повторный запрос
GET /api/user HTTP/1.1
If-None-Match: "abc123"
Ответ:
HTTP/1.1 304 Not Modified # данные не изменились — не передаём тело

Last-Modified / If-Modified-Since:

То же самое, но по дате (менее точно, чем ETag — секундная точность).

Стратегия для SPA:

Как читать *Cache-Control: public, max-age=31536000, immutable*: «ЭТОТ файл можно кэшировать везде (CDN + браузер) целый год (31536000 секунд), и он НИКОГДА не изменится (*immutable*) — даже не делай повторный 304-запрос». Применяется только к бандлам с хешем в имени (*app.a3f8.js*): хеш поменялся → имя файла поменялось → это другой URL, старый кэш не используется.

```
# index.html – никогда не кэшируем (точка входа, меняется с каждым  
депლოем)  
location = /index.html {  
    add_header Cache-Control "no-cache";  
}  
  
# Бандлы с хешем – кэшируем навсегда (имя файла меняется при изменении  
содержимого)  
location /assets/ {  
    add_header Cache-Control "public, max-age=31536000, immutable";  
}
```

Immutable: говорит браузеру «этот файл никогда не изменится, не делай даже 304-запрос».

6. HTTP/2 и HTTP/3

Что HTTP/2 даёт на практике:

- **Мультиплексирование:** несколько запросов в одном TCP-соединении без head-of-line blocking. Не нужно объединять файлы в бандлы и делать спрайты
- **Приоритеты:** можно указать, какой ресурс важнее (критический CSS > изображения)
- **Server Push:** сервер может отправить ресурс до того, как клиент попросит (устаревает, заменяется Early Hints 103)
- **Сжатие заголовков (HPACK):** экономия на повторяющихся заголовках (Cookie, User-Agent)

Вывод: при HTTP/2 не нужно конкатенировать JS/CSS файлы — лучше много маленьких бандлов (code splitting), которые загружаются параллельно.

HTTP/3 (QUIC):

- Вместо TCP — UDP (быстрее handshake, нет блокировки при потере пакета)
- Миграция соединения (переключение WiFi → 4G без разрыва)
- Встроенное шифрование (TLS 1.3)

Связанное

- HTML и Accessibility (a11y) — семантический HTML и accessibility tree
- CSS и стилизация — влияние CSS на Critical Rendering Path
- Event Loop макротаски микротаски — асинхронность и порядок выполнения
- Вопросы на собеседовании FullStack

- Вопросы на собеседовании FullStack

Web API

Web API

Браузерные API, выходящие за рамки DOM. Проверяют понимание платформы на глубоком уровне.

1. Service Workers и PWA

Service Worker (SW) — JavaScript-файл, который браузер запускает в фоне, отдельно от страницы. Это прокси между приложением и сетью.

Жизненный цикл

```
Register → Install → Waiting → Activate → Fetch/Message
                        ↑
                    skipWaiting()
```

1. **Register:** `navigator.serviceWorker.register('/sw.js')`

Как читать `navigator.serviceWorker.register()`: читай как «установи фоновый агент, который будет перехватывать ВСЕ сетевые запросы с этой страницы». Service Worker — это не рабочий, это прокси-сервер внутри браузера, который живёт отдельно от страницы. Мнемоника: «зарегистрировал SW — поставил шлагбаум между страницей и интернетом».

1. **Install:** событие `install` — кэшируем статику для офлайна
2. **Waiting:** SW ждёт, пока закроются все вкладки со старым SW
3. **Activate:** событие `activate` — чистим старые кэши
4. **Fetch:** перехватывает ВСЕ сетевые запросы страницы


```
// sw.js
const CACHE_NAME = 'app-v1';
const ASSETS = ['/', '/styles.css', '/app.js', '/logo.png'];

// Install: кэшируем статику
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => cache.addAll(ASSETS))
  );
});

// Activate: чистим старые кэши
self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(keys =>
      Promise.all(keys.filter(k => k !== CACHE_NAME).map(k => caches.delete(k)))
    )
  );
});

// Fetch: стратегия «сначала кэш, потом сеть»
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(cached =>
      cached || fetch(event.request)
    )
  );
});
});
```

Почему SW не имеет доступа к DOM:

SW живёт в отдельном потоке и может работать, когда страница закрыта (push-уведомления, фоновая синхронизация). Доступа к DOM нет, общение со страницей — через `postMessage`.

Стратегии кэширования:

- **Cache First:** кэш → если нет, то сеть (для статики)
- **Network First:** сеть → если нет, то кэш (для API)
- **Stale While Revalidate:** кэш сразу, потом обновить из сети (для баланса скорости и свежести)

2. WebSocket vs SSE vs Long Polling

	WebSocket	SSE (Server-Sent Events)	Long Polling
Направление	Двустороннее	Сервер → клиент (одностороннее)	Клиент → сервер → ответ
Протокол	ws:// / wss://	HTTP	HTTP
Переподключение	Вручную	Автоматически (встроено)	Вручную
Бинарные данные	Да	Нет (только текст)	Нет
Сложность	Средняя	Низкая	Низкая

Когда что выбирать:

- **WebSocket:** чат, real-time коллаборация, биржевые котировки (нужен двусторонний поток)
- **SSE:** лента уведомлений, прогресс загрузки, логи сервера (только сервер → клиент)
- **Long Polling:** когда WebSocket заблокирован корпоративным прокси (fallback)

```
// SSE – на удивление просто
const source = new EventSource('/api/events/stream');
source.onmessage = (event) => {
  console.log('Новое событие:', JSON.parse(event.data));
};
// Автоматически переподключается при разрыве!
source.onerror = () => console.log('Соединение потеряно, идёт переподключение...');
```

Почему WebSocket может быть избыточен:

Если нужно только получать уведомления, SSE проще: не нужен heartbeat, авто-реконнект из коробки, работает через HTTP/2 (мультиплексирование), не требует отдельной инфраструктуры.

3. History API (SPA-роутинг)

Как работает роутинг без хешей (#/page):

```
// Переход на новую страницу БЕЗ перезагрузки
history.pushState({ page: 'catalog' }, '', '/catalog?sort=price');

// Замена текущей записи (не создаёт новую)
history.replaceState({ page: 'catalog' }, '', '/catalog');

// Кнопка «Назад» в браузере
window.addEventListener('popstate', (event) => {
  console.log('Вернулись на:', document.location.href);
  console.log('Состояние:', event.state); // { page: 'catalog' }
  // Тут перерисовываем UI под новый URL
});
```

Почему pushState, а не location.hash:

- Чистые URL (/catalog вместо /#/catalog)
- Работает SSR (сервер видит путь и может отрендерить страницу)
- Не ломает якорные ссылки (#section)
- SEO: поисковики индексируют пути, но не хеши

Обязанности SPA-роутера:

```
// Роутер должен:
// 1. Перехватывать клики по <a> (чтобы не было перезагрузки)
document.addEventListener('click', (e) => {
  const link = (e.target as Element).closest('a');
  if (link?.origin === location.origin) {
    e.preventDefault();
    history.pushState(null, '', link.href);
    renderRoute(link.pathname);
  }
});

// 2. Реагировать на popstate (кнопки Назад/Вперёд)
window.addEventListener('popstate', () => renderRoute(location.pathname))
;

// 3. При прямом заходе — сервер должен отдать index.html (Nginx try_files)
```

Nginx для SPA:

```
location / {
  try_files $uri $uri/ /index.html;
}
```

4. Web Workers

Зачем: вынести тяжёлые вычисления в отдельный поток, чтобы не блокировать UI.

```
// main.js
const worker = new Worker('/worker.js');
worker.postMessage({ numbers: [1, 2, 3, 4, 5] });
worker.onmessage = (event) => {
  console.log('Результат:', event.data); // не блокирует UI во время
  вычислений
};

// worker.js
self.onmessage = (event) => {
  const { numbers } = event.data;
  const result = heavyComputation(numbers); // может занять секунды
  self.postMessage(result);
};
```

Ограничения Workers:

- Нет доступа к DOM
- Нет доступа к window, document, localStorage
- Общение только через postMessage (сериализация: structured clone)
- Для передачи больших данных — Transferable Objects (ArrayBuffer без копирования)

5. Intersection Observer и Resize Observer

Intersection Observer — наблюдение за видимостью элемента (ленивая загрузка, infinite scroll, аналитика):

Как читать *new IntersectionObserver(callback, options)*: читай как «создай датчик движения для элемента — он сработает когда элемент появится или исчезнет из видимой области». Колбэк получает массив *entries* — это не один элемент, а ВСЕ элементы, которые изменили видимость в этом кадре. Мнемоника: «IntersectionObserver — это датчик: вижу / не вижу».

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      console.log('Элемент появился в viewport!');
      loadLazyImage(entry.target);
      observer.unobserve(entry.target); // больше не следим
    }
  });
}, { threshold: 0.1 }); // 10% элемента видимо

observer.observe(document.querySelector('.lazy-image'));
```

Почему это лучше, чем scroll-событие:

- Не вызывает reflow при чтении позиций

- Браузер оптимизирует проверки на compositor-потоке
- Не нужно debounce/throttle

Resize Observer — отслеживание изменения размеров элемента (контейнерные запросы без CSS):

```
const ro = new ResizeObserver(entries => {
  for (const entry of entries) {
    const { width } = entry.contentRect;
    entry.target.classList.toggle('compact', width < 400);
  }
});
ro.observe(document.querySelector('.sidebar'));
```

Fetch API и AbortController

AbortController

Отмена fetch-запроса при размонтировании компонента или смене параметров:

Как читать `new AbortController()` + `signal`: читай как «создай пульт дистанционного управления запросом». `signal` — это провод от пульта к `fetch`, а `controller.abort()` — это красная кнопка «отмена». Если нажал — `fetch` выбросит ошибку `AbortError`, которую ты просто глотаешь. Мнемоника: «AbortController — это выключатель для fetch».

```
const controller = new AbortController();

useEffect(() => {
  fetch('/api/data', { signal: controller.signal })
    .then(res => res.json())
    .then(setData)
    .catch(err => {
      if (err.name === 'AbortError') return; // нормально
      throw err;
    });

  return () => controller.abort();
}, []);
```

Fetch Streaming

Чтение ответа по мере поступления (потокковая загрузка):

```
const response = await fetch('/api/stream');
const reader = response.body.getReader();
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  // value – Uint8Array с очередной порцией данных
}
```

IndexedDB

Клиентская NoSQL-база в браузере. Полезна для:

- Офлайн-данных (PWA)
- Кэширования больших объёмов (аналитика, логи)
- Работы с бинарными данными

Связанное

- Браузер и HTTP
- Вопросы на собеседовании FullStack
- React рендеринг и производительность

CSS и стилизация

CSS и стилизация

CSS — это язык, который превращает HTML-разметку в визуальный интерфейс. Без него любое веб-приложение выглядит как белый лист с чёрным текстом. Эта страница охватывает весь спектр: от базовых механик до продвинутых стратегий стилизации в 2026 году.

Объяснение для ребёнка

Представь, что HTML — это скелет дома: стены, двери, окна. А CSS — это отделка: какого цвета стены, какие занавески, где стоит мебель, насколько большие окна.

CSS говорит браузеру:

- Сделай эту кнопку **синей и круглой**
- Расположи картинки **в ряд, а не друг под другом**
- Когда наводишь мышку — кнопка **немного увеличивается**

Без CSS интернет выглядел бы как текст в блокноте. С CSS — сайты становятся красивыми, удобными и «живыми»: кнопки реагируют на нажатия, меню выезжает, картинки плавно появляются.

Самое простое правило CSS выглядит так:

```
/* КТО — ЧТО ДЕЛАЕМ — КАК */
button {
  background: blue;
  color: white;
  border-radius: 8px;
}
```

Браузер читает это и понимает: «все кнопки должны быть синими с белым текстом и скруглёнными углами». Это как давать инструкции художнику, который рисует страницу.

Junior-уровень

Базовые механики CSS

Каскад и специфичность

CSS расшифровывается как Cascading Style Sheets. **Каскад** означает, что стили применяются по порядку: если два правила конфликтуют, побеждает то, которое «последнее по порядку чтения» — но только при равной специфичности.

Специфичность — это система весов селекторов. Представь счёт (a, b, c, d):

Селектор	Вес	Пример
style="" (inline)	(1,0,0,0)	<div style="color:red">
#id	(0,1,0,0)	#header
.class, [attr], :hover	(0,0,1,0)	.button, [type="text"]
div, span, ::before	(0,0,0,1)	div, ::after

```
/* Специфичность (0,0,0,1) */
div { color: blue; }

/* Специфичность (0,0,1,0) — побеждает! */
.button { color: green; }
```

Важные правила:

- !important ломает каскад — используй только для переопределения сторонних стилей
- Селекторы по тегам — минимальный вес, их легко переопределить
- Вложенность через пробел (div p) суммирует вес

Наследование

Некоторые свойства передаются от родителя к потомкам автоматически, другие — нет:

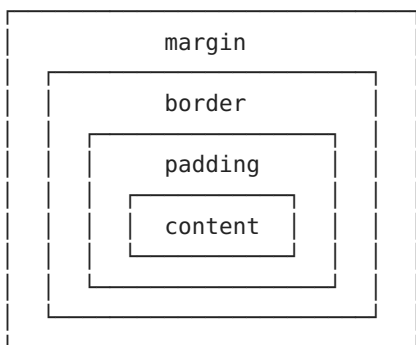
```
/* Наследуются: */  
/* color, font-family, font-size, text-align, line-height, visibility,  
cursor */  
  
/* НЕ наследуются: */  
/* margin, padding, border, width, height, background, position */
```

Ключевые слова для управления наследованием:

- inherit — принудительно наследовать от родителя
- initial — сбросить до значения по умолчанию (спецификация)
- unset — если свойство наследуемое → inherit, иначе → initial
- revert — откатить до стилей браузера (user-agent stylesheet)

Box Model (блочная модель)

Каждый HTML-элемент — это прямоугольник, состоящий из слоёв (изнутри наружу):



box-sizing: border-box — самое важное правило для Junior:

```
/* Без него: width = только контент */  
/* С ним: width = контент + padding + border */  
*, *::before, *::after {  
    box-sizing: border-box;  
}
```

С border-box легче управлять размерами: задал width: 200px — получил 200px вместе с padding и border'ом.

Единицы измерения

Единица	Значение	Когда использовать
px	Абсолютный пиксель	Точные размеры, border
rem	Размер шрифта <html>	Типографика, отступы (доступность)
em	Размер шрифта родителя	Отступы внутри компонента
%	Процент от родителя	Ширина колонок
vh / vw	% от высоты / ширины viewport	Полноэкранные секции
dvh / svh / lvh	Динамический viewport height	Мобильные (учитывают адресную строку)

Ключевое правило: rem для типографики, px для границ, избегай абсолютных px для шрифтов.

```
html {  
  /* По умолчанию 16px, но НЕ трогай – пользователь может настроить */  
  font-size: 100%;  
}  
  
h1 { font-size: 2rem; }      /* 32px при стандартных настройках */  
p { font-size: 1rem; }      /* 16px */  
.button { padding: 0.75rem 1.5rem; }
```

Расположение элементов: Flow, Position, Display

Базовые значения display:

- block — занимает всю ширину родителя, перенос строки (div, p, h1)
- inline — ведёт себя как текст, нельзя задать width/height (span, a)
- inline-block — внешне inline, но можно задать размеры
- none — элемент скрыт, не занимает места

position:

- static — по умолчанию, в потоке
- relative — сдвиг относительно своего места в потоке
- absolute — вынимается из потока, позиционируется от ближайшего позиционированного предка
- fixed — от viewport, не скроллится
- sticky — скроллится до границы, потом «прилипает»

Middle-уровень

Методологии CSS и архитектура

На Middle-уровне ты уже не просто пишешь стили — ты проектируешь систему. Ключевой вопрос: **как организовать CSS, чтобы через год не было больно?**

Методология BEM (Block Element Modifier)

BEM — конвенция именования, делающая CSS предсказуемым и изолированным:

```
/* Блок */
.card { }
/* Элемент */
.card__title { }
.card__body { }
/* Модификатор */
.card--featured { }
.card__title--large { }
```

Плюсы: плоская специфичность, самодокументирование, нет конфликтов.

Минусы: длинные имена, boilerplate, вручную.

Современный CSS: Flexbox

Flexbox — одномерная модель раскладки (строка ИЛИ колонка):

```
.container {
  display: flex;
  /* Главная ось */
  justify-content: space-between; /* start | center | end | space-between | space-around | space-evenly */
  /* Поперечная ось */
  align-items: center;           /* stretch | start | center | end | baseline */
  /* Перенос */
  flex-wrap: wrap;
  /* Промежутки */
  gap: 1rem;                    /* row-gap + column-gap */
}

/* Дочерние элементы */
.item {
  flex: 1;                      /* grow */
}
.item-fixed {
  flex: 0 0 200px; /* grow shrink basis */
}
```

Ключевые моменты:

- margin: auto во flex-контейнере «съедает» свободное пространство
- flex-shrink: 0 предотвращает сжатие элемента
- gap работает и во Flexbox, и в Grid

Современный CSS: Grid

Grid — двумерная модель (строки И колонки одновременно):

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 1rem;
}

/* Размещение по линиям */
.header { grid-column: 1 / -1; }           /* на всю ширину */
.sidebar { grid-column: 1; grid-row: 2 / 4; }
.content { grid-column: 2 / -1; }

/* Именованные области */

> **Как читать `grid-template-areas`:** читай как ASCII-рисунок твоего макета — каждая строка в кавычках это ряд, каждое слово внутри — ячейка. Одинаковые имена объединяются в одну область. ` "header header" ` значит «header занимает две ячейки в первом ряду». Мнемоника: *«grid-template-areas — рисуешь макет текстом, как смайлик»*.

.layout {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar content"
    "footer footer";
  grid-template-columns: 250px 1fr;
}
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
```

Главное отличие от Flexbox: Grid управляет и строками, и колонками одновременно. Flexbox — только одной осью. Grid — для страничных раскладок. Flexbox — для компонентов (навигация, карточки, кнопки).

Адаптивность и responsive-дизайн

Mobile-first

Начинай с мобильной версии, расширяй через min-width (а не max-width):

```

/* Desktop-first: сложнее переопределять */
.sidebar { width: 300px; }
@media (max-width: 768px) {
  .sidebar { width: 100%; }
}

/* Mobile-first: меньше кода, естественное расширение */
.sidebar { width: 100%; }
@media (min-width: 768px) {
  .sidebar { width: 300px; }
}

```

Fluid Typography с clamp()

clamp(MIN, PREFERRED, MAX) — один из самых полезных инструментов:

Как читать *clamp(MIN, PREFERRED, MAX)*: читай как «возьми значение PREFERRED, но не меньше MIN и не больше MAX». Средний параметр — обычно vw (процент от ширины экрана), который плавно растёт/уменьшается, а крайние значения не дают шрифту стать слишком мелким или огромным. Мнемоника: «clamp — это min и max в одной функции: гуляй между ними, но за забор не выходи».

```

h1 {
  /* Минимум 1.5rem, максимум 3rem, между ними — fluid */
  font-size: clamp(1.5rem, 4vw, 3rem);
}

p {
  /* 16px на маленьких экранах, 20px на больших, без breakpoint'ов */
  font-size: clamp(1rem, 0.5rem + 1vw, 1.25rem);
}

```

Формула для плавного масштабирования между двумя точками:

```

/*
 * font-size: clamp(
 *   min-font-size,
 *   min-font-size + (max-font-size - min-font-size) * (100vw - min-
viewport) / (max-viewport - min-viewport),
 *   max-font-size
 * )
 */

```

Container Queries

В отличие от Media Queries (смотрят на ширину viewport), Container Queries смотрят на ширину **родительского контейнера**. Это революция для переиспользуемых компонентов:

Как читать `@container`: читай как «если мой КОНТЕЙНЕР (не весь экран!) стал уже 400px — перестрой меня». В отличие от `@media`, который смотрит на ширину окна браузера, `@container` смотрит на ближайшего родителя с `container-type`. Мнемоника: «`@container` — спроси не у окна, а у родителя».

```
/* Определяем контейнер */
.card-wrapper {
  container-type: inline-size;
  container-name: card;
}

/* Стили зависят от ширины контейнера, а не viewport! */
@container card (min-width: 400px) {
  .card {
    display: grid;
    grid-template-columns: 200px 1fr;
  }
}

@container card (max-width: 399px) {
  .card {
    display: flex;
    flex-direction: column;
  }
}
```

Container Queries решают проблему: один и тот же компонент в узком сайдбаре выглядит иначе, чем в широкой основной области — без JavaScript и без привязки к глобальным breakpoint'ам.

Новые viewport-единицы

На мобильных устройствах адресная строка браузера «съедает» часть viewport. `100vh` игнорирует это:

```
.hero {
  /* На iOS Safari 100vh = высота с учётом скрытой адресной строки → скролл */
  height: 100vh;

  /* dvh динамически учитывает видимую область */
  height: 100dvh;

  /* svh — Small Viewport Height (когда адресная строка показана) */
  /* lvh — Large Viewport Height (когда адресная строка скрыта) */
}
```

CSS-препроцессоры и PostCSS

Sass/SCSS

Препроцессор, который добавляет в CSS возможности языков программирования:

```
// Переменные
$primary: #3b82f6;
$spacing: 1rem;

// Вложенность
.card {
  background: white;
  &__title { font-size: 1.5rem; } // .card__title
  &:hover { box-shadow: 0 4px 12px rgba(0,0,0,0.1); }

  &--featured {
    border-color: $primary;
    .card__title { color: $primary; }
  }
}

// Миксины
@mixin flex-center($direction: row) {
  display: flex;
  flex-direction: $direction;
  justify-content: center;
  align-items: center;
}

// Функции
@function rem($px) { @return calc($px / 16) + rem; }

// Циклы — генерация utility-классов
@for $i from 1 through 4 {
  .mt-#{ $i } { margin-top: $i * $spacing; }
}
```

PostCSS

Это **не препроцессор**, а постпроцессор — плагин-система, манипулирующая уже валидным CSS:

- **Autoprefixer** — добавляет вендорные префиксы (-webkit-, -moz-)
- **cssnano** — минификация CSS
- **postcss-nesting** — вложенность по стандарту CSS Nesting
- **postcss-preset-env** — полифилы для будущих фич CSS

```

/* Пишешь современный CSS: */
.card {
  & .title { font-size: 1.5rem; } /* CSS Nesting — станет стандартом */

  &:has(img) { padding:
0; } /* :has() — уже в современных браузерах */
}

/* PostCSS конвертирует для старых браузеров: */
/* .card .title { font-size: 1.5rem; } */
/* Добавляет -webkit- префиксы где нужно */

```

CSS Modules — scoping без рантайма

CSS Modules решают главную проблему глобального CSS: **конфликт имён**. Файл Button.module.css:

```

/* Button.module.css */
.root { padding: 8px 16px; }
.primary { background: var(--color-primary); }

```

Импорт в компоненте React:

```

import styles from './Button.module.css';

function Button({ variant = 'primary' }) {
  return <button className={` ${styles.root} ${styles[variant]} `>Click</button>;
}

```

Что происходит под капотом (сборщик, например Vite/webpack):

```

.root    → .Button_root_abc123
.primary → .Button_primary_abc123

```

- Каждый класс получает **уникальный хеш** (обычно [filename]_[classname]_[hash])
- Глобальных конфликтов имён не существует
- composes: позволяет переиспользовать стили внутри модуля:

```

.base { padding: 8px; }
.primary { composes: base; background: blue; }

```

Плюсы: нулевой рантайм, tree-shaking (неиспользуемые классы удаляются сборщиком), TypeScript-поддержка через *.module.css.d.ts.

Минусы: динамические стили требуют inline styles или CSS-переменных; стили — отдельные файлы, а не colocated с компонентом.

Анимации

CSS Transitions

Для простых переходов между двумя состояниями:

```
.button {
  background: #3b82f6;
  transition: background 0.2s ease, transform 0.15s ease;
}
.button:hover {
  background: #2563eb;
  transform: scale(1.05);
}
```

Функции сглаживания (easing):

- ease — плавный старт, быстрая середина, плавный конец (по умолчанию)
- linear — равномерная скорость
- ease-in / ease-out — ускорение / замедление
- cubic-bezier(0.4, 0, 0.2, 1) — кастомная кривая

CSS Keyframe Animations

Для сложных, повторяющихся или многоэтапных анимаций:

```
@keyframes shimmer {
  0% { background-position: -200% 0; }
  100% { background-position: 200% 0; }
}

.skeleton {
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0
75%);
  background-size: 200% 100%;
  animation: shimmer 1.5s ease-in-out infinite;
}
```

Управление анимацией:

- animation-fill-mode: forwards — сохраняет последний кадр
- animation-play-state: paused — ставит на паузу (полезно для prefers-reduced-motion)
- animation-composition: add — комбинирует трансформации вместо перезаписи

Анимации и accessibility

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}
```


Framer Motion против CSS-анимаций

	CSS Animations/ Transitions	Framer Motion
Сложность	Простые переходы, keyframes	Сложные жесты, layout-анимации, exit-анимации
Контроль	Декларативный	Императивный (animate(), useAnimationControls())
Unmount	Нельзя анимировать размонтирование	AnimatePresence
Жесты	Только :hover, :active	drag, tap, hover, pan, whileInView
Производительность	GPU-ускорение из коробки	layout-анимации через FLIP, иногда дорого
Бандл	0 KB	~30 KB gzipped
Когда использовать	Простые UI-переходы, скелетоны, hover	Сложные интерактивные анимации, drag&drop, page transitions

Правило выбора: начинай с CSS. Если нужны exit-анимации, жесты или layout-анимации — подключай Framer Motion.

Senior-уровень

Стратегия стилизации: обзор подходов

Senior выбирает подход к стилизации для **всей команды**, а не для одного компонента. Каждый вариант имеет компромиссы.

Tailwind CSS — utility-first

Tailwind — это набор атомарных utility-классов. Вместо написания CSS ты составляешь стили из готовых «кирпичиков»:

```
<button class="px-4 py-2 bg-blue-500 text-white rounded-lg hover:bg-blue-600 transition-colors duration-200 font-medium">
  Click me
</button>
```

Настройка (tailwind.config.ts):

```
export default {
  theme: {
    extend: {
      colors: {
        brand: {
          50: '#eff6ff',
          500: '#3b82f6',
          900: '#1e3a5f',
        }
      },
      spacing: {
        '4.5': '1.125rem',
      }
    }
  },
  // Оптимизация для production
  content: ['./src/**/*.{ts,tsx}'],
}
```

Как Tailwind работает под капотом (сборка):

1. Сканирует content-файлы на наличие классов
2. Генерирует CSS **только для используемых классов** (tree-shaking)
3. Итоговый бандл CSS — 3-10 KB gzipped (после очистки)

Критика Tailwind:

- Захламляет JSX — много классов в разметке
- Динамические классы не работают без конфига (нельзя `bg-${color}-500`)
- Отрыв от реального CSS — разработчик забывает, как писать CSS
- При одинаковых стилях — дублирование (решается компонентами)
- Мгновенный прототип — не переключаешься между файлами
- Дизайн-система из коробки (spacing, colors, breakpoints)
- Консистентность в команде — нет 50 оттенков серого
- Предсказуемый бандл — растёт линейно с количеством уникальных классов

CSS-in-JS: эволюция

Styled-components (runtime CSS-in-JS):

Первопроходец. Стили — это tagged template literals. Генерация CSS происходит в рантайме:

```
import styled from 'styled-components';

const Button = styled.button<{ $variant: 'primary' | 'secondary' }>`
  padding: 8px 16px;
  border-radius: 8px;
  background: ${props => props.$variant === 'primary' ? '#3b82f6' : '#e5e7eb'};
  color: ${props => props.$variant === 'primary' ? 'white' : '#1f2937'};

  &:hover {
    opacity: 0.9;
  }
`;
```

Проблемы runtime CSS-in-JS (styled-components, Emotion):

- **Бандл вырастает** на 12-15 KB gzipped (библиотека)
- **JavaScript-накладные расходы** при SSR — стили нужно «вытянуть» из JS в CSS перед отправкой HTML
- **RSC (React Server Components) несовместимость** — серверные компоненты не могут использовать runtime CSS-in-JS
- **FOUC (Flash of Unstyled Content)** при неправильном SSR

Vanilla Extract — zero-runtime CSS-in-JS:

Стили пишутся в TypeScript, но **компилируются в статический CSS на этапе сборки**:

```
// styles.css.ts
import { style } from '@vanilla-extract/css';

export const button = style({
  padding: '8px 16px',
  background: 'blue',
  ':hover': { opacity: 0.9 }
});

export const primary = style({
  background: '#3b82f6',
  color: 'white'
});
```

```
// Button.tsx
import * as styles from './styles.css';

<button className={`${styles.button} ${styles.primary}`}>Click</button>
```

Zero-runtime означает: **в production нет JS-библиотеки стилизации**.

Vanilla Extract компилируется в CSS-файлы на этапе сборки.

Panda CSS — compile-time, utility-first + целостность типов:

```
import { css } from '../styled-system/css';

<button className={css({ bg: 'blue.500', px: '4', py: '2', rounded:
'lg' })}>
  Click
</button>
```

Panda CSS генерирует статический CSS с атомарными классами, как Tailwind, но с type-safe API. bg: 'blue.500' проверяется TypeScript'ом на этапе компиляции.

Сравнительная таблица подходов:

Подход	Рантайм	Бандл	RSC	TypeScript	Динамика
CSS Modules	0 KB			*.module.css.d.ts	CSS-переменные
Tailwind	0 KB (CSS)			(строка класса)	data-[state=open]:flex
styled-components	~14 KB				Полная (props)
Vanilla Extract	0 KB			(типы)	styleVariants, recipes
Panda CSS	0 KB				cva, recipes

Совет Senior: в 2025-2026 индустрия движется к **zero-runtime** решениям. Tailwind доминирует по простоте. Panda CSS и Vanilla Extract — компромисс между type-safety и производительностью. Styled-components и Emotion уходят в legacy.

CSS-архитектура: @layer и :has()

@layer — управление каскадом

@layer позволяет явно задать порядок приоритета стилей, независимо от порядка в коде:

```

/* Порядок приоритета: чем позже объявлен слой, тем выше его приоритет */
@layer reset, base, components, utilities;

@layer reset {
  *, *::before, *::after { box-sizing: border-box; margin: 0; }
}

@layer base {
  a { color: var(--color-link); }
}

@layer components {
  .card { padding: 1rem; }
}

@layer utilities {
  .mt-0 { margin-top: 0 !important; } /* !important в низкоприоритетном
слое — ок */
}

```

Зачем: utility-классы всегда перебивают компоненты, компоненты — базу, база — reset. Без !important и магии специфичности. Tailwind использует @layer для этого же.

:has() — «родительский селектор»

:has() позволяет стилизовать родителя на основе содержимого:

```

/* Карточка с картинкой стилизуется иначе */
.card:has(img) {
  padding: 0;
}

/* Форма с ошибкой — красная рамка */
.form-group:has(input:invalid) {
  border-color: red;
}

/* Hover на всей карточке при наведении на кнопку */
.card:has(.card__button:hover) {
  box-shadow: 0 4px 12px rgba(0,0,0,0.15);
}

/* Соседний элемент изменился */
.label:has(+ input:focus) {
  color: var(--color-primary);
}

```

:has() + Grid — мощный тандем:

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
}

/* Если в галерее всего 1 элемент – растяни на весь ряд */
.gallery:has(> :only-child) {
  grid-template-columns: 1fr;
}

/* Если 2 элемента – два столбца */
.gallery:has(> :nth-child(2):last-child) {
  grid-template-columns: 1fr 1fr;
}
```

Производительность CSS

Что дорого в CSS

Не все CSS-свойства одинаковы. Браузер выполняет рендеринг в несколько этапов:



Чем раньше этап, тем дороже изменение:

Этап	Свойства	Стоимость
Layout (reflow)	width, height, margin, padding, top, left, display, font-size	Максимальная
Paint (repaint)	color, background, box-shadow, outline, border-color	Средняя
Composite	transform, opacity	Минимальная

Лучшие практики:

```

/* Дорого: анимация width/height вызывает layout на каждом кадре */
.bad-animation {
  transition: width 0.3s;
}

/* Дешево: transform запускает только composite */
.good-animation {
  transition: transform 0.3s;
}
.good-animation:hover {
  transform: scale(1.05); /* ← только composite */
}

/* Дорого: box-shadow при скролле */
.card { box-shadow: 0 4px 20px rgba(0,0,0,0.15); }

/* Дешево: filter на отдельном слое */
.card { filter: drop-shadow(0 4px 6px rgba(0,0,0,0.1)); }

```

will-change — предупреждение браузеру

will-change говорит браузеру заранее подготовить GPU-слой:

```

.animated-element {
  will-change: transform, opacity;
}

```

Правила использования:

- Применяй **незадолго до** анимации и **убирай после**
- НЕ вешай на всё подряд — каждый слой потребляет GPU-память
- Не используй для исправления производительности «на всякий случай»

```

// Правильный паттерн в React
function AnimatedCard() {
  const [animating, setAnimating] = useState(false);

  return (
    <div
      style={{ willChange: animating ? 'transform' : 'auto' }}
      onMouseEnter={() => setAnimating(true)}
      onMouseLeave={() => setAnimating(false)}
    >
      Content
    </div>
  );
}

```

content-visibility — пропуск рендеринга вне экрана

content-visibility: auto говорит браузеру: «не рендерь содержимое этого элемента, пока он не появится в viewport»:

```
.list-item {
  content-visibility: auto;
  /* Браузеру нужна оценка высоты — иначе скроллбар будет дёргаться */
  contain-intrinsic-size: 0 200px; /* ширина высота */
}
```

Эффект: на странице с 1000+ элементов рендеринг ускоряется в 5-10 раз. Браузер пропускает layout и paint для невидимых элементов.

Осторожно: content-visibility: auto ломает scrollIntoView(), поиск по странице (Ctrl+F) и accessibility-дерево для скрытых элементов. Не применяй на критическом контенте.

CSS Containment

```
.widget {
  contain: layout style paint;
}
```

Говорит браузеру, что изменения внутри .widget не влияют на внешний layout. Позволяет браузеру пропускать перерасчёт остальной страницы.

CSS-переменные (Custom Properties) — продвинутое использование

```
:root {
  /* Базовый дизайн-токен */
  --color-primary: #3b82f6;
  --color-primary-light: color-mix(in srgb, var(--color-primary), white 20%);
  --color-primary-dark: color-mix(in srgb, var(--color-primary), black 20%);

  /* Типографика */
  --font-size-base: clamp(1rem, 0.5rem + 1vw, 1.25rem);
  --line-height: 1.5;

  /* Анимации */
  --transition-base: 200ms ease;
}

/* Переопределение в контексте */
.dark-theme {
  --color-primary: #60a5fa;
}

/* Динамика через JavaScript */
/* element.style.setProperty('--mouse-x', e.clientX + 'px'); */
```

color-mix() — новая функция в CSS, позволяющая смешивать цвета без препроцессора:


```
.button {
  background: var(--color-primary);
}
.button:hover {
  background: color-mix(in srgb, var(--color-primary), black 15%);
}
```

Style Queries (новое)

Style Queries — дополнение к Container Queries, реагирующее на значения CSS-переменных:

```
@container style(--theme: dark) {
  .card {
    background: #1e293b;
    color: #e2e8f0;
  }
}
```

Позволяет менять стили компонента в зависимости от CSS-переменной родителя, а не только от класса или контекста React.

Логические свойства (Logical Properties)

Для поддержки RTL (right-to-left) и мультиязычности:

```
/* . Физические */
.element {
  margin-left: 1rem;
  padding-right: 1rem;
  border-top: 1px solid;
}

/* Логические */
.element {
  margin-inline-start: 1rem;
  padding-inline-end: 1rem;
  border-block-start: 1px solid;
}
```

Физическое	Логическое
margin-left	margin-inline-start
margin-right	margin-inline-end
margin-top	margin-block-start
margin-bottom	margin-block-end
text-align: left	text-align: start
width / height	inline-size / block-size

CSS Nesting (нативный)

С 2023-2024 — встроенная вложенность в CSS без препроцессоров:

```
.card {
  background: white;

  & .title {
    font-size: 1.5rem;
  }

  &:hover {
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);

    & .title {
      color: var(--color-primary);
    }
  }

  @media (width >= 768px) {
    padding: 2rem;
  }
}
```

Поддержка браузерами: 90%+. Для продакшена — PostCSS Nesting для старых браузеров.

Когда что использовать — итоговая матрица Senior

Задача	Решение
Быстрый прототип, стартап	Tailwind CSS
Enterprise, несколько команд, дизайн-система	Tailwind + конфиг + компоненты ИЛИ Panda CSS
Type-safe стили, строгая типизация	Vanilla Extract / Panda CSS
Переиспользуемые компоненты в разных контекстах	Container Queries + Style Queries
Микросервисный фронтенд (Module Federation)	CSS Modules (изоляция на уровне сборки)
RSC + Next.js App Router	Tailwind / CSS Modules / Vanilla Extract / Panda CSS
Анимации	CSS Animations + Framer Motion для сложных сценариев
RTL / мультиязычность	Logical Properties + dir="rtl"

Связанное

- Архитектура React компонентов — как стилизация связана с композицией компонентов
- Сборщики и инструменты — как Vite/webpack обрабатывают CSS, PostCSS, CSS Modules
- Браузер и HTTP — Critical Rendering Path, reflow/repaint, как CSS влияет на скорость загрузки
- React рендеринг и производительность — влияние стилизации на рендеринг React
- Интернационализация и локализация i18n — RTL, Logical Properties в CSS
- Feature-sliced design и Atomic Design — как организовать стили в масштабе приложения
- Виртуализация рендеринга — content-visibility и contain-intrinsic-size как CSS-альтернативы

HTML и Accessibility (a11y)

HTML и Accessibility (a11y)

Доступность — это не «фича для незрячих», а базовая характеристика качественного веба. ~15% населения имеют ту или иную форму инвалидности, и почти каждый временно оказывается в контексте ограниченных возможностей (сломанная мышь, яркое солнце, шумное окружение).

Уровень 1: Объясни ребёнку (10 лет)

Представь, что ты строишь дом из LEGO. У тебя есть кубики разной формы:

- **Красный длинный** — это крыша
- **Синий квадратный** — это стена
- **Жёлтый маленький** — это окно
- **Зелёный плоский** — это дверь

Если ты построишь дом из одинаковых серых кубиков, никто не поймёт, где дверь, а где окно. Даже ты сам через неделю запутаешься!

HTML-теги — это как цветные кубики LEGO. Когда ты используешь `<nav>` — ты говоришь браузеру: «Это меню, здесь ссылки для навигации». Когда `<button>` — «Это кнопка, на неё можно нажать». Когда `<main>` — «Это самое важное на странице».

А теперь представь, что твой друг не видит. Он «смотрит» сайт не глазами, а ушами — специальная программа читает ему всё вслух. Если сайт собран из серых кубиков (`<div>`), программа не знает, что важно, а что нет. Она просто говорит: «блок, блок, блок, блок». Скучно и непонятно!

Но если использовать правильные теги, программа говорит: «Заголовок: Новости спорта. Кнопка: Подписаться. Ссылка: Контакты». Так сайт становится понятным для всех.

Главное правило: всегда выбирай правильный «кубик» (тег), а не лепи всё из серого `<div>`.

Уровень 2: Junior-разработчик

На этом уровне ты должен знать базовые правила и применять их в повседневной вёрстке.

Семантический HTML — фундамент доступности

Каждый тег имеет встроенную **роль** (role), которую понимают скринридеры и браузеры:

Тег	Роль	Когда использовать
<code><header></code>	banner	Шапка сайта
<code><nav></code>	navigation	Навигационное меню
<code><main></code>	main	Основной контент (только один на странице!)
<code><section></code>	region	Тематическая секция с заголовком
<code><article></code>	article	Самодостаточный контент (пост, карточка)
<code><aside></code>	complementary	Боковая панель, дополнительная информация
<code><footer></code>	contentinfo	Подвал сайта
<code><h1>-<h6></code>	heading	Заголовки с уровнем
<code><button></code>	button	Кнопка действия
<code><a></code>	link	Ссылка для перехода

```

<!-- Див-суп: скринридер не понимает структуру -->
<div class="header">
  <div class="nav">
    <div class="link" onclick="...">Главная</div>
  </div>
</div>
<div class="content">...</div>

<!-- Семантически: скринридер озвучит роли -->
<header>
  <nav aria-label="Главное меню">
    <a href="/">Главная</a>
  </nav>
</header>
<main>
  <article>
    <h1>Заголовок статьи</h1>
    <p>Содержание...</p>
  </article>
</main>

```

Заголовки: иерархия h1-h6

Правильная структура заголовков — это **оглавление** для скринридера:

```

<!-- Пропуск уровней -->
<h1>Мой сайт</h1>
<h3>Раздел</h3>      <!-- где h2? -->
<h5>Подраздел</h5>   <!-- где h4? -->

<!-- Правильная иерархия -->
<h1>Мой сайт</h1>
  <h2>Раздел 1</h2>
    <h3>Подраздел 1.1</h3>
    <h3>Подраздел 1.2</h3>
  <h2>Раздел 2</h2>
    <h3>Подраздел 2.1</h3>

```

Скринридеры позволяют навигировать по заголовкам — пользователь нажимает клавишу n и прыгает между ними. Это как содержание книги.

Доступность форм

```
<!-- Placeholder вместо label -->
<input type="email" placeholder="Введите email">

<!-- Явная связь label-input -->
<label for="email">Ваш email</label>
<input type="email" id="email" name="email">

<!-- Обращивание label (работает без for/id) -->
<label>
  <input type="checkbox"> Я согласен с условиями
</label>

<!-- Группировка с fieldset/legend -->
<fieldset>
  <legend>Способ оплаты</legend>
  <label><input type="radio" name="payment" value="card"> Карта</label>
  <label><input type="radio" name="payment" value="cash"> Наличные</label>
</fieldset>
```

Изображения: alt-текст

```
<!-- Содержательное изображение — описываем -->


<!-- Декоративное (иконка, разделитель) — пустой alt -->


<!-- Сложная инфографика — краткий alt + ссылка на описание -->

<p id="desc">Подробное описание: микросервисная архитектура из 5 сервисов...</p>
```

Базовое ARIA

Первое правило ARIA (First Rule of ARIA): Если можно использовать нативный HTML-элемент со встроенной семантикой — **НЕ используй ARIA**. Нативный `<button>` всегда лучше, чем `<div role="button">`.

```
<!-- Не делай так -->
<div role="button" tabindex="0" onclick="...">Нажми меня</div>

<!-- Делай так -->
<button type="button">Нажми меня</button>
```

Когда ARIA оправдано: кастомные виджеты, которых нет в HTML (вкладки/tabs, древовидные списки/tree, живые регионы).

Уровень 3: Middle-разработчик

На Middle ты понимаешь, КАК это работает под капотом, и можешь осознанно выбирать инструменты.

Accessibility Tree

Браузер строит **Accessibility Tree** параллельно с DOM. Это упрощённая версия DOM, содержащая только семантическую информацию:

Как читать Accessibility Tree: DOM — это что видит браузер (все `<div>`, `<span>`, атрибуты). Accessibility Tree — это что «слышит» скринридер (только смысл: «заголовок», «кнопка», «ссылка»). Визуальный мусор вроде `<div>`-обёрток исчезает, остаётся чистая структура страницы.

DOM Tree

```
<html>
  <body>
    <header>
      <nav>
        <a>Главная</a>
        <a>0 нас</a>
      </nav>
    </header>
    <main>
      <h1>Мой сайт</h1>
      <article>
        <h2>Статья</h2>
        <p>Текст...</p>
      </article>
    </main>
    <button>Подписаться</button>
```

Accessibility Tree

```
WebArea
├── banner (header)
│   ├── navigation (nav)
│   │   ├── link "Главная"
│   │   └── link "0 нас"
│   └── main
│       ├── heading "Мой сайт" (h1)
│       ├── article
│       │   ├── heading "Статья" (h2)
│       │   └── paragraph "..."
│       └── button "Подписаться"
```

Важные моменты:

- `display: none` и `visibility: hidden` убирают элемент из Accessibility Tree
- `aria-hidden="true"` убирает элемент ТОЛЬКО из Accessibility Tree (остаётся видимым зрячим)
- CSS content (псевдоэлементы) **попадает** в Accessibility Tree — осторожно с `::before` на интерактивных элементах
- Порядок в Accessibility Tree определяется DOM-порядком, а не визуальным (flexbox order, `position: absolute` не меняют его)

ARIA: роли, свойства, состояния

ARIA — это контракт между разработчиком и скринридером. Ты обещаешь определённое поведение через роли и свойства.

Роли (roles) — «что это такое»

```
<!-- Виджет-роли: интерактивные элементы -->
<div role="tablist">
  <button role="tab" aria-selected="true">Первая</button>
  <button role="tab" aria-selected="false">Вторая</button>
</div>
<div role="tabpanel">Содержимое первой вкладки</div>

<!-- Структурные роли -->
<div role="alert">Важно! Ошибка сохранения.</div>
<div role="status">Загрузка завершена.</div>
<div role="dialog" aria-modal="true" aria-labelledby="dlg-title">
  <h2 id="dlg-title">Подтверждение</h2>
</div>
```

Свойства (properties) — «какой он»

- `aria-label="Заккрыть меню"` — текст для скринридера (переопределяет видимый текст)
- `aria-labelledby="id1 id2"` — ссылается на другие элементы как на заголовок
- `aria-describedby="error-msg"` — связывает с описанием/подсказкой
- `aria-controls="panel1"` — указывает, чем управляет элемент

Состояния (states) — «что с ним сейчас»

- `aria-expanded="true/false"` — раскрыто/свёрнуто
- `aria-selected="true/false"` — выбрано
- `aria-disabled="true"` — заблокировано
- `aria-hidden="true"` — скрыто от screen reader
- `aria-current="page"` — текущая страница в навигации

```
<!-- Аккордеон с ARIA -->
<button
  aria-expanded="false"
  aria-controls="faq-1"
  id="faq-btn-1"
>
  Как мне вернуть деньги?
</button>
<div
  role="region"
  id="faq-1"
  aria-labelledby="faq-btn-1"
  hidden
>
  Откройте раздел «Мои заказы» и нажмите «Возврат».
</div>
```


ARIA Live Regions

Живые регионы (live regions) сообщают скринридеру об изменениях в DOM без фокуса:

Как читать *aria-live="polite" aria-atomic="true"*: «скринридер, когда в этом блоке что-то изменится — вежливо дождись паузы в речи и прочитай ВСЁ блок целиком, а не только изменившийся кусочек». *polite* = «не перебивай», *assertive* = «перебей и прочитай немедленно», *atomic* = «читай блок как единое целое».

```
<!-- aria-live="polite" – объявит, когда пользователь закончит текущее действие -->
<div aria-live="polite" aria-atomic="true">
  <!-- Динамически обновляется через JS -->
  Найдено: <span id="search-count">0</span> результатов
</div>

<!-- aria-live="assertive" – прервёт текущую речь и объявит немедленно -->
<div aria-live="assertive" role="alert">
  <!-- Сообщения об ошибках -->
</div>

<!-- role="alert" – эквивалент aria-live="assertive" + aria-atomic="true" -->
<div role="alert">Форма не отправлена: проверьте email.</div>
```

Правила live regions:

- **Элемент ДОЛЖЕН быть в DOM при загрузке** (пустым), иначе не работает
- *aria-atomic="true"* — читать весь регион целиком (не только изменившуюся часть)
- *aria-relevant="additions removals"* — что именно отслеживать
- **Не злоупотребляй** *assertive* — это как кричать на пользователя

Клавиатурная навигация

Веб должен быть полностью управляем с клавиатуры: Tab, Shift+Tab, Enter, Space, стрелки.

Skip Links (пропускные ссылки)

Первая интерактивная ссылка на странице — «перейти к содержимому»:

```

<!-- В самом начале body, ДО хедера -->
<a href="#main-content" class="skip-link">
  Перейти к содержимому
</a>

<style>
.skip-link {
  position: absolute;
  top: -100%;
  left: 0;
  background: #000;
  color: #fff;
  padding: 8px 16px;
  z-index: 10000;
}
.skip-link:focus {
  top: 0;
}
</style>

<header>...</header>
<main id="main-content">...</main>

```

TabIndex

```

<!-- tabindex="0" — добавить в порядок табуляции (порядок = DOM-порядок) -->
<div tabindex="0" role="button">Клихни меня</div>

<!-- tabindex="-1" — убрать из табуляции, но можно фокусировать через JS -->
<div tabindex="-1" id="modal-content">...</div>
<script>
  document.getElementById('modal-content').focus(); // программный фокус
</script>

<!--    tabindex > 0 — НИКОГДА. Ломает естественный порядок табуляции -->
<div tabindex="3">Сначала я</div>
<div tabindex="1">Нет, я!</div>

```

Focus Management

После действий важно управлять фокусом:

- Открытие модалки → фокус на первом интерактивном элементе
- Закрытие модалки → фокус обратно на кнопку, которая её открывала
- SPA-навигация → переместить фокус на <h1> новой страницы или в <main>
- Удаление элемента → фокус на предыдущий/следующий логичный элемент

```
// Пример: фокус-менеджмент для модального окна
function openModal() {
  modal.showModal();
  const firstFocusable = modal.querySelector('button, [href], input,
select, textarea, [tabindex]:not([tabindex="-1"])');
  firstFocusable?.focus();
  // Ловушка фокуса внутри модалки – использовать inert или focus-trap
библиотеку
}

function closeModal() {
  modal.close();
  // Возвращаем фокус на кнопку-триггер
  triggerButton.focus();
}
```

Цветовой контраст (WCAG)

Уровень	Обычный текст	Крупный текст (18px+ жирный / 24px+)
AA (минимум)	4.5:1	3:1
AAA (улучшенный)	7:1	4.5:1

Инструменты:

- Chrome DevTools → Elements → Styles → клик на цвет → калькулятор контраста
- [WebAIM Contrast Checker](#)
- Axe DevTools — автоматическая проверка в DevTools
- Плагин Stark для Figma — проверка на этапе дизайна

```
/* Серый текст на белом – нечитаемый */
.error { color: #ccc; background: #fff; } /* контраст 1.6:1 */

/* Тёмно-красный – контраст 4.6:1 (проходит AA) */
.error { color: #d32f2f; background: #fff; }
```

Важно: цвет НЕ должен быть единственным способом передачи информации:

```
<!-- Только цвет – незрячий не поймёт -->
<span class="error">Ошибка</span>
<span class="success">Успешно</span>

<!-- Цвет + иконка + текст -->
<span class="error" role="alert">
  <span aria-hidden="true"> </span> Ошибка: неверный пароль
</span>
<span class="success">
  <span aria-hidden="true"> </span> Успешно: данные сохранены
</span>
```

Тестирование доступности

Автоматическое (ловит ~30% проблем)

```
# Axe-core в браузере (Chrome DevTools → Lighthouse → Accessibility)
# Axe DevTools – плагин для Chrome/Firefox
```

```
# jest-axe для unit-тестов
npm install --save-dev jest-axe
```

```
// jest-axe в React-тестах
import { axe, toHaveNoViolations } from 'jest-axe';
expect.extend(toHaveNoViolations);

it('форма логина не имеет ally-нарушений', async () => {
  const { container } = render(<LoginForm />);
  const results = await axe(container);
  expect(results).toHaveNoViolations();
});
```

```
# Lighthouse CI – встроить в пайплайн
npx lighthouse https://mysite.com --only-categories=accessibility
```

Ручное тестирование (обязательно!)

1. **Клавиатура:** отключи мышь. Пройди весь флоу с Tab/Shift+Tab/Enter/Space. Виден ли фокус? Не застревает ли? Логичен ли порядок?
2. **Скринридер:**
3. **macOS:** VoiceOver (встроен, Cmd+F5)
4. **Windows:** NVDA (бесплатно), JAWS (платно)
5. **Linux:** Orca
6. **Увеличение:** приблизь страницу до 200% — ничего не ломается?
7. **Цвета:** включи эмуляцию цветовой слепоты в DevTools (Rendering → Emulate vision deficiencies)
8. **Семантика:** инспектируй Accessibility Tree в DevTools (Elements → Accessibility)

Уровень 4: Senior-разработчик

На Senior ты проектируешь систему доступности, выстраиваешь процессы и обучаешь команду.

WCAG 2.1/2.2 — полный обзор

WCAG построен на 4 принципах **POUR**:

Принцип	Значение	Примеры критериев
Perceivable (Воспринимаемость)	Информация должна быть доступна органам чувств	Альтернативный текст (1.1.1), субтитры (1.2.2), контраст (1.4.3)
Operable (Управляемость)	Интерфейсом можно управлять	Клавиатура (2.1.1), достаточно времени (2.2.1), нет мигания >3 раз/сек (2.3.1)
Understandable (Понятность)	Информация и интерфейс понятны	Язык страницы (3.1.1), предсказуемость (3.2), помощь при ошибках (3.3)
Robust (Надёжность)	Контент интерпретируется разными agent'ами	Валидный HTML (4.1.1), правильные name/role/value (4.1.2)

Уровни соответствия:

- **A** (минимальный) — базовые требования. Невыполнение = барьер для некоторых пользователей. Пример: у изображений есть alt.
- **AA** (средний) — индустриальный стандарт. Большинство законов требуют AA. Пример: контраст 4.5:1, видимый фокус.
- **AAA** (максимальный) — не всегда достижим для всего контента. Пример: контраст 7:1, язык жестов для видео.

Что нового в WCAG 2.2 (октябрь 2023):

- **2.4.11 Focus Not Obscured (AA)**: фокус не должен быть скрыт другим контентом (sticky header!)
- **2.5.7 Dragging Movements (AA)**: альтернатива жесту перетаскивания
- **2.5.8 Target Size (AA)**: размер цели $\geq 24 \times 24$ px
- **3.2.6 Consistent Help (A)**: помощь (чат, телефон) в одном месте на всех страницах
- **3.3.7 Accessible Authentication (A)**: без CAPTCHA и когнитивных тестов

Проектирование дизайн-системы с a11y

Доступность закладывается на уровне дизайн-токенов и компонентов:

```
// Дизайн-токены с гарантированным контрастом
const colors = {
  text: {
    primary: '#1a1a1a',    // на белом: контраст ~16:1 (AAA)
    secondary: '#595959',  // на белом: контраст ~5.2:1 (AA)
    disabled: '#949494',   // использовать ТОЛЬКО с aria-disabled
  },
  interactive: {
    focus: '#0066CC',      // видимый outline 3px
  },
};

// Компонент кнопки с полным ally
interface ButtonProps {
  children: React.ReactNode;
  isLoading?: boolean;
  'aria-label'?: string;
  // ...
}

function Button({ children, isLoading, ...props }: ButtonProps) {
  return (
    <button
      {...props}
      aria-busy={isLoading}
      aria-disabled={isLoading || props.disabled}
    >
      {isLoading && (
        <span role="status" aria-label="Загрузка">
          <Spinner aria-hidden="true" />
        </span>
      )}
      <span aria-hidden={isLoading}>{children}</span>
    </button>
  );
}
```

Сложные виджеты: паттерны ARIA Authoring Practices

WAI-ARIA Authoring Practices Guide (APG) — это проверенные паттерны для сложных виджетов: tabs, accordion, combobox, tree, grid, dialog.

```
<!-- Пример: autocomplete/combobox по паттерну APG -->
<label for="city-input">Город</label>
```

Как читать ARIA-атрибуты combobox (расшифровка каждого):

role="combobox" — «я — выпадающий список с автодополнением».

aria-expanded="true" — «список сейчас раскрыт».

aria-controls="city-listbox" — «я управляю элементом с id *city-listbox*».

aria-activedescendant="city-2" — «сейчас подсвечен вариант с id *city-2*».

aria-autocomplete="list" — «я предлагаю варианты из выпадающего списка».

aria-haspopup="listbox" — «при активации появится список для выбора».

Каждый атрибут — это предложение, которое ты говоришь скринридеру.

```
<input
  type="text"
  id="city-input"
  role="combobox"
  aria-expanded="true"
  aria-controls="city-listbox"
  aria-activedescendant="city-2"
  aria-autocomplete="list"
  aria-haspopup="listbox"
>
<ul id="city-listbox" role="listbox" aria-label="Города">
  <li role="option" id="city-1">Москва</li>
  <li role="option" id="city-2" aria-selected="true">Минск</li>
  <li role="option" id="city-3">Киев</li>
</ul>
```

Ключевые паттерны (изучи обязательно):

- Disclosure (аккордеон)
- Tabs с ручным/автоматическим переключением
- Modal Dialog с ловушкой фокуса
- Grid с навигацией стрелками
- Tree View с вложенностью и expand/collapse

Интеграция в CI/CD

```
# Пример: GitHub Actions
name: Accessibility Checks
on: [pull_request]

jobs:
  axe:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install
        run: npm ci
      - name: Run axe (unit)
        run: npx jest --testPathPattern="a11y"
      - name: Lighthouse CI
        run: |
          npx lhci autorun --config=.lighthouserc.js
```

```
// .lighthouserc.js
module.exports = {
  ci: {
    assert: {
      preset: 'lighthouse:no-pwa',
      assertions: {
        'categories:accessibility': ['error', { minScore: 0.9 }],
        'color-contrast': 'error',
      },
    },
  },
};
```

Процесс в команде

Как Senior ты выстраиваешь культуру:

1. **Definition of Done включает a11y:** у каждого компонента в Storybook есть вкладка Accessibility (addon-a11y).
2. **Линтинг на этапе написания кода:** eslint-plugin-jsx-a11y ловит отсутствие label, alt, неправильные роли:


```
// .eslintrc.js
module.exports = {
  extends: ['plugin:jsx-ally/recommended'],
  rules: {
    'jsx-ally/anchor-is-valid': 'error',
    'jsx-ally/alt-text': 'error',
    'jsx-ally/label-has-associated-control': 'error',
    'jsx-ally/no-autofocus': 'warn',
    'jsx-ally/tabindex-no-positive': 'error',
  },
};
```

1. **A11y-роудмап:** не пытайся исправить всё сразу. Приоритизируй:
2. Блокирующие проблемы (невозможно использовать клавиатуру)
3. Критический пользовательский путь (логин, покупка, поиск)
4. Второстепенные страницы
5. **Обучение:** проведи воркшоп по скринридерам. Разработчики должны хотя бы раз пройти свой сайт с выключенным экраном и VoiceOver/ NVDA.

Серверный рендеринг и a11y

При SSR/SSG важно:

- Заголовки корректно отрендерены на сервере (скринридер парсит HTML до гидрации)
- Skip-link видна без JavaScript
- <html lang="ru"> установлен на сервере
- Мета-теги <title> уникальны для каждой страницы

Инструменты аудита (сравнение)

Инструмент	Тип	Что находит	Точность
axe-core	Автоматический	~30-50% проблем	Высокая (мало ложных срабатываний)
Lighthouse	Автоматический	~30% + best practices	Средняя
WAVE	Автоматический + визуальный	Контраст, структура, ARIA	Высокая
Ручное тестирование	—	~70-80% проблем	Зависит от экспертизы
Юзер-тесты с людьми	—	UX-проблемы, неудобство	Максимальная

Главный вывод: автоматика — необходимый минимум, но никогда не достаточна. Только реальный пользователь скринридера скажет, удобно это или нет.

Связанное

- Браузер и HTTP — Critical Rendering Path, как браузер строит DOM (основа для понимания Accessibility Tree)
- Тестирование React — jest-axe, интеграция а11y-тестов в юнит/интеграционные тесты
- Безопасность фронтенда — XSS через innerHTML и безопасная работа с пользовательским вводом (пересекается с а11y)
- Web API — ARIA, Accessibility Object Model (AOM)
- Сборщики и инструменты — ESLint, плагины для а11y-линтинга

React: рендеринг и производительность

React рендеринг и производительность

Понимание того, как React решает ЧТО перерисовать — ключевой маркер Middle-разработчика. На собеседовании спросят не «как использовать useМемо», а «когда он НЕ нужен и почему».

1. Reconciliation — как React понимает, что менять

Проблема

При каждом изменении состояния React вызывает рендер компонента. Но менять весь DOM целиком — медленно. React должен понять: что именно изменилось и какие DOM-операции минимально необходимы.

Алгоритм reconciliation (сверки)

React строит **виртуальное дерево** (Virtual DOM) — легковесное JS-представление реального DOM. При обновлении:

1. Создаётся **новое** виртуальное дерево
2. Сравнивается с **предыдущим** (diffing)
3. Вычисляется **минимальный набор изменений**
4. Применяется к реальному DOM (commit)

Алгоритм diffing (два допущения):

1. **Элементы разных типов** → полная замена поддерева
2. **Ключи (key)** → React считает элементы с одинаковым ключом «теми же», даже если их порядок изменился

```
// Без key: React пересоздаст все <li> при изменении порядка  
{items.map(item => <li>{item.name}</li>)}
```

```
// С key: React переместит DOM-узлы, а не пересоздаст  
{items.map(item => <li key={item.id}>{item.name}</li>)}
```

Почему index как key — плохо

```
// Исходный список: [A, B, C] → key: [0, 1, 2]  
// Добавили в начало: [D, A, B, C] → key: [0, 1, 2, 3]  
// React думает: элемент с key=0 теперь D (а был A), key=1 теперь A (а  
был B)...\br/>// Результат: пересоздаёт ВСЕ элементы, теряет состояние (фокус,  
анимации)
```

Всегда используй стабильные ID из данных. `crypto.randomUUID()` в рендере — тоже плохо (новый key каждый рендер = пересоздание).

2. Fiber — переписанный движок (React 16+)

Что такое Fiber

Как читать «Fiber»: читай Fiber как «лёгкая заметка о том, что нужно сделать компоненту» — React создаёт дерево таких заметок вместо реального DOM. Каждая заметка знает: какой это компонент, что изменилось, и можно ли прервать работу над ней. Мнемоника: «Fiber — это не дерево, а список дел, который можно отложить».

Fiber — это **переписанный reconciliation-движок**, решающий главную проблему старого React: **блокировка главного потока**.

Старый React (Stack Reconciler):

- Рекурсивный обход дерева — синхронный
- Пока React считает diff — страница не отвечает на клики
- На большом дереве — заметные фриззы

Fiber:

- Дерево обрабатывается **инкрементально** — по кусочкам
- React может **прервать** работу, обработать срочное событие и **продолжить**
- Приоритеты: пользовательский ввод > анимация > загрузка данных

Две фазы

Фаза	Название	Что делает	Можно прервать?
Render	Reconciliation	Строит новое дерево, вычисляет изменения	Да
Commit	Применение к DOM	Меняет реальный DOM, вызывает useEffect	Нет

```
// Render-фаза может быть прервана — поэтому:
// - Не делай side-effect'ов в теле компонента
// - Не мутируй переменные
// - Не делай fetch в render

// Side-effect в render
function Bad() {
  analytics.track('page_view'); // вызовется несколько раз!
  return <div>...</div>;
}

// Side-effect — в useEffect
function Good() {
  useEffect(() => { analytics.track('page_view'); }, []);
  return <div>...</div>;
}
```

Приоритеты в Fiber (Lane Model)

React назначает обновлениям «полосы» (lanes) — чем выше приоритет, тем раньше обработается:

- **Синхронный** (SyncLane): `useLayoutEffect`, `componentDidMount`
- **Высокий**: пользовательский ввод, анимации
- **По умолчанию**: `useEffect`, `setState`
- **Низкий**: загрузка данных, ленивая загрузка
- **Idle**: `useTransition`, отложенные обновления

3. Что вызывает ререндер

1. Изменение state

```
function Counter() {
  const [count, setCount] = useState(0);
  // setCount(1) → ререндер Counter
}
```

2. Изменение props (от родителя)

```
function Parent() {
  const [count, setCount] = useState(0);
  return <Child count={count} />; // Ререндер Parent → ререндер Child
}
```

3. Изменение контекста

```
const ThemeContext = createContext('light');
// Провайдер изменил value → все потребители ререндерятся
```

4. Ререндер родителя (даже без изменения props!)

```
function Parent() {
  const [, force] = useState(0);
  return (
    <>
      <button onClick={() => force(x => x + 1)}>Ререндер</button>
      <Child /> { /* ← тоже ререндерится! Даже без пропсов */ }
    </>
  );
}
```

4. Инструменты предотвращения ререндеров

React.memo — поверхностное сравнение пропсов

```
// Без мемо: ререндерится всегда при ререндере родителя
function ExpensiveChild({ data }) {
  return <div>{/* рендер стоит дорого */}</div>;
}

// С мемо: ререндерится только при изменении пропсов
const ExpensiveChild = React.memo(({ data }) => {
  return <div>{/* рендер стоит дорого */}</div>;
});

ExpensiveChild.displayName = 'ExpensiveChild';
```

Важно: React.memo сравнивает пропсы через Object.is. Объекты и функции — новые ссылки каждый рендер → мемо НЕ помогает.

```
//      мемо бесполезен — handleClick и style новые каждый рендер
<MemoChild
  onClick={() => doSomething(id)}      // новая ссылка
  style={{ color: 'red' }}             // новый объект
/>

//      useCallback + useMemo делают мемо рабочим
const handleClick = useCallback(() => doSomething(id), [id]);
const style = useMemo(() => ({ color: 'red' })), [id]);
<MemoChild onClick={handleClick} style={style} />
```

useMemo — кэширование вычислений

Как читать `useMemo(() => вычисление, [зависимости])`: читай как «запомни результат этого вычисления и пересчитай только когда что-то из списка зависимостей изменится». Массив `[deps]` — это НЕ «выполнить когда изменится», а «НЕ пересчитывать ПОКА не изменится». Мнемоника: «useMemo — запомни и не трогай, пока я не скажу».

```
// . Тяжёлое вычисление при КАЖДОМ рендере
function List({ items, filter }) {
  const filtered = items.filter(i => i.category === filter); // каждый раз!
  return filtered.map(i => <Item key={i.id} {...i} />);
}

//      Пересчитываем только при изменении зависимостей
function List({ items, filter }) {
  const filtered = useMemo(
    () => items.filter(i => i.category === filter),
    [items, filter]
  );
  return filtered.map(i => <Item key={i.id} {...i} />);
}
```

Когда useMemo НЕ нужен:

- Вычисление дешёвое (`arr.length`, конкатенация строк)
- Зависимости меняются каждый рендер (мемоизация бесполезна)
- Мемоизация дороже самого вычисления

useCallback — стабильная ссылка на функцию

```
//      Новая функция каждый рендер — мемо-компоненты ререндерятся
function Parent() {
  const handleClick = () => doSomething(id);
  return <MemoChild onClick={handleClick} />;
}

//      Та же ссылка при тех же зависимостях
function Parent() {
  const handleClick = useCallback(() => doSomething(id), [id]);
  return <MemoChild onClick={handleClick} />;
}
```

useCallback != useMemo для функций:

```
useCallback(fn, deps)    // ≡ useMemo(() => fn, deps)
```

5. Три закона оптимизации React

Закон 1: Не оптимизируй преждевременно

Напиши работающий код → измерь → оптимизируй ТОЛЬКО узкие места.
Большинство приложений не имеют проблем с производительностью React.

Закон 2: Двигай состояние вниз

```
// Состояние в родителе → ререндерится ВСЁ дерево
function App() {
  const [count, setCount] = useState(0);
  return (
    <>
      <Header />
      <Sidebar />
      <Counter count={count} onInc={() => setCount(c => c + 1)} />
      <Footer />
    </>
  );
}
// Каждый setCount → ререндер Header, Sidebar, Footer (они не
изменились!)

// Состояние там, где используется
function App() {
  return (
    <>
      <Header />
      <Sidebar />
      <CounterSection /> {/* состояние внутри */}
      <Footer />
    </>
  );
}

function CounterSection() {
  const [count, setCount] = useState(0);
  return <Counter count={count} onInc={() => setCount(c => c + 1)} />;
}
// setCount → ререндер только CounterSection и Counter
```


Закон 3: Children как паттерн

```
//    Ререндер родителя → ререндер детей (через props)
function Parent() {
  const [count, setCount] = useState(0);
  return (
    <ExpensiveChild count={count} />
  );
}

//    Children не ререндерятся при изменении состояния родителя!
function Parent({ children }) {
  const [count, setCount] = useState(0);
  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>+1 ({count})</button>
      {children} {/* ← Не ререндерится! */}
    </div>
  );
}
function App() {
  return (
    <Parent>
      <ExpensiveChild />
    </Parent>
  );
}
```

6. Инструменты профилирования

React DevTools Profiler

1. Открой React DevTools → вкладка Profiler
2. Нажми запись → выполни действие → останови
3. **Flamegraph**: видно, какие компоненты рендерились и сколько времени заняли
4. **Ranked**: сортировка по времени — сразу видны самые дорогие

На что смотреть:

- Компоненты с серым фоном — не рендерились (хорошо)
- Компоненты с жёлтым/красным — долгий рендер
- **Commit**: каждый commit — это одно обновление DOM

Почему компонент ререндерился?

В Profiler нажми на компонент → «Why did this render?» — React покажет, какое свойство изменилось.

Performance API в коде

```
// Измерение конкретного рендера
function ExpensiveComponent() {
  const start = performance.now();
  // ... рендер ...
  useEffect(() => {
    const duration = performance.now() - start;
    if (duration > 16) { // больше одного кадра (60fps)
      console.warn(`Slow render: ${duration.toFixed(1)}ms`);
    }
  });
}
```

7. useTransition и useDeferredValue (React 18+)

Проблема: дорогой рендер блокирует UI

```
function App() {
  const [query, setQuery] = useState('');
  const filtered = heavyFilter(items, query); // может занять >100ms

  return (
    <>
      <input value={query} onChange={e =>
setQuery(e.target.value)} />
      {/* Пока фильтруется – input не отвечает! */}
      <List items={filtered} />
    </>
  );
}
```

Решение: useTransition — пометить обновление как низкоприоритетное

Как читать *startTransition(() => ...)*: читай как «React, это обновление не срочное — если пользователь кликнет или начнёт печатать, брось это и займись клиентом». Внутри *startTransition* ты МЕНЯЕШЬ состояние, но React может отложить, прервать или перезапустить этот рендер. Мнемоника: «startTransition — это фоновый режим для setState».

```
function App() {
  const [query, setQuery] = useState('');
  const [deferredQuery, setDeferredQuery] = useState('');
  const [isPending, startTransition] = useTransition();

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setQuery(e.target.value); // высокий
    // приоритет — input обновляется сразу
    startTransition(() => {
      setDeferredQuery(e.target.value); // низкий
      // приоритет — фильтрация может подождать
    });
  };

  const filtered = useMemo(
    () => heavyFilter(items, deferredQuery),
    [items, deferredQuery]
  );

  return (
    <>
      <input value={query} onChange={handleChange} />
      {isPending && <Spinner />}
      <List items={filtered} />
    </>
  );
}
```

useDeferredValue — проще, когда нет контроля над setState

```
function App() {
  const [query, setQuery] = useState('');
  const deferredQuery = useDeferredValue(query); // React сам решит,
  // когда обновить

  const filtered = useMemo(
    () => heavyFilter(items, deferredQuery),
    [items, deferredQuery]
  );

  return (
    <>
      <input value={query} onChange={e =>
setQuery(e.target.value)} />
      <List items={filtered} style={{ opacity: query !== deferredQ
query ? 0.5 : 1 }} />
    </>
  );
}
```

8. Распространённые ошибки

Ошибка	Почему плохо	Как исправить
useMemo везде	Мемоизация не бесплатна — память + сравнение	Только для реально дорогих вычислений
useCallback без мемо	Дочерний компонент всё равно ререндерится	мемо на дочернем компоненте
Состояние в корне	Ререндер всего приложения	Двигать состояние вниз
Анонимные компоненты в рендере	function Child() { ... } внутри Parent — новый тип каждый рендер	Вынести наружу или использовать useMemo
key={Math.random()}	Каждый рендер — новый key → пересоздание DOM	Стабильный ID из данных
useEffect без зависимостей	useEffect(() => { ... }) — на каждый рендер	Явно указать [deps]
new Date() в зависимостях	Каждый рендер — новый объект → бесконечный useEffect	useMemo или вынести за компонент

9. Чек-лист: когда реально нужна оптимизация

Только если:

- [] Есть измеримый фриз (>100ms) при взаимодействии
- [] Профилировщик React DevTools показывает жёлтые/красные компоненты
- [] Пользователи жалуются на лаги

Оптимизировать надо саму причину, а не симптомы:

1. Сначала — архитектура (двигать состояние вниз, children-паттерн)
2. Потом — мемо + useCallback/useMemo
3. В последнюю очередь — useTransition

React 19 и React Compiler

React Forget (Compiler)

React Compiler (ранее React Forget) — компилятор, который **автоматически** применяет мемоизацию. Он анализирует код на этапе сборки и добавляет useMemo, useCallback, React.memo там, где это действительно нужно.

Что компилятор делает:

- Автоматически мемоизирует значения и коллбэки

- Определяет, где нужен `React.memo`
- Не требует ручного указания зависимостей
- Работает с правилами хуков (ESLint-плагин `react-compiler`)

Что компилятор НЕ делает:

- Не оптимизирует дорогие вычисления без мемоизации
- Не трогает `useRef` (он вне модели мемоизации)
- Не мемоизирует значения, передаваемые в нативные DOM-элементы

Что станет не нужно:

- `useCallback` — в 90% случаев
- `useMemo` — в 80% случаев
- `React.memo` — почти полностью

Что останется нужным:

- `useMemo` для действительно дорогих вычислений (сортировка 10000 элементов)
- `useCallback` при передаче в сторонние библиотеки, не знающие о компиляторе
- `useRef` — компилятор не заменяет refs

Новые хуки React 19

- **`useOptimistic`** — оптимистичные обновления UI (форма «комментарий появился сразу»)
- **`useActionState`** — управление состоянием Server Actions
- **`useFormStatus`** — статус отправки формы (`pending`, `data`)

Что изменилось в рендеринге

- **Document Metadata:** `<title>`, `<meta>`, `<link>` теперь работают в любом компоненте (не только в `<head>`)
- **Pre-rendering API:** `prerender()` и `prerenderToNodeStream()` для статической генерации

Связанное

- TypeScript продвинутый
- Управление состоянием
- Архитектура React компонентов
- Тестирование React
- Виртуализация рендеринга
- Браузер и HTTP

Управление состоянием

Главный сдвиг от Junior к Middle: перестать класть всё в Redux и разделить серверное и клиентское состояние. На собеседовании проверяют именно это разделение, а не знание API конкретной библиотеки.

1. Серверное vs клиентское состояние

Тип	Что это	Кто владелец	Пример
Серверное	Данные с бэкенда	Сервер (мы кэшируем копию)	Список пользователей, события, заказы
Клиентское	UI-состояние	Только фронтенд	Модальное окно открыто, тема, фильтр

Почему это разделение важно:

```
// Junior: всё в Redux/Zustand, включая серверные данные
// Проблемы:
// - Дублирование логики (fetch + store + очистка кэша)
// - Нет инвалидации кэша
// - Каждый компонент должен знать, загружены ли данные
const useUsers = () => {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    fetch('/api/users').then(r =>
r.json()).then(setUsers).finally(() => setLoading(false));
  }, []);
  return { users, loading };
};

// Middle: TanStack Query для серверного, Zustand – только для
клиентского
function UsersList() {
  const { data: users, isLoading, error } = useQuery({
    queryKey: ['users'],
    queryFn: () => fetch('/api/users').then(r => r.json()),
  });
  // Всё остальное (loading, error, кэш, refetch) – из коробки
}
```

2. Серверное состояние: TanStack Query (React Query)

Зачем нужен

TanStack Query решает проблемы, которые **неизбежны** при ручном управлении запросами:

1. **Кэширование** — не делать повторный запрос, если данные свежие
2. **Инвалидация** — обновить данные после мутации
3. **Фоновая перезагрузка** — данные могли устареть, пока пользователь смотрел
4. **Retry** — автоматически повторить при ошибке
5. **Dedup** — два компонента запрашивают одни данные → один запрос

Базовое использование

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

// Запрос на чтение
function EventsList() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['events'],
    queryFn: () => fetch('/api/events').then(r => r.json()),
    staleTime: 5 * 60 * 1000, // 5 минут считаем данные свежими
    gcTime: 30 * 60 * 1000,   // 30 минут держим в кэше после unmount
  });

  if (isLoading) return <Spinner />;
  if (error) return <Error message={error.message} />;
  return data.map(e => <EventCard key={e.id} event={e} />);
}

// Мутация (создание/обновление/удаление)
function CreateEventForm() {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    mutationFn: (newEvent: CreateEventDTO) =>
      fetch('/api/events', {
        method: 'POST',
        body: JSON.stringify(newEvent),
      }).then(r => r.json()),
    onSuccess: () => {
      // Инвалидируем кэш – список обновится автоматически
      queryClient.invalidateQueries({ queryKey: ['events'] });
    },
  });

  return (
    <form onSubmit={e => { e.preventDefault(); mutation.mutate(formData); }}>
      <button disabled={mutation.isPending}>
        {mutation.isPending ? 'Создаю...' : 'Создать'}
      </button>
      {mutation.isError && <p>Ошибка: {mutation.error.message}</p>}
    </form>
  );
}
```


Ключи (queryKey) — как работает кэш

```
// Ключ — это массив. Первый элемент — категория, остальные — параметры
useQuery({ queryKey: ['events'] }); // все события
useQuery({ queryKey: ['events', eventId] }); // конкретное
useQuery({ queryKey: ['events', 'category', categoryId] }); // по категории

// Инвалидация частичная:
queryClient.invalidateQueries({ queryKey: ['events'] }); // все events*
queryClient.invalidateQueries({ queryKey: ['events', id] }); // только конкретное
```

Оптимистичное обновление (Optimistic Update)

Как читать цепочку *onMutate* → *onError* → *onSettled*: «до запроса (*onMutate*) — сохрани старые данные и оптимистично обнови UI; если запрос упал (*onError*) — откати по сохранённому *context.previous*; в любом случае (*onSettled*) — перезапроси актуальные данные с сервера». *context* — это объект, который *onMutate* возвращает, а *onError* и *onSettled* получают третьим аргументом.

```
const toggleFavorite = useMutation({
  mutationFn: (eventId: string) => api.toggleFavorite(eventId),
  onMutate: async (eventId) => {
    // 1. Отменяем фоновый refetch
    await queryClient.cancelQueries({ queryKey: ['events', eventId] });

    // 2. Сохраняем на случай отката
    const previous = queryClient.getQueryData(['events', eventId]);
    // 3. Оптимистично обновляем UI
    queryClient.setQueryData(['events', eventId], (old: Event) => ({
      ...old,
      isFavorite: !old.isFavorite,
    }));
    return { previous }; // для отката
  },
  onError: (err, eventId, context) => {
    // Откат при ошибке
    queryClient.setQueryData(['events', eventId], context.previous);
  },
  onSettled: () => {
    // В любом случае — перезагрузить актуальные данные с сервера
    queryClient.invalidateQueries({ queryKey: ['events', eventId] });
  },
});
```

Паттерн: prefetch для мгновенной навигации

```
function EventListItem({ event }) {
  const queryClient = useQueryClient();

  return (
    <Link
      to={` /events/${event.id}`}
      onMouseEnter={() => {
        // Загружаем данные ДО клика – страница откроется
        мгновенно
        queryClient.prefetchQuery({
          queryKey: ['events', event.id],
          queryFn: () => fetchEvent(event.id),
        });
      }}
    >
      {event.title}
    </Link>
  );
}
```

Бесконечная прокрутка (useInfiniteQuery)

Как читать `getNextPageParam: (lastPage) => lastPage.nextCursor ??`

undefined: «получи следующую страницу — из последнего ответа сервера вытащи курсор *nextCursor*, а если его нет (*?? undefined*), то страниц больше нет». Паттерн *cursor-based* пагинации: сервер возвращает курсор на следующую страницу, а не номер страницы — это защищает от пропуска/дублирования элементов при добавлении новых данных.

```
const { data, fetchNextPage, hasNextPage, isFetchingNextPage } = useInfiniteQuery({
  queryKey: ['events', filter],
  queryFn: ({ pageParam = 0 }) => fetchEvents({ cursor: pageParam, filter }),
  getNextPageParam: (lastPage) => lastPage.nextCursor ?? undefined,
  initialPageParam: 0,
});

// data.pages – массив страниц, data.pages[0].events – первая страница
const allEvents = data?.pages.flatMap(page => page.events) ?? [];

// Кнопка «Загрузить ещё»
<button onClick={() => fetchNextPage()} disabled={!hasNextPage}
  {isFetchingNextPage ? 'Загружаю...' : 'Ещё'}
>
</button>
```

3. Клиентское состояние: Zustand

Почему Zustand, а не Redux/Context

Инструмент	Плюсы	Минусы
useState	Просто	Не делится между компонентами
Context	Встроен в React	Ререндер ВСЕХ потребителей при изменении
Redux Toolkit	Мощный, middleware, devtools	Много бойлерплейта, тяжёлый для простого
Zustand	Минимум кода, нет провайдеров, селекторы	Нет встроенного middleware

Базовое хранилище

Как читать `create<UIStore>((set) => ({...}))`: «создай хранилище типа `UIStore`, передав функцию, которая получает `set` — единственный способ изменить состояние — и возвращает объект с начальными значениями и экшенами». `set` работает как `setState` в React: можно передать объект-замену или функцию `(state) => ({...})` для обновления на основе предыдущего состояния.

```
import { create } from 'zustand';

interface UIStore {
  isSidebarOpen: boolean;
  activeModal: string | null;
  toggleSidebar: () => void;
  openModal: (id: string) => void;
  closeModal: () => void;
}

const useUIStore = create<UIStore>((set) => ({
  isSidebarOpen: true,
  activeModal: null,
  toggleSidebar: () => set(state => ({ isSidebarOpen: !state.isSidebar
Open })),
  openModal: (id) => set({ activeModal: id }),
  closeModal: () => set({ activeModal: null }),
})));
```

Селекторы — рендер только при изменении нужного поля

```
// Рендер при ЛЮБОМ изменении store
function Header() {
  const { isSidebarOpen, toggleSidebar } = useUIStore();
  // Рендерится даже когда открывается модалька!
}

// Рендер только при изменении isSidebarOpen
function Header() {
  const isSidebarOpen = useUIStore(state => state.isSidebarOpen);
  const toggleSidebar = useUIStore(state => state.toggleSidebar);
}

// Селектор с shallow-сравнением (несколько полей)
import { useShallow } from 'zustand/react/shallow';

function Header() {
  const { isSidebarOpen, activeModal } = useUIStore(
    useShallow(state => ({ isSidebarOpen: state.isSidebarOpen, activeModal: state.activeModal })))
};
}
```

Persist — сохранение в localStorage

```
import { persist } from 'zustand/middleware';

interface CartStore {
  items: CartItem[];
  addItem: (item: CartItem) => void;
}

const useCartStore = create<CartStore>()(
  persist(
    (set) => ({
      items: [],
      addItem: (item) => set(state => ({ items: [...state.items, item] })),
    }),
    {
      name: 'cart-storage', // ключ в localStorage
      partialize: (state) => ({ items: state.items }), // что
сохранять
    }
  )
);
```

Реальный пример: корзина билетов

```
interface TicketCartStore {  
  tickets: Map<string, { quantity: number; price: number }>;  
  addTicket: (eventId: string, quantity: number, price: number) =>  
void;  
  removeTicket: (eventId: string) => void;  
  getTotal: () => { count: number; price: number };  
}  
  
const useTicketCart = create<TicketCartStore>((set, get) => ({  
  tickets: new Map(),  
  addTicket: (eventId, quantity, price) =>  
    set(state => {  
      const newTickets = new Map(state.tickets);  
      newTickets.set(eventId, { quantity, price });  
      return { tickets: newTickets };  
    }),  
  removeTicket: (eventId) =>  
    set(state => {  
      const newTickets = new Map(state.tickets);  
      newTickets.delete(eventId);  
      return { tickets: newTickets };  
    }),  
  getTotal: () => {  
    const { tickets } = get();  
    let count = 0, price = 0;  
    tickets.forEach(t => { count += t.quantity; price += t.quantity  
* t.price; });  
    return { count, price };  
  },  
}));
```

4. Когда что использовать — дерево решений

Данные приходят с сервера?

- └─ Да → TanStack Query
 - └─ Список → useQuery
 - └─ Бесконечный скролл → useInfiniteQuery
 - └─ Изменение данных → useMutation + invalidateQueries
- └─ Нет (клиентское состояние)
 - └─ Локальное (1 компонент) → useState
 - └─ Ветка дерева (родитель + дети) → useState + props
 - └─ Много потребителей в разных частях → Zustand
 - └─ Тема, язык интерфейса → Context (редко меняется)
 - └─ URL (фильтры, страницы) → useSearchParams / React Router

Что НЕЛЬЗЯ хранить в Zustand

- Данные с сервера (пользователи, события, заказы) — для этого TanStack Query
- Состояние форм — используй React Hook Form (uncontrolled)
- URL-параметры — используй useSearchParams
- Производные данные — `const fullName = firstName + ' ' + lastName` в рендере, не в store

5. URL как состояние (React Router)

```
// Фильтры и пагинация должны быть в URL, а не в Zustand
function EventsPage() {
  const [searchParams, setSearchParams] = useSearchParams();
  const category = searchParams.get('category') || 'all';
  const page = Number(searchParams.get('page')) || 1;

  const { data } = useQuery({
    queryKey: ['events', { category, page }],
    queryFn: () => fetchEvents({ category, page }),
  });

  return (
    <>
      <CategoryFilter
        value={category}
        onChange={c => setSearchParams({ category: c, page: '1' })}
      />
      <EventList events={data?.events} />
      <Pagination
        page={page}
        onChange={p => setSearchParams({ category, page:
String(p) })}
      />
    </>
  );
}
```

Почему URL, а не store:

- Фильтры можно скопировать и поделиться ссылкой
- Работает кнопка «Назад»
- SSR: сервер видит фильтры и рендерит правильную страницу

6. Антипаттерны

Антипаттерн	Почему плохо	Как исправить
Всё в Redux/Zustand/ Effector	Серверные данные дублируются, кэш ручной	TanStack Query для серверных данных
Prop drilling (3+ уровня)	Хрупко, трудно рефакторить	Context или Zustand
Context для часто меняющихся данных	Ререндер всех потребителей	Zustand с селекторами
useEffect для производного состояния	<code>useEffect(() => setFullName(first + ' ' + last), [first, last])</code>	Просто const в рендере
Стейт для URL-параметров	Ломает навигацию, шаринг ссылок	<code>useSearchParams</code>

Связанное

- React рендеринг и производительность
- Виртуализация рендеринга
- Архитектура React компонентов
- TypeScript продвинутый
- Архитектура React компонентов

Архитектура React компонентов

Архитектура React компонентов

Сдвиг от «накидать пропсов» к «спроектировать API компонента». На собеседовании отличают Middle от Junior по тому, как кандидат проектирует переиспользуемые компоненты.

1. Composition over Configuration

Проблема конфигурационного подхода

```
// Junior: компонент-божество с 20 пропсами
<Modal
  isOpen={true}
  title="Удалить событие?"
  body="Вы уверены? Это действие нельзя отменить."
  confirmText="Удалить"
  cancelText="Отмена"
  onConfirm={handleDelete}
  onCancel={closeModal}
  size="medium"
  showCloseButton={true}
  closeOnOverlay={true}
  footer={<><button>Ещё что-то</button></>}
  headerIcon={<TrashIcon />}
  animation="fade"
  // ... и так далее
/>
```

Проблемы:

- Каждое новое требование → новый проп
- Невозможно кастомизировать непредусмотренное
- Типизация раздувается
- Сложно понять API без документации

Решение: композиция

```
// Middle: компонент – контейнер, собираемый из частей
<Modal isOpen={isOpen} onClose={closeModal}>
  <Modal.Header>
    <TrashIcon />
    <Modal.Title>Удалить событие?</Modal.Title>
  </Modal.Header>
  <Modal.Body>
    Вы уверены? Это действие нельзя отменить.
  </Modal.Body>
  <Modal.Footer>
    <Button variant="secondary" onClick={closeModal}>Отмена</Button>
    <Button variant="danger" onClick={handleDelete}>Удалить</Button>
  </Modal.Footer>
</Modal>
```

Реализация:


```
// Modal.tsx
interface ModalProps { isOpen: boolean; onClose: () => void; children: R
eact.ReactNode; }
interface ModalSectionProps { children: React.ReactNode; className?: str
ing; }

function Modal({ isOpen, onClose, children }: ModalProps) {
  if (!isOpen) return null;
  return createPortal(
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal-content" onClick={e => e.stopPropagation()
on()}>
        {children}
      </div>
    </div>,
    document.body
  );
}

Modal.Header = ({ children }: ModalSectionProps) => <div className="modal
-header">{children}</div>;
Modal.Title = ({ children }: ModalSectionProps) => <h2 className="modal-
title">{children}</h2>;
Modal.Body = ({ children }: ModalSectionProps) => <div className="modal-
body">{children}</div>;
Modal.Footer = ({ children }: ModalSectionProps) => <div className="modal
-footer">{children}</div>;

export { Modal };
```

2. Compound Components

Compound Components — когда несколько компонентов работают вместе и разделяют неявное состояние через Context.

Как читать `Modal.Header = ({ children }) => ...`: читай как «прикрепи подкомпонент Header прямо к функции Modal как свойство». Это не наследование и не класс — обычная функция, к которой «приклеили» другие функции-компоненты. `Modal.Header` живёт в модуле `Modal` и не существует отдельно от него. Мнемоника: «точка между Modal и Header — это клей, а не иерархия».

Проблема без Compound

```
// Взрыв пропсов: каждый элемент табов должен знать всё
<Tabs
  items={[
    { id: 'tab1', label: 'Информация', content: <InfoPanel /> },
    { id: 'tab2', label: 'Билеты', content: <TicketsPanel /> },
  ]}
  activeTab="tab1"
  onChange={handleChange}
/>
```

Решение: Compound Components

```
// Интуитивное API, гибкая композиция
<Tabs defaultTab="info" onChange={handleChange}>
  <Tabs.List>
    <Tabs.Tab id="info">Информация</Tabs.Tab>
    <Tabs.Tab id="tickets">Билеты</Tabs.Tab>
    <Tabs.Tab id="reviews">Отзывы</Tabs.Tab>
  </Tabs.List>
  <Tabs.Panel id="info"><InfoPanel /></Tabs.Panel>
  <Tabs.Panel id="tickets"><TicketsPanel /></Tabs.Panel>
  <Tabs.Panel id="reviews"><ReviewsPanel /></Tabs.Panel>
</Tabs>
```

Реализация:

```

import { createContext, useContext, useState } from 'react';

interface TabsContextValue {
  activeTab: string;
  setActiveTab: (id: string) => void;
}

const TabsContext = createContext<TabsContextValue | null>(null);
const useTabs = () => {
  const ctx = useContext(TabsContext);
  if (!ctx) throw new Error('Tabs.* must be used inside <Tabs>');
  return ctx;
};

function Tabs({ defaultTab, onChange, children }: {
  defaultTab: string;
  onChange?: (id: string) => void;
  children: React.ReactNode;
}) {
  const [activeTab, setActiveTab] = useState(defaultTab);
  const handleChange = (id: string) => { setActiveTab(id); onChange?.(id); };
  return (
    <TabsContext.Provider value={{ activeTab, setActiveTab: handleChange }}>
      {children}
    </TabsContext.Provider>
  );
}

Tabs.List = ({ children }: { children: React.ReactNode }) => (
  <div role="tablist" className="tabs-list">{children}</div>
);

Tabs.Tab = ({ id, children }: { id: string; children: React.ReactNode })
=> {
  const { activeTab, setActiveTab } = useTabs();
  return (
    <button
      role="tab"
      aria-selected={activeTab === id}
      onClick={() => setActiveTab(id)}
      className={activeTab === id ? 'tab active' : 'tab'}
    >
      {children}
    </button>
  );
};

Tabs.Panel = ({ id, children }: { id: string; children: React.ReactNode })

```

```
) => {  
  const { activeTab } = useTabs();  
  if (activeTab !== id) return null;  
  return <div role="tabpanel">{children}</div>;  
};
```

3. Headless Components

Headless компонент управляет состоянием, accessibility и логикой, но НЕ рендерит UI — он отдаёт всё через render-prop или хук.

Когда нужен

Когда нужно переиспользовать **поведение** с разным внешним видом:

- Модалки, дропдауны, тултипы, аккордеоны
- Компоненты из дизайн-системы vs страницы лендинга — разная стилизация, одна логика

Пример: Headless Dropdown

Как читать *children*: (api) => *ReactNode* (render-prop): читай как «я дам тебе инструменты (api), а ты реши как это будет выглядеть». *children* здесь — не JSX, а ФУНКЦИЯ, которая получает объект с методами и возвращает JSX. Это позволяет одному компоненту отдавать логику, а другому — внешний вид. Мнемоника: «children-функция — это курьер: приносит инструменты, а рисуешь ты сам».

```
// useDropdown.ts – хук с логикой, без UI
function useDropdown() {
  const [isOpen, setIsOpen] = useState(false);
  const [activeIndex, setActiveIndex] = useState(0);
  const triggerRef = useRef<HTMLButtonElement>(null);
  const listRef = useRef<HTMLUListElement>(null);

  const toggle = () => setIsOpen(o => !o);
  const close = () => { setIsOpen(false); setActiveIndex(0); };

  // Закрытие по клику снаружи
  useEffect(() => {
    if (!isOpen) return;
    const handler = (e: MouseEvent) => {
      if (!triggerRef.current?.contains(e.target as Node) &&
        !listRef.current?.contains(e.target as Node)) {
        close();
      }
    };
    document.addEventListener('mousedown', handler);
    return () => document.removeEventListener('mousedown', handler);
  }, [isOpen]);

  // Клавиатурная навигация
  const handleKeyDown = (e: React.KeyboardEvent, itemCount: number) =>
  {
    switch (e.key) {
      case 'ArrowDown': setActiveIndex(i => (i + 1) % itemCount); break;
      case 'ArrowUp': setActiveIndex(i => (i - 1 + itemCount) % itemCount); break;
      case 'Enter': /* выбрать активный элемент */ close(); break;
      case 'Escape': close(); break;
    }
  };

  return { isOpen, activeIndex, triggerRef, listRef, toggle, close, handleKeyDown };
}

// HeadlessDropdown.tsx – render-prop
function HeadlessDropdown({ children: { children: (api: ReturnType<typeof useDropdown>) => ReactNode } }) {
  const api = useDropdown();
  return <>{children(api)}</>;
}

// Использование: разная стилизация – одно поведение
<HeadlessDropdown>
  ({ isOpen, toggle, close }) => (
```

```

    <>
      <button onClick={toggle}>Меню {isOpen ? '▲' : '▼'}</button>
      {isOpen && <div className="dropdown-menu" onClick={close}
    >...</div>}
    </>
  )}
</HeadlessDropdown>

```

Headless UI (библиотека Tailwind)

```

import { Listbox } from '@headlessui/react';

// Headless UI даёт accessibility из коробки (ARIA-атрибуты, клавиатура,
focus trap)
function EventCategorySelect({ value, onChange }) {
  return (
    <Listbox value={value} onChange={onChange}>
      <Listbox.Button>{value || 'Выбрать категорию'}</Listbox.Button>
      <Listbox.Options>
        {categories.map(c => (
          <Listbox.Option key={c} value={c}>
            {{{ active, selected }} => (
              <li className={` ${active ? 'bg-blue-100' :
                ''} ${selected ? 'font-bold' : ''}`}>
                {c}
              </li>
            )}
          </Listbox.Option>
        ))}
      </Listbox.Options>
    </Listbox>
  );
}

```

4. Slots Pattern (подход Next.js / shadcn/ui)

Идея от Radix UI и shadcn/ui: компонент принимает «слоты» — фрагменты JSX как пропсы.

```
interface CardProps {
  header?: ReactNode;
  footer?: ReactNode;
  children: ReactNode;
}

function Card({ header, footer, children }: CardProps) {
  return (
    <div className="card">
      {header} && <div className="card-header">{header}</div>
      <div className="card-body">{children}</div>
      {footer} && <div className="card-footer">{footer}</div>
    </div>
  );
}

// Использование
<Card
  header={<><EventIcon /><span>React Conf</span></>}
  footer={<Button>Купить билет</Button>}
>
  <p>Описание события...</p>
  <p>Дата: 01.08.2026</p>
</Card>
```

5. Custom Hooks — вынос логики из компонентов

Принцип: компонент — только UI

```
//    Вся логика в компоненте
function EventPage({ eventId }) {
  const [event, setEvent] = useState(null);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    fetch(`/api/events/${eventId}`).then(r =>
r.json()).then(setEvent).finally(() => setLoading(false));
  }, [eventId]);
  const handleBuy = () => { /* логика покупки */ };
  if (loading) return <Spinner />;
  return <EventView event={event} onBuy={handleBuy} />;
}

//    Логика в хуке, компонент чистый
function useEvent(eventId: string) {
  return useQuery({
    queryKey: ['events', eventId],
    queryFn: () => fetchEvent(eventId),
  });
}

function EventPage({ eventId }) {
  const { data: event, isLoading } = useEvent(eventId);
  if (isLoading) return <Spinner />;
  return <EventView event={event} onBuy={() => buyTicket(eventId)} />;
}
```

Хуки должны быть атомарными

```
//    Монолитный хук
function useEventDashboard() {
  // смешано: запрос, фильтрация, пагинация, WebSocket
}

//    Разделение ответственности
function useEventList(filter: Filter) { ... }           // только загрузка
списка
function useEventFilters()
{ ... }           // только фильтры из URL
function useEventSubscription(eventId: string) { ... } // только
WebSocket
function usePagination() { ... }           // только пагинация
```


6. Принцип единственной ответственности в компонентах

```
//    God Component: 300+ строк, делает всё
function EventDashboard() {
  // запрос данных
  // фильтрация
  // сортировка
  // графики
  // таблица
  // кнопки действий
  // WebSocket
}

//    Разделение на маленькие компоненты
function EventDashboard() {
  return (
    <DashboardLayout>
      <EventStats />
      <EventFilters />
      <EventTable />
      <EventActions />
    </DashboardLayout>
  );
}
```

Правило: если компонент не помещается на одном экране (150-200 строк) — его пора разбивать.

7. Render Props vs Hooks — эволюция

```
// 2016: HOC (Higher-Order Components)
const withAuth = (Component) => (props) => {
  const user = useAuth();
  if (!user) return <Login />;
  return <Component {...props} user={user} />;
};
export default withAuth(Dashboard);

// 2018: Render Props
<Auth>{user => user ? <Dashboard user={user} /> : <Login />}</Auth>

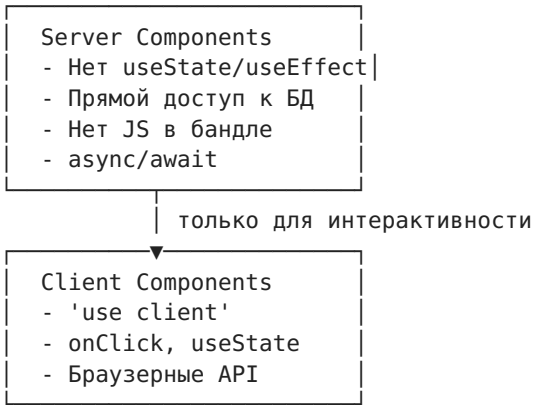
// 2020+: Hooks (победили)
function Dashboard() {
  const user = useAuth();
  if (!user) return <Login />;
  return <DashboardView user={user} />;
}
```

Hooks — текущий стандарт. Render Props и HOC — знать для чтения старого кода.

8. Server Components (React 19 / Next.js 14+)

Новый ментальный сдвиг: не все компоненты выполняются в браузере.

Как читать `'use client'`: читай как «внимание, сборщик! Этот компонент — островок интерактивности, его нужно отправить в браузер». Без этой директивы компонент выполняется на сервере, его JavaScript НЕ попадает в бандл, а в разметку попадает только готовый HTML. Мнемоника: «`'use client'` — это штамп 'в браузер' на компоненте».



```
// Server Component – выполняется на сервере, JS не доходит до клиента
async function EventPage({ params }: { params: { id: string } }) {
  const event = await db.event.findUnique({ where: { id: params.id } })
  ;
  return (
    <div>
      <h1>{event.title}</h1>
      <p>{event.description}</p>
      { /* Клиентский островок для кнопки покупки */ }
      <BuyButton eventId={event.id} price={event.price} />
    </div>
  );
}

// Client Component – только там, где нужна интерактивность
'use client';
function BuyButton({ eventId, price }: { eventId: string; price: number }
) {
  const { mutate, isPending } = useBuyTicket();
  return (
    <button onClick={() => mutate(eventId)} disabled={isPending}>
      {isPending ? 'Покупаю...' : `Купить за ${price}₽`}
    </button>
  );
}
```

9. Чек-лист хорошего API компонента

- [] Можно использовать без чтения исходников (понятные названия пропсов)
- [] Компонуемость: части можно собрать по-разному
- [] Нет «пропсов на все случаи» (признак: >10 пропсов)
- [] Accessibility из коробки (role, aria, keyboard)
- [] Типизация через TypeScript, а не PropTypes
- [] ForwardRef для возможности повесить ref
- [] displayName для Compound Components

Связанное

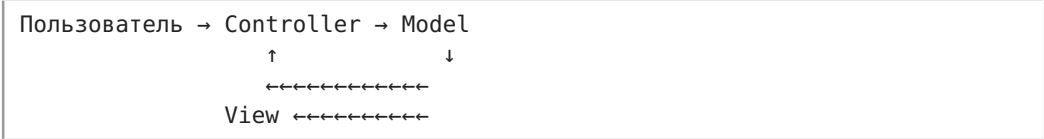
- CSS и стилизация — стилизация компонентов
- Интернационализация и локализация i18n — адаптация компонентов под языки
- React рендеринг и производительность
- TypeScript продвинутый
- Управление состоянием

Архитектурные паттерны

Паттерны, которые спрашивают на собеседованиях фронтенд-разработчиков — от эволюции управления состоянием до классических GoF-паттернов в JavaScript.

1. MVC → Flux → Redux → Zustand/Query

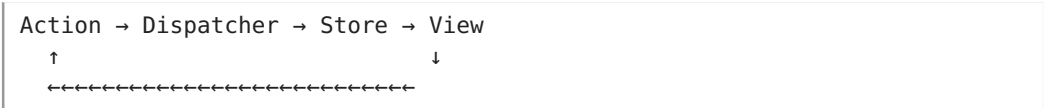
MVC (Model-View-Controller)



Проблема MVC в большом фронтенде:

- Одна Model может обновлять много View
- Одна View может обновлять много Model
- При 50+ компонентов — хаос, невозможно отследить поток данных

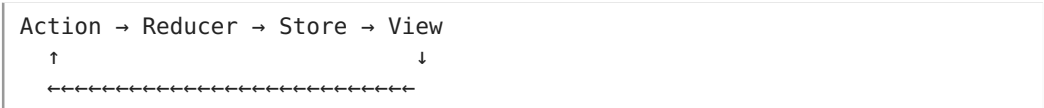
Flux (Facebook, 2014)



Ключевые идеи Flux:

1. **Однонаправленный поток данных:** Action → Dispatcher → Store → View
2. **Dispatcher — единственная точка входа** для всех Actions
3. **Store содержит ВСЁ состояние**, View его не меняет напрямую
4. **View генерирует Actions**, а не мутит Store

Redux (Dan Abramov, 2015)



Что Redux упростил относительно Flux:

- Один Store вместо множества (единое дерево состояния)
- Reducer — чистая функция (state, action) → newState вместо кучи колбэков
- Immutable updates — всегда новый объект, а не мутация

```
// Redux: reducer — чистая функция
```

> **Как читать `(state, action) => newState` (reducer):** читай как «вот текущее состояние и действие — верни НОВОЕ состояние, не мутируя старое». `{ ...state, count: state.count + 1 }` — это не изменение `state`, а создание нового объекта, где все поля скопированы, а `count` заменён. Мнемоника: **«reducer — не меняй, а создай заново»**.

```
function counterReducer(state = { count: 0 }, action) {  
  switch (action.type) {  
    case 'INCREMENT':  
      return { ...state, count: state.count + 1 }; // новый объект  
    case 'DECREMENT':  
      return { ...state, count: state.count - 1 };  
    default:  
      return state;  
  }  
}
```

Почему Redux (без Redux Toolkit) считается устаревшим для новых проектов для новых проектов

- **Много бойлерплейта:** action types, action creators, reducer, connect/mapStateToProps
- **Серверные данные в Redux — антипаттерн:** кэш, инвалидация, дедупликация — всё вручную
- **TanStack Query решает проблемы серверного состояния из коробки**
- **Zustand** — тот же Flux, но без бойлерплейта

Современный стек

Серверное состояние	→ TanStack Query (useQuery, useMutation, кэш)
Клиентское состояние	→ Zustand (UI, тема, корзина)
URL-состояние	→ React Router (useSearchParams)
Локальное состояние	→ useState

2. Observer / PubSub (EventEmitter)

Observer — объект (Subject) уведомляет подписчиков (Observers) об изменениях.

Где в браузере:

- addEventListener — DOM-элемент это Subject, обработчик — Observer
- MutationObserver — следит за изменениями DOM
- IntersectionObserver — следит за видимостью
- Redux subscribe(), Zustand subscribe()

Реализация EventEmitter (см. Алгоритмические задачи).

```
// Использование на практике:
const emitter = new EventEmitter();

// Подписка
const unsubscribe = emitter.on('user-login', (user) => {
  console.log('Залогинился:', user.name);
});

// Публикация
emitter.emit('user-login', { id: 1, name: 'Alice' });

// Отписка
unsubscribe();
```

Observer vs PubSub:

- **Observer:** Subject знает о своих Observers (прямая связь)
- **PubSub:** издатель и подписчик не знают друг друга (через посредника — Event Bus)

3. Singleton

Singleton — класс, у которого может быть только ОДИН экземпляр.

Когда нужен: логгер, конфигурация приложения, пул соединений к БД.

```
// Классическая реализация (до ESM)
```

> ****Как читать** ``if (Logger.instance) return Logger.instance``:****** читай как «если экземпляр уже существует – верни ЕГО, а новый НЕ создавай». Конструктор ведёт себя как ``getInstance()`` – притворяется, что создаёт новый объект, но на самом деле возвращает тот же самый. Мнемоника: ***«Single ton-конструктор – это турникет: второй раз не пройдёшь»***.

```
class Logger {
  static instance = null;

  constructor() {
    if (Logger.instance) {
      return Logger.instance;
    }
    this.logs = [];
    Logger.instance = this;
  }

  log(message) {
    this.logs.push(`${new Date().toISOString()} ${message}`);
    console.log(message);
  }
}

const a = new Logger();
const b = new Logger();
console.log(a === b); // true – один экземпляр
```

Singleton в ESM (лучший способ):

```
// logger.js
class Logger {
  logs = [];
  log(msg) {
    this.logs.push(msg);
    console.log(msg);
  }
}

export const logger = new Logger(); // модуль выполняется один раз = singleton
```

ESM-модули кэшируются: сколько бы раз ни импортировали `logger.js`, экземпляр будет один.

Почему Singleton часто вреден:

- Глобальное состояние — трудно тестировать (нужно сбрасывать между тестами)
- Неявные зависимости (компонент зависит от синглтона, но это не

видно из пропсов)

- Усложняет tree-shaking (сборщик не может выкинуть модуль с сайд-эффектами)

Альтернатива: Dependency Injection.

```
// Singleton — неявная зависимость
function Component() {
  Logger.log('render'); // не видно в пропсах
}

// DI — явная зависимость
function Component({ logger }: { logger: Logger }) {
  logger.log('render');
}
```

4. Factory и Strategy в React

Factory (фабрика)

```
// Фабрика компонентов: создаём компонент в зависимости от типа
function fieldFactory(type: 'text' | 'number' | 'select' | 'date') {
  const components = {
    text: TextField,
    number: NumberField,
    select: SelectField,
    date: DateField,
  };
  return components[type];
}

// Использование:
const Field = fieldFactory(fieldConfig.type);
<Field {...fieldConfig.props} />
```

Strategy (стратегия)

```
// Стратегия валидации: разные правила для разных полей
const validators = {
  email: (v: string) => /^[^\s@]+@[^\s@]+\.\test(v),
  phone: (v: string) => /^\+?\d{10,15}$/.test(v),
  required: (v: string) => v.trim().length > 0,
};

function validate(value: string, rules: string[]) {
  return rules.every(rule => validators[rule]?.(value) ?? true);
}

validate('test@example.com', ['required', 'email']); // true
validate('', ['required', 'email']); // false
```


5. Module Pattern и Revealing Module Pattern

Как читать `const X = (function() { ... })()` (IIFE): читай как «выполни эту функцию прямо сейчас и сохрани то, что она вернёт». Круглые скобки вокруг `function` и `()` в конце — это не ошибка, а способ создать изолированную область видимости и сразу же её использовать. Всё, что не попало в `return` — приватное и снаружи не видно. Мнемоника: «IIFE — это сейф: открыл, положил, закрыл, вернул только нужное».

```
// Классический Module Pattern (через IIFE)
const UserService = (function() {
  const apiUrl = 'https://api.example.com'; // приватное

  function formatUser(user) { // приватное
    return { ...user, displayName: `${user.firstName} ${user.lastName}` }
  };

  return {
    getUsers() { // публичное
      return fetch(`${apiUrl}/users`).then(r => r.json());
    },
    getUser(id) { // публичное
      return fetch(`${apiUrl}/users/${id}`)
        .then(r => r.json())
        .then(formatUser);
    },
  };
})();

// ESM эквивалент (современный подход):
const apiUrl = 'https://api.example.com';
function formatUser(user) { ... }
export function getUsers() { ... }
export function getUser(id) { ... }
```

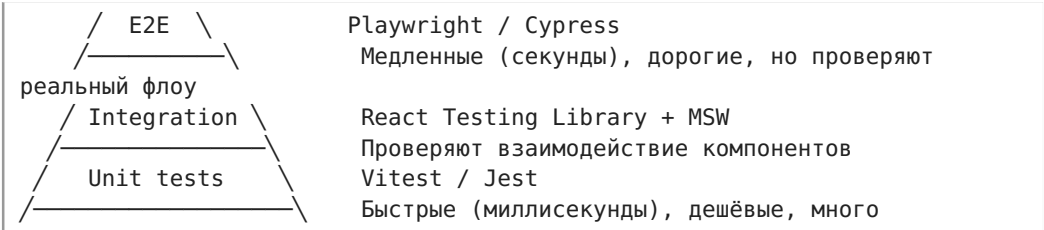
Связанное

- Управление состоянием
- Архитектура React компонентов
- Micro Frontends и Module Federation
- Feature-sliced design и Atomic Design
- Алгоритмические задачи

Тестирование React

На собеседованиях Middle+ спрашивают не «умеешь ли ты писать тесты», а «КАКИЕ тесты ты пишешь и почему». Главное — понимать пирамиду тестирования и выбирать правильный инструмент под задачу.

1. Пирамида тестирования



Уровень	Инструмент	Объём	Скорость	Что тестирует
Unit	Vitest	60-70%	~1ms	Чистые функции, хуки, утилиты
Integration	RTL + MSW	20-30%	~50-200ms	Компоненты + API + хранилище
E2E	Playwright	5-10%	~1-5s	Критические пользовательские сценарии

Правило: если баг можно отловить уровнем ниже — тестируй уровнем ниже. E2E — только для критических путей (оплата, регистрация).

2. Unit-тесты: Vitest

Почему Vitest, а не Jest

- Совместим с Jest API (describe, it, expect)
- Работает с ESM из коробки
- В 2-3 раза быстрее
- Нативная интеграция с Vite (тот же vite.config.ts)

Тестирование чистых функций

```
// utils/ticket.ts
export function formatPrice(price: number, currency = '₽'): string {
  return `${price.toLocaleString('ru-RU')}${currency}`;
}

export function canBuy(available: number, requested: number): boolean {
  return available >= requested && requested > 0;
}

// utils/ticket.test.ts
import { describe, it, expect } from 'vitest';

describe('formatPrice', () => {
  it('форматирует целое число', () => {
    expect(formatPrice(1500)).toBe('1 500 ₽');
  });
  it('форматирует дробное', () => {
    expect(formatPrice(1499.99)).toBe('1 499,99 ₽');
  });
  it('принимает другую валюту', () => {
    expect(formatPrice(100, '$')).toBe('100 $');
  });
});

describe('canBuy', () => {
  it('возвращает true если достаточно билетов', () => {
    expect(canBuy(10, 3)).toBe(true);
  });
  it('возвращает false если не хватает', () => {
    expect(canBuy(2, 5)).toBe(false);
  });
  it('возвращает false для нуля', () => {
    expect(canBuy(10, 0)).toBe(false);
  });
});
```

Тестирование хуков

Как читать `renderHook(() => useCounter(5)) + result.current`:

«отрендерить хук в изолированной песочнице (без компонента-обёртки) и читать его текущее состояние через `result.current` — это как `ref.current`, которое обновляется после каждого `act()`». `act()` нужен, чтобы React применил все обновления состояния перед следующей проверкой.

```

import { renderHook, act } from '@testing-library/react';
import { describe, it, expect } from 'vitest';

// Хук из предыдущего раздела
function useCounter(initial = 0) {
  const [count, setCount] = useState(initial);
  const increment = () => setCount(c => c + 1);
  const decrement = () => setCount(c => c - 1);
  return { count, increment, decrement };
}

describe('useCounter', () => {
  it('начинает с initial значения', () => {
    const { result } = renderHook(() => useCounter(5));
    expect(result.current.count).toBe(5);
  });

  it('увеличивает счётчик', () => {
    import { useState } from 'react';

    const { result } = renderHook(() => useCounter());
    act(() => result.current.increment());
    expect(result.current.count).toBe(1);
  });

  it('уменьшает счётчик', () => {
    const { result } = renderHook(() => useCounter(3));
    act(() => result.current.decrement());
    expect(result.current.count).toBe(2);
  });
});

```

Моки таймеров

```

import { vi, describe, it, expect, beforeEach } from 'vitest';

describe('debounce', () => {
  beforeEach(() => { vi.useFakeTimers(); });

  it('вызывает функцию через указанную задержку', () => {
    const fn = vi.fn();
    const debounced = debounce(fn, 300);
    debounced();
    expect(fn).not.toHaveBeenCalled();
    vi.advanceTimersByTime(300);
    expect(fn).toHaveBeenCalledOnce();
  });
});

```

3. Интеграционные тесты: React Testing Library

Философия RTL

Тестируй как пользователь, а не как разработчик:

- `wrapper.state().isOpen` — тест сломается при рефакторинге
- `screen.getByText('Меню')` — тест живёт, пока пользователь видит кнопку

Тестирование компонента

```
// EventCard.tsx
interface EventCardProps {
  title: string;
  date: string;
  price: number;
  available: number;
  onBuy: () => void;
}

export function EventCard({ title, date, price, available, onBuy }: EventCardProps) {
  return (
    <article>
      <h2>{title}</h2>
      <time>{date}</time>
      <p>{formatPrice(price)}</p>
      <button onClick={onBuy} disabled={available === 0}>
        {available === 0 ? 'Продано' : 'Купить'}
      </button>
    </article>
  );
}

// EventCard.test.tsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { describe, it, expect, vi } from 'vitest';

describe('EventCard', () => {
  it('отображает название и цену', () => {
    render(<EventCard title="React Conf" date="2026-08-01" price={500}
0} available={10} onBuy={vi.fn()} />);
    expect(screen.getByText('React Conf')).toBeInTheDocument();
    expect(screen.getByText('5 000 ₺')).toBeInTheDocument();
  });

  it('вызывает onBuy при клике', async () => {
    const onBuy = vi.fn();
    render(<EventCard title="React Conf" date="2026-08-01" price={500}
0} available={10} onBuy={onBuy} />);
    const user = userEvent.setup();
    await user.click(screen.getByRole('button', { name: 'Купить' }));
    expect(onBuy).toHaveBeenCalledOnce();
  });

  it('блокирует кнопку если available = 0', () => {
    render(<EventCard title="React Conf" date="2026-08-01" price={500}
0} available={0} onBuy={vi.fn()} />);
    expect(screen.getByRole('button', { name: 'Продано' })).toBeDisabled();
  });
});
```

```
});  
});
```

Тестирование асинхронных операций

```
import { render, screen, waitFor } from '@testing-library/react';  
  
it('показывает список событий после загрузки', async () => {  
  render(<EventsList />);  
  // Сначала — индикатор загрузки  
  expect(screen.getByText('Загрузка...')).toBeInTheDocument();  
  // Ждём появления данных  
  await waitFor(() => {  
    expect(screen.getByText('React Conf')).toBeInTheDocument();  
  });  
  // Индикатор исчез  
  expect(screen.queryByText('Загрузка...')).not.toBeInTheDocument();  
});
```

Поиск элементов — приоритет

```
// 1. По роли (лучший способ)  
screen.getByRole('button', { name: 'Купить' });  
screen.getByRole('heading', { name: /react conf/i });  
  
// 2. По тексту  
screen.getByText('5 000 ₽');  
screen.getByText(/продано/i);  
  
// 3. По test-id (только если никак иначе)  
screen.getByTestId('event-card-123');  
  
//    Никогда: по классам, по структуре DOM
```

4. Моки API: MSW (Mock Service Worker)

Зачем MSW

Мокать `fetch` напрямую — хрупко. MSW перехватывает запросы на уровне сети (Service Worker), компонент ведёт себя как в реальности.

Как читать `http.get('/api/events/:id', ({ params }) => ...)`:
«перехвати GET-запрос, где `:id` — динамический сегмент URL, и получи его значение через деструктуризацию `{ params }`». Сигнатура колбэка MSW зеркалит Express.js: `:param` в пути → `params.param` в обработчике.


```
// mocks/handlers.ts
import { http, HttpResponse } from 'msw';

export const handlers = [
  // Мок списка событий
  http.get('/api/events', () => {
    return HttpResponse.json([
      { id: '1', title: 'React Conf', price: 5000, date: '2026-08-01' },
      { id: '2', title: 'VueConf', price: 3000, date: '2026-09-15' },
    ]);
  }),

  // Мок конкретного события
  http.get('/api/events/:id', ({ params }) => {
    return HttpResponse.json({
      id: params.id,
      title: 'React Conf',
      price: 5000,
    });
  }),

  // Мок ошибки
  http.post('/api/events', () => {
    return new HttpResponse(null, { status: 500 });
  }),
];
```

```
// setup.ts
import { setupServer } from 'msw/node';
import { handlers } from './mocks/handlers';

const server = setupServer(...handlers);
beforeAll(() => server.listen());
afterEach(() => server.resetHandlers()); // сброс между тестами
afterAll(() => server.close());
```

Переопределение мока в конкретном тесте

```
it('показывает ошибку при падении API', async () => {
  server.use(
    http.get('/api/events', () => {
      return new HttpResponse(null, { status: 500 });
    })
  );
  render(<EventsList />);
  await waitFor(() => {
    expect(screen.getByText(/ошибка/i)).toBeInTheDocument();
  });
});
```

5. E2E-тесты: Playwright

Когда нужны E2E

Только для критических пользовательских сценариев:

- Регистрация / логин
- Покупка билета (весь флоу от выбора до оплаты)
- Создание события организатором

```
// tests/checkout.spec.ts
import { test, expect } from '@playwright/test';

test('полный флоу покупки билета', async ({ page }) => {
  // 1. Открываем каталог
  await page.goto('/events');
  await expect(page.getByRole('heading', { name: 'События' })).toBeVisible();

  // 2. Выбираем событие
  await page.getByText('React Conf').click();
  await expect(page.getByText('5 000 ₽')).toBeVisible();

  // 3. Добавляем билет
  await page.getByRole('button', { name: 'Купить' }).click();
  await expect(page.getByText('Корзина (1)')).toBeVisible();

  // 4. Оформляем заказ
  await page.getByRole('button', { name: 'Оформить' }).click();
  await page.getByLabel('Email').fill('test@example.com');
  await page.getByRole('button', { name: 'Оплатить' }).click();

  // 5. Подтверждение
  await expect(page.getByText('Заказ подтверждён')).toBeVisible();
  await expect(page.getByText('React Conf')).toBeVisible();
});
```

Playwright vs Testing Library

	Testing Library	Playwright
Где запускается	Node.js (JSDOM)	Реальный браузер
Скорость	Быстро (50-200ms)	Медленно (1-5s)
Для чего	Логика компонентов, состояния, рендер	Пользовательские сценарии, кросс-браузерность
Сеть	Моки (MSW)	Можно и реальный бэкэнд
Запуск в CI	Всегда	Выборочно (дорого)

6. Что тестировать — приоритеты

- **Обязательно (без этого нельзя в продакшен)**
- Критические бизнес-флоу (покупка, регистрация) — E2E
- Компоненты с условной логикой (disabled, loading, error) — RTL
- Чистые функции с бизнес-логикой (расчёт цены, валидация) — Unit

Желательно

- Обработка ошибок API (500, 403, network error) — RTL + MSW
- Состояния загрузки — RTL
- Кастомные хуки — Unit (renderHook)

Опционально

- Верстка (пиксель-пёрфект) — скриншотные тесты Playwright
- Accessibility — jest-axe
- Производительность — Lighthouse CI

7. Антипаттерны в тестах

Антипаттерн	Почему плохо	Как исправить
Тестировать реализацию (state.isOpen)	Сломается при рефакторинге	Тестировать поведение (getText('Меню'))
data-testid везде	Тест не проверяет, что видят пользователи	Кнопки — getByRole, текст — getByText
100% покрытие	Дорого поддерживать, мнимая ценность	Покрывать критическое, остальное по необходимости
Один огромный тест	Непонятно что сломалось	Маленькие тесты: один тест = одна проверка
Тесты зависят друг от друга	Порядок выполнения влияет на результат	Каждый тест независим (beforeEach)
Мокать всё	Тест проходит, а в реальности — нет	Мокать только внешние зависимости (API)

Связанное

- HTML и Accessibility (a11y) — тестирование доступности (jest-axe, Lighthouse)
- React рендеринг и производительность
- Управление состоянием
- Архитектура React компонентов

Сборщики и инструменты

Вопросы про инструменты сборки проверяют, понимает ли кандидат, что происходит между `npm run build` и продакшеном.

1. Webpack vs Vite

	Webpack	Vite
Dev-сервер	Бандлит ВСЁ перед запуском	ESM на лету (по требованию), мгновенный старт
Скорость HMR	Зависит от размера бандла (медленно на больших проектах)	Мгновенно (меняется только изменённый модуль)
Prod-сборка	Webpack	Rollup
Конфигурация	Сложная (<code>webpack.config.js</code>)	Минимальная (<code>vite.config.ts</code>)
Плагины	Огромная экосистема	Совместим с Rollup-плагинами

Почему Vite быстрее в dev

Webpack dev:
src/*.ts → bundle → dev-server → браузер
Все файлы проходят через webpack ПЕРЕД тем, как dev-сервер готов

Vite dev:
src/*.ts —(по запросу браузера)→ esbuild (транспилиция TS) → ESM модуль
→ браузер
Обрабатывается ТОЛЬКО то, что запросил браузер. Холодный старт <1с.

esbuild (Go) транпилирует TypeScript в 20-100 раз быстрее, чем tsc/ Babel.

Что выбрать

- **Vite** — новые проекты, SPA. Быстрее, проще, современнее
- **Webpack** — легаси, микрофронтенды (Module Federation), сложные кастомные конфигурации

Минимальный vite.config.ts

Как читать `defineConfig({ build: { rollupOptions: ... } })`:

«определяю конфиг Vite, который под капотом прокидывает настройки в Rollup — читай `manualChunks` как "вырежи react и react-dom в отдельный файл `vendor.js`». Запомни: Vite в продакшене — это Rollup, и всё, что внутри `build.rollupOptions`, — прямой доступ к его API.

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: { port: 3000 },
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'], // отдельный чанк для React
        },
      },
    },
  },
});
```

2. Tree-shaking

Tree-shaking — удаление неиспользуемого кода из финального бандла.

Как это работает

```
// math.js
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
export const multiply = (a, b) => a * b; // не импортируется

// app.js
import { add } from './math';
console.log(add(2, 3));
```

Webpack/Rollup видят, что `subtract` и `multiply` не импортируются → удаляют из бандла.

Условия для tree-shaking

1. **ESM (import/export)** — статический импорт. CommonJS (require) — динамический, tree-shaking невозможен
2. **Нет побочных эффектов** — модуль не должен менять глобальное состояние при импорте

3. **"sideEffects": false** в `package.json` — говорит сборщику «можешь смело удалять неиспользуемые экспорты»

```
// package.json
{
  "sideEffects": false, // все модули чистые
  // или:
  "sideEffects": ["*.css", "*.global.js"] // только эти имеют сайд-
  эффекты
}
```

Почему `import { debounce } from 'lodash'` не tree-shake'ится

```
// Весь lodash (500+ KB) попадает в бандл
import { debounce } from 'lodash';

// Tree-shake — только debounce (~2 KB)
import debounce from 'lodash/debounce';

// Или lodash-es (ESM-версия, tree-shake из коробки)
import { debounce } from 'lodash-es';
```

lodash использует CJS, у него `"sideEffects": false` не указан, каждый файл — потенциально с сайд-эффектами. `lodash-es` — ESM-версия с правильным `package.json`.

3. Babel и полифилы

Зачем Babel

Babel транпилирует современный JS (ES2020+) в код, понятный старым браузерам.

```
// Исходный код (ES2020)
const result = items?.map?.(x => x.name) ?? 'empty';

// После Babel (ES5)
var result = items === null || items === void 0
  ? void 0
  : items.map(function(x) { return x.name; });
if (result === null || result === void 0) result = 'empty';
```

@babel/preset-env + browserslist

```
// .browserslistrc
> 0.5%
last 2 versions
not dead
not IE 11
```

```
// babel.config.js
module.exports = {
  presets: [
    ['@babel/preset-env', {
      targets: 'defaults', // читает из .browserslistrc
      useBuiltIns:
'usage', // добавляет полифилы ТОЛЬКО для используемых фич
      corejs: 3,
    }],
  ],
};
```

useBuiltIns: 'usage' — Babel сканирует код и добавляет полифилы только для реально используемых фич. Не тянет весь core-js.

Что делает core-js

Полифилы для отсутствующих в браузере API:

```
// Без полифила – ошибка в IE11
[1, 2, 3].includes(2);
Array.from(document.querySelectorAll('div'));
Promise.all([fetch('/a'), fetch('/b')]);

// core-js добавляет реализации этих методов
import 'core-js/actual/array/includes';
import 'core-js/actual/array/from';
```

Почему не надо полифилить ВСЁ

```
// В бандле 200+ KB полифилов, 90% не используется
{ "useBuiltIns": "entry" }

// Только используемые фичи, ~20 KB
{ "useBuiltIns": "usage" }
```

4. HMR (Hot Module Replacement)

HMR — замена модулей в браузере БЕЗ полной перезагрузки страницы (сохраняется состояние).

Как читать `import.meta.hot.accept()`: «если запущен HMR-сервер (`import.meta.hot` существует), то когда изменится `./App`, вызови колбэк с новым модулем». `import.meta` — это спец-объект, который Vite/Webpack inject'ят в модуль на этапе сборки; в обычном JS его нет.

```
// React + HMR: состояние компонента сохраняется при изменении кода
if (import.meta.hot) {
  import.meta.hot.accept('./App', (newModule) => {
    // Перерендерить с новым модулем, но сохранить state
  });
}
```

Как работает:

1. Webpack/Vite отслеживает изменения файлов
2. Отправляет обновлённый модуль через WebSocket
3. Браузер заменяет модуль в рантайме
4. React Fast Refresh перерендеривает только изменённые компоненты

5. Code Splitting

Разделение бандла на части для загрузки по требованию.

Как читать `React.lazy(() => import(/* webpackChunkName: "admin" */ './AdminPage'))`: «лениво загрузи модуль `./AdminPage` и назови чанк `admin` — но только когда React реально попытается его отрендерить». Магический комментарий `/* webpackChunkName */` — это директива Webpack'у, не JavaScript; в Vite/Rollup она игнорируется.

```
// Статический импорт — попадает в основной бандл
import { EventPage } from './EventPage';

// Динамический импорт — отдельный чанк (загружается при переходе)
const EventPage = React.lazy(() => import('./EventPage'));

// С магическим комментарием (webpackChunkName)
const AdminPage = React.lazy(() =>
  import(/* webpackChunkName: "admin" */ './AdminPage')
);

// В роутере
<Route path="/events/:id" element={
  <Suspense fallback={<Spinner />}>
    <EventPage />
  </Suspense>
} />
```

Что **splitting** даёт на практике:

- Пользователь на `/catalog` не скачивает код `/admin` и `/checkout`
- Быстрее первый экран (меньше JS)
- Кэширование: чанки с хешем не меняются при изменении других частей приложения

6. Source Maps

Source Map — файл, который связывает минифицированный/транспирированный код с исходным.

```
// bundle.js (минифицированный)
var a=1,b=2;console.log(a+b);
///

```
sourceMappingURL=bundle.js.map

// bundle.js.map — содержит исходные имена переменных, строки, файлы
```


```

Типы source maps (от медленных-точных к быстрым-приблизительным):

Тип	Скорость сборки	Качество	Когда
source-map	Медленно	Полное	Prod (загружается только при открытии DevTools)
inline-source-map	Медленно	Полное	Dev (встроен в бандл)
cheap-module-source-map	Быстро	Строки, без колонок	Dev (хороший баланс)
eval-source-map	Быстро	Полное	Dev (быстрая пересборка)
nosources-source-map	Быстро	Стек-трейс, без кода	Prod (для Sentry: видно где ошибка, но код скрыт)

Связанное

- CSS и стилизация — CSS Modules, PostCSS, Tailwind
- Web API
- DevOps для фронтендера
- React рендеринг и производительность

Безопасность фронтенда

Безопасность фронтенда

Три кита фронтенд-безопасности: XSS, CSRF и CSP. Спрашивают ВСЕГДА.

1. XSS (Cross-Site Scripting)

XSS — внедрение зловредного JavaScript в страницу, который выполняется в браузере жертвы.

Три типа XSS

Тип	Где хранится	Как попадает
Stored	Сервер (БД, комментарии, профиль)	Злоумышленник сохраняет скрипт через форму → жертва открывает страницу → скрипт выполняется
Reflected	В URL / параметрах запроса	Жертва переходит по ссылке ?q=<script>...</script> → сервер вставляет параметр в HTML → скрипт выполняется
DOM-based	В DOM (через JS на клиенте)	element.innerHTML = location.hash → скрипт в хеше выполняется

Пример DOM-based XSS

```
//      Прямая вставка пользовательских данных в HTML
const params = new URLSearchParams(location.search);
document.getElementById('greeting').innerHTML = `Привет, ${params.get('name')}!`;
// Атакующий переходит по ?name=<img src=x onerror="stealCookies()">
```

Защита на фронте

```
// 1..textContent вместо innerHTML
element.textContent = userInput; // безопасно: текст, не HTML

// 2. Экранирование (если innerHTML необходим)
function escapeHtml(str) {
  const div = document.createElement('div');
  div.textContent = str;
  return div.innerHTML;
}
element.innerHTML = escapeHtml(userInput);

// 3. DOMPurify – санитизация HTML (если нужен rich text)

> **Как читать `DOMPurify.sanitize(userHtml, { ALLOWED_TAGS: [...],
ALLOWED_ATTR: [...] })`** «прочеши пользовательский HTML и вырежи всё, к
роме тегов из белого списка `ALLOWED_TAGS` и атрибутов из `ALLOWED_ATTR`
– всё остальное выброси». DOMPurify парсит HTML в DOM и физически
удаляет запрещённые узлы, а не regex'ами – поэтому безопасен.

import DOMPurify from 'dompurify';
element.innerHTML = DOMPurify.sanitize(userHtml, {
  ALLOWED_TAGS: ['b', 'i', 'a', 'p', 'br'],
  ALLOWED_ATTR: ['href', 'title'],
});

// 4. Опасные функции, которых избегать:
eval(userInput);           // НИКОГДА
new Function(userInput);   // НИКОГДА
setTimeout(userInput, 0);  // строка-код – eval под капотом
location.href = userInput; // javascript:alert(1)
```

Защита на бэке (важно понимать фронтендеру)

- **Экранировать вывод** — любой пользовательский ввод, вставленный в HTML, должен экранироваться
- **Content-Type: application/json** — если API отдаёт JSON, браузер не выполнит его как HTML даже при XSS
- **Валидировать на бэке** — не доверять клиентской валидации

2. CSRF (Cross-Site Request Forgery)

Атака: злоумышленник заставляет браузер жертвы выполнить нежелательный запрос к сайту, где жертва авторизована.

1. Пользователь залогинен в bank.com (кука сессии в браузере)
2. Пользователь заходит на evil.com
3. evil.com содержит:

```
<form action="https://bank.com/transfer" method="POST">
    <input name="to" value="attacker">
    <input name="amount" value="1000000">
</form>
<script>document.forms[0].submit()</script>
```
4. Браузер отправляет запрос с кукой → банк переводит деньги

Почему SPA с JWT в заголовке Authorization НЕ уязвим к CSRF

```
// SPA отправляет JWT в заголовке:
fetch('/api/transfer', {
  method: 'POST',
  headers: { 'Authorization': 'Bearer eyJ...' },
  body: JSON.stringify({ to: 'attacker', amount: 1000000 }),
});

// Злоумышленник с evil.com НЕ может заставить браузер добавить кастомный заголовок.
// Браузер автоматически прикрепляет ТОЛЬКО куки (Cookie).
// Заголовок Authorization прикрепляется только JS-кодом.
// Поэтому CSRF через <form> не работает — заголовок не добавится.
```

Когда CSRF всё ещё нужен:

- Куки используются для аутентификации (сессии)
- Формы с enctype="text/plain" могут обойти preflight

Защита (если используются куки для аутентификации)

- **SameSite=Strict/Lax** — кука не отправляется с чужого сайта
- **CSRF-токен** — скрытое поле в форме, проверяется на сервере
- **Проверка Origin / Referer** — сервер проверяет, что запрос пришёл со своего домена

3. CSP (Content Security Policy)

CSP — HTTP-заголовок, который говорит браузеру, каким источникам доверять. Мощнейшая защита от XSS.

Базовый CSP

Как читать *Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'*: «по умолчанию разрешаю ресурсы только с моего домена ('self'); скрипты — тоже только с моего домена; стили — с моего домена ПЛЮС inline-стили ('unsafe-inline')». Мнемоника: *-src* = «откуда можно брать», *'self'* = «только свой домен», *'unsafe-inline'* = «разрешаю *<style>* и *style=""* прямо в HTML».

Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'

Директива	Что контролирует
default-src	Источники по умолчанию
script-src	Откуда можно загружать и выполнять JS
style-src	Откуда можно загружать CSS
img-src	Откуда можно загружать изображения
connect-src	Куда можно делать fetch/XHR/WebSocket
font-src	Откуда можно загружать шрифты
frame-src	Какие iframe разрешены

Почему 'unsafe-inline' — плохо

<!-- С unsafe-inline – XSS всё ещё работает: -->
<script>alert('XSS')</script> <!-- inline script разрешён -->

<!-- Без unsafe-inline – только скрипты из разрешённых источников: -->
<script src="/app.js"></script> <!-- ОК -->
<script>alert('XSS')</script> <!-- БЛОКИРОВАНО CSP -->

Nonce и Hash — как разрешить легитимный inline

<!-- Сервер генерирует уникальный nonce для каждого запроса -->
<script nonce="r4nd0mN0nc3">{{ INLINE_CONFIG }}</script>

<!-- HTTP-заголовок: -->
Content-Security-Policy: script-src 'nonce-r4nd0mN0nc3'

Nonce (number used once) — сервер генерирует случайную строку, вставляет в CSP-заголовок и в атрибут `<script>`. Злоумышленник не знает nonce → его inline-скрипт заблокирован.

CSP на практике

```
# Strict CSP (рекомендация Google)
add_header Content-Security-Policy "
    default-src 'self';
    script-src 'self' 'nonce-{NONCE}';
    style-src 'self' 'unsafe-inline';
    img-src 'self' https: data:;
    connect-src 'self' https://api.example.com;
    frame-src 'none';
    object-src 'none';
    base-uri 'self';
    form-action 'self';
";
```

4. Другие заголовки безопасности

Заголовок	Что делает
Strict-Transport-Security	HSTS: всегда использовать HTTPS, нельзя перейти на HTTP
X-Content-Type-Options: nosniff	Не даёт браузеру угадывать MIME-тип (защита от MIME-sniffing атак)
X-Frame-Options: DENY	Запрещает открывать сайт в iframe (защита от clickjacking)
Referrer-Policy: strict-origin-when-cross-origin	Контролирует, сколько информации об источнике передаётся при переходе
Permissions-Policy	Контролирует доступ к API браузера (камера, микрофон, геолокация)

5. Чек-лист безопасности для фронтендера

- [] JWT Refresh в httpOnly Secure SameSite cookie, не в localStorage
- [] CSP-заголовок без 'unsafe-inline' и 'unsafe-eval'
- [] Все входные данные экранируются (textContent, не innerHTML)
- [] DOMPurify для rich text
- [] Нет eval(), new Function(), javascript: URL
- [] CORS разрешает только свой домен
- [] Rate limiting на бэке
- [] HTTPS enforced (HSTS)
- [] Зависимости проверены (npm audit, Snyk)
- [] Content-Type: application/json для API

Связанное

- Вопросы на собеседовании FullStack
- Браузер и HTTP
- Web API
- Micro Frontends и Module Federation
- DevOps для фронтендера

DevOps для фронтендера

DevOps для фронтендера

На Middle+ собеседованиях всё чаще спрашивают базовый DevOps: Docker, CI/CD, Nginx. Не нужно быть админом, но понимать, как твой код попадает в продакшен — обязательно.

1. Docker

Docker — контейнеризация: упаковка приложения со всеми зависимостями в изолированное окружение. «Работает на моей машине» → «Работает везде одинаково».

Ключевые понятия

- **Image** — слепок файловой системы (как ISO-образ). Неизменяемый
- **Container** — запущенный экземпляр образа. Изолирован, имеет свою ФС, сеть, процессы
- **Layer** — каждый RUN/COPY/ADD в Dockerfile создаёт новый слой
- **Dockerfile** — инструкция по сборке образа

Dockerfile для React SPA

Как читать многоэтапный Dockerfile: *FROM ... AS builder* — «это первый этап, сборочный цех с Node.js, исходниками и node_modules». *FROM nginx:alpine* — «это второй этап, чистый лист с одним Nginx». *COPY --from=builder* — «возьми из сборочного цеха только готовый результат (dist/) и положи в боевой контейнер». Весь мусор (node_modules, исходники, кэш) остаётся в builder и в итоговый образ не попадает. Поэтому образ весит 20 MB, а не 500.

```
# === STAGE 1: Сборка ===
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# === STAGE 2: Production (только статика) ===
FROM nginx:alpine
# Копируем собранную статику в Nginx
COPY --from=builder /app/dist /usr/share/nginx/html
# Копируем конфиг Nginx
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Почему `COPY . .` после `npm install` убивает кэш

```
# Плохо: любое изменение кода → npm ci выполняется заново
COPY . .
RUN npm ci

# Хорошо: кэшируем слои по частоте изменений
COPY package*.json ./      # ← меняется РЕДКО
RUN npm ci                  # ← кэшируется
COPY . .                    # ← меняется ЧАСТО, но слои выше уже в кэше!
```

Docker кэширует слои: если `package.json` не изменился, слой с `npm ci` берётся из кэша.

Multi-stage build

Первый этап (`builder`) содержит `Node.js`, исходники, `node_modules` — большой образ (~500 MB). Второй этап (`nginx:alpine`) — только собранная статика, итоговый образ ~20 MB.

Основные команды

```
docker build -t my-app .      # Собрать образ
docker run -p 8080:80 my-app  # Запустить контейнер
docker ps                     # Список запущенных контейнеров
docker logs container_id      # Логи контейнера
docker exec -it container_id sh # Зайти в контейнер
```

2. CI/CD Pipeline

CI/CD — автоматизация от пуша до продакшена.

Типичный пайплайн для фронтенда

```
Push в GitHub
↓
[CI] Установка зависимостей (npm ci)
↓
[CI] Линтинг (ESLint)           ← параллельно
[CI] Проверка типов (tsc)       ← параллельно
[CI] Unit-тесты (Vitest)        ← параллельно
↓
[CI] Сборка (npm run build)
↓
[CI] Интеграционные тесты (RTL)
↓
[CI] E2E-тесты (Playwright) — опционально
↓
[CD] Деплой
    ├── staging: автоматически из main
    └── production: manual approve (кнопка)
```

Пример GitHub Actions

Как читать YAML пайплайна GitHub Actions: *on: push: branches: [main]* — «запускай этот пайплайн, когда кто-то делает push в ветку main». *jobs:* — список задач. *needs: lint-and-test* — «эта задача ждёт, пока завершится *lint-and-test*». *if: github.ref == 'refs/heads/main'* — «выполняй только если пуш был именно в main». *runs-on: ubuntu-latest* — «запусти виртуальную машину с Ubuntu».

```

# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: 'npm' }

      - run: npm ci

      # Параллельно:
      - run: npm run lint
      - run: npm run typecheck
      - run: npm run test -- --coverage

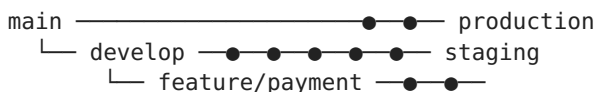
      - run: npm run build

      # Артефакт для деплоя
      - uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist/

  deploy-staging:
    needs: lint-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with: { name: dist }
      - run: rsync -avz ./ user@staging-server:/var/www/

```

Git Flow (упрощённый)



- **feature/** → создаём от develop, мёржим в develop (через PR)
- **develop** → CI прогоняет тесты, деплоит на staging
- **main** → manual approve, деплой на production

3. Nginx для SPA

Nginx — веб-сервер, который отдаёт статику и проксирует API-запросы.

Базовая конфигурация

```
server {
    listen 80;
    server_name mysite.com;
    root /usr/share/nginx/html;
    index index.html;

    # SPA: все пути → index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Статика с долгим кэшем (файлы с хешем)
    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Проксирование API
    location /api/ {
        proxy_pass http://backend:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # Gzip сжатие
    gzip on;
    gzip_types text/plain text/css application/json application/
javascript;
    gzip_min_length 1000;
}
```

try_files — магия SPA

Как читать `try_files $uri $uri/ /index.html`; «Nginx, когда приходит запрос — попробуй отдать файл по запрошенному пути; если файла нет — попробуй отдать папку; если и папки нет — плюнь на всё и отдай index.html, пусть React Router внутри браузера сам разбирается, что показывать». Без этой строки любой прямой заход на `/events/123` вернёт 404.

```
Запрос: /events/123
```

```
try_files $uri $uri/ /index.html;
```

1. Ищем файл /events/123 → нет такого
2. Ищем папку /events/123/ → нет такой
3. Отдаём /index.html → React Router разберётся

Без try_files — 404 на всех маршрутах кроме /.

Кэширование

```
# index.html – никогда не кэшируем
location = /index.html {
    add_header Cache-Control "no-cache, must-revalidate";
}

# Бандлы с хешем (main.a1b2c3.js) – кэшируем навсегда
location ~* \.(js|css|woff2)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
```

Почему index.html нельзя кэшировать: имя бандла меняется → index.html должен ссылаться на новый бандл. Если браузер закэширует index.html — пользователь будет получать старую версию приложения.

4. Переменные окружения

```
# .env.production
VITE_API_URL=https://api.example.com
VITE_SENTRY_DSN=https://xxx@sentry.io/123

# В коде:
const apiUrl = import.meta.env.VITE_API_URL; // Vite
const apiUrl = process.env.REACT_APP_API_URL; // CRA/Webpack
```

Важно: переменные окружения в VITE_* или REACT_APP_* вшиваются в бандл НА МОМЕНТ СБОРКИ. Нельзя поменять их после сборки (статический хостинг). Для runtime-конфигурации: загружать config.json при старте приложения или использовать Server-Side конфигурацию.

5. Мониторинг и алертинг

Инструмент	Для чего
Sentry	Отлов ошибок на фронте (стек-трейс, breadcrumbs, replay)
LogRocket / FullStory	Session Replay: запись того, что делал пользователь
Google Analytics / Plausible	Аналитика посещений
Lighthouse CI	Проверка производительности в CI
UptimeRobot / Grafana	Мониторинг доступности и метрик

Связанное

- Сборщики и инструменты
- Безопасность фронтенда
- Web API

Micro Frontends и Module Federation

Micro Frontends и Module Federation

Что это и зачем

Micro Frontends — архитектурный подход, при котором фронтенд разбивается на независимые микро-приложения, каждое со своим стеком, репозиторием и командой. Аналог микросервисов на бэкенде.

Когда применять:

- Большая команда (20+ фронтендеров)
- Несколько независимых потоков разработки
- Разные части приложения обновляются с разной скоростью
- Постепенная миграция с легаси-стека

Когда НЕ применять:

- Команда из 3-5 человек — избыточно
- Нет проблем с масштабированием команд
- Приложение монолитное и стабильное

Module Federation (Webpack 5)

Сборка во время выполнения (runtime), а не на этапе билда. Один микрофронтенд (host) загружает другой (remote) прямо в браузере.

Базовая конфигурация

Remote (микрофронтенд, который ОТДАЁТ):

Как читать *exposes*: ключ слева (`'./Button'`) — это «витрина», имя, по которому другие будут запрашивать твой модуль. Значение справа (`'./src/Button'`) — где этот модуль реально лежит у тебя в проекте. Грубо: «по запросу Button отдай файл из src/Button».

```
// webpack.config.js (remote)
const { ModuleFederationPlugin } = require('webpack').container;

module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: 'remote_app',          // уникальное имя
      filename: 'remoteEntry.js',  // точка входа для host'a
      exposes: {
        './Button': './src/Button', // что отдаём наружу
        './Header': './src/Header',
      },
      shared: {                   // общие зависимости
        react: { singleton: true, eager: true },
        'react-dom': { singleton: true, eager: true },
      },
    }),
  ],
};
```

Host (приложение-оболочка, которое ПРИНИМАЕТ):

Как читать *remotes*: `remote: 'remote_app@http://localhost:3001/remoteEntry.js'` — «создай переменную *remote*, которая ссылается на приложение с именем *remote_app*, загрузив его точку входа с этого URL». После этого `import('remote/Button')` будет работать как обычный импорт, но код прилетит по сети.

```
// webpack.config.js (host)
new ModuleFederationPlugin({
  name: 'host',
  remotes: {
    remote: 'remote_app@http://localhost:3001/remoteEntry.js',
  },
  shared: {
    react: { singleton: true },
    'react-dom': { singleton: true },
  },
});
```

Использование в коде:

Как читать `React.lazy(() => import('remote/Button'))`: «React, не загружай код этой кнопки сразу при старте. Когда она реально понадобится в первый раз — сходи по сети, загрузи микрофронтенд, и только потом отрендери». `Suspense` — это заглушка «Загрузка...», которую пользователь видит, пока летит сетевой запрос.

```
// Ленивая загрузка микрофронтенда
const RemoteButton = React.lazy(() => import('remote/Button'));

// В компоненте — обязательно Suspense!
function App() {
  return (
    <Suspense fallback={<div>Загрузка...</div>}>
      <RemoteButton />
    </Suspense>
  );
}
```

Shared-зависимости: ключевой момент

`singleton: true` гарантирует, что React загрузится ОДИН раз, а не по копии на каждый микрофронтенд.

Как читать `shared: { react: { singleton: true, eager: true } }`: «React — общая библиотека. `singleton: true` — пусть будет только одна её копия на всё приложение, иначе сломаются хуки. `eager: true` — загрузи её прямо сейчас, не жди первого импорта (нужно для host-приложения, которое рендерит React с первой секунды)».

Без `singleton`:
`host/react` (версия 18.2) + `remote/react` (версия 18.2) = 2 копии → ошибка хуков

С `singleton`:
`host/react` → общий экземпляр → оба используют одну версию

Eager vs non-eager:

- `eager: true` — загружается СРАЗУ при старте (нужно для host, который всегда рендерит React)
- `eager: false` — загружается по требованию (для remote — загрузится при первом `import()`)

Module Federation 2 (Rspack / ByteDance)

Module Federation 2 — эволюция от создателей Webpack (Zack Jackson, теперь в ByteDance). Ключевые отличия:

	MF 1 (Webpack)	MF 2 (Rspack)
Рантайм	Только Webpack	Webpack, Rspack, Vite (через плагин)
Типы	Нет встроенной типизации	Генерация .d.ts для remote
Динамические remote	Через __webpack_init_sharing__	Нативный API
Версионирование	Ручное управление	Manifest-файл с версиями
Dev-режим	Медленная сборка	HMR между микрофронтами

MF2: динамическая загрузка remote

```
// MF2 – динамический remote без правки webpack.config
import { loadRemote } from '@module-federation/runtime';

const RemoteApp = React.lazy(() =>
  loadRemote({
    url: 'http://localhost:3001/remoteEntry.js',
    scope: 'remote_app',
    module: './App',
  })
);
```

Роутинг между микрофронтендами

Два подхода:

1. Роутинг на уровне оболочки (Shell)

```
// Host (оболочка) управляет ВСЕМИ маршрутами
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/dashboard/*" element={<DashboardMicroFrontend />} />
  <Route path="/settings/*" element={<SettingsMicroFrontend />} />
</Routes>
```

2. Децентрализованный роутинг

Каждый микрофронтенд объявляет свои маршруты:

```
// dashboard-mf
export const routes = [
  { path: '/dashboard', component: Dashboard },
  { path: '/dashboard/analytics', component: Analytics },
];

// host собирает все маршруты
const allRoutes = microFrontends.flatMap(mf => mf.routes);
```


Рекомендация: оболочка + децентрализованные объявления.
Оболочка знает дерево, микрофронтенды — свои листья.

Коммуникация между микрофронтендами

Правило: микрофронтенды НЕ импортируют код друг друга напрямую.
Только через контракты:

1. Custom Events (для редких событий)

```
// MF1: отправляет
window.dispatchEvent(new CustomEvent('cart:updated', { detail: { items:
3 } }));

// MF2: слушает
window.addEventListener('cart:updated', (e) => updateCartBadge(e.detail.i
tems));
```

2. Shared Store (через оболочку)

```
// Оболочка предоставляет EventEmitter или Zustand-store
// MF1: обновляет
shellStore.set('user', newUser);

// MF2: подписывается
shellStore.subscribe('user', (user) => updateUI(user));
```

3. URL (для состояния навигации)

```
// MF1: устанавливает
navigate('/search?q=react');

// MF2: читает
const [searchParams] = useSearchParams();
const query = searchParams.get('q');
```

Версионирование и деплой

Оболочка (Host) ← всегда загружает актуальную версию Remote
через remoteEntry.js

Развёртывание:

- Remote v1.0 на <http://cdn/remote/v1.0/remoteEntry.js>
- Remote v1.1 на <http://cdn/remote/v1.1/remoteEntry.js>

Host знает актуальную версию → бесшовное обновление

Антипаттерны

- Общий стейт-менеджер на ВСЁ приложение — микрофронтенды должны быть независимы

- Прямой импорт из другого микрофронтенда — нарушает изоляцию
 - Слишком много микрофронтендов (50+) — управление версиями становится адом
 - Забыть про `Suspense` — `React.lazy()` без `Suspense` → белый экран
 - Разные версии React без `singleton` — хуки ломаются
-

Что отвечать на собеседовании

«Работали с микрофронтендами?»

Да, использовали Module Federation в Webpack 5. Host-приложение загружало 3 микрофронтенда: дашборд, настройки и профиль. Настроили *singleton* для React, чтобы избежать дублирования. Роутинг — децентрализованный: каждый MF объявлял свои маршруты, оболочка собирала в дерево. Коммуникация через кастомные события и URL-параметры.

«Какие проблемы решали?»

Основная — версионирование. Когда host и remote используют одну версию библиотеки, а remote её обновляет — нужен механизм согласования. Решили через *shared с requiredVersion* и CI-проверкой совместимости.

«MF1 vs MF2 — отличие?»

MF2 работает на уровне runtime-API, не привязан к Webpack. Можно микшировать Webpack, Rspack, Vite. Плюс встроенная типизация для remote-модулей — host получает автокомплит.

Связанное

- Сборщики и инструменты
- Архитектурные паттерны
- Архитектура React компонентов

Feature-sliced design и Atomic Design

Feature-sliced design и Atomic Design

Две методологии организации кода, которые проверяют на собеседованиях Middle+/Senior.

Feature-Sliced Design (FSD)

FSD — методология, которая делит приложение на слои по ЗОНАМ ОТВЕТСТВЕННОСТИ. Каждый слой имеет чёткие правила импорта: сверху вниз, без циклов.

Слои (сверху вниз)

```
app/           — точка входа, роутер, глобальные стили, провайдеры
├── pages/      — композиция фиच и виджетов в страницы
│   ├── widgets/ — крупные самостоятельные блоки (Header, Sidebar)
│   │   ├── features/ — действия пользователя (AddToCart, Search)
│   │   │   ├── entities/ — бизнес-сущности (User, Product, Order)
│   │   │   └── shared/   — переиспользуемые утилиты, UI-kit
```

Правило импорта: слой может импортировать только из нижележащих слоёв.

Как читать цепочку *app* → *pages* → *widgets* → *features* → *entities* → *shared*: стрелка означает «может импортировать из». App импортирует из pages, pages из widgets... и никогда наоборот. Представь водопад: вода течёт только вниз, снизу вверх — запрещено.

```
app → pages → widgets → features → entities → shared
```

Структура слайса (внутри любого слоя)

```
features/auth-by-phone/
├── ui/           # компоненты (презентационные)
├── model/        # логика, сторы, хуки
├── lib/          # утилиты
├── api/          # запросы к серверу
└── index.ts      # Public API — только то, что можно импортировать
извне
```

Public API — ключевая концепция. Только `index.ts` экспортирует наружу:

Как читать `index.ts` в FSD: это входной билет слайса. Всё, что не указано в `export` этого файла — невидимо снаружи, как служебный вход в ресторан: клиенты только через парадную.

```
// features/auth-by-phone/index.ts
export { AuthByPhoneForm } from './ui/AuthByPhoneForm';
export { useAuthByPhone } from './model/useAuthByPhone';
// Внутренние детали НЕ экспортируются
```

Когда применять FSD

- Проект от 3+ разработчиков

- Есть чёткая бизнес-логика (не лендинг)
- Ожидается рост и поддержка 1+ год

Когда НЕ применять

- Маленький проект (1-2 человека) — избыточно
- Лендинг/визитка — нечего разбивать на фичи

Преимущества для собеседования

FSD показывает, что ты:

- Думаешь о масштабировании кода
- Понимаешь управление зависимостями
- Можешь объяснить, ПОЧЕМУ выбрал такую структуру

Atomic Design

Atomic Design — методология Брэда Фроста, делит UI на иерархические уровни:

Как читать иерархию Atoms → Molecules → Organisms → Templates → Pages: каждый следующий уровень собирается из предыдущих, как в химии. Атомы (кнопка, инпут) собираются в молекулы (строка поиска), молекулы — в организмы (хедер), организмы расставляются по шаблону страницы, а шаблон наполняется реальными данными.

- Atoms → базовые HTML-элементы (Button, Input, Label)
- Molecules → группа атомов (SearchBar = Input + Button)
- Organisms → группа молекул (Header = Logo + SearchBar + Nav)
- Templates → каркас страницы (разметка без данных)
- Pages → страница с реальными данными

Отличие от FSD

	Atomic Design	Feature-Sliced
Фокус	UI-компоненты, визуальная иерархия	Бизнес-логика, зоны ответственности
Группировка	По сложности (атом→молекула→организм)	По бизнес-роли (фича→сущность→шаред)
Подход	Дизайнерский (от визуала)	Инженерный (от бизнес-логики)
Когда	Дизайн-системы, UI-kit	Бизнес-приложения с командами

Как совмещать FSD + Atomic Design

```
shared/ui/          ← Atomic Design здесь
├── atoms/
│   ├── Button/
│   ├── Input/
│   └── Label/
├── molecules/
│   ├── SearchBar/
│   └── FormField/
└── organisms/
    ├── DataTable/
    └── Modal/

features/           ← FSD здесь
├── add-to-cart/    ← использует shared/ui/atoms/Button
└── search/         ← использует shared/ui/molecules/SearchBar
```

Правило: Atomic Design — внутри `shared/ui/`. FSD — для всего остального.

Что отвечать на собеседовании

«Какие методологии организации кода знаете?»

Использую Feature-Sliced Design. Проект разбит на слои: *shared* → *entities* → *features* → *widgets* → *pages* → *app*. Каждый слайс имеет Public API через *index.ts* — извне видно только то, что экспортировано. Это защищает от случайных связей и упрощает рефакторинг. На уровне *shared/ui* применяю Atomic Design — кнопки, инпуты, карточки разложены по уровням *atoms/molecules/organisms*.

«Почему FSD, а не просто папки `components/containers`?»

Папки *components/* и *containers/* не показывают бизнес-смысл. Через полгода там 100+ файлов, и непонятно, какие связаны друг с другом. FSD группирует по бизнес-роли — сразу видно, что *features/add-to-cart* содержит всё для добавления в корзину: UI, логику, API, тесты.

«Atomic Design vs FSD — что выбрать?»

Это не «или», а «и». Atomic Design — для дизайн-системы внутри *shared/ui*. FSD — для организации бизнес-логики. Вместе дают понятную структуру: дизайн-компоненты на атомарном уровне, бизнес-фичи на feature-уровне.

Антипаттерны

- Импорт из слайса в обход Public API (`import {...} from 'features/auth/ui/Form'` ВМЕСТО `...from 'features/auth'`)
- Перекрёстные импорты между фичами — фичи должны быть независимы
- Раздутый `shared/` — свалка всего подряд. В `shared` только переиспользуемое
- Atom 4-го уровня вложенности — если «атом» содержит 10 подкомпонентов, это уже организм

Связанное

- Архитектура React компонентов
- Архитектурные паттерны
- Micro Frontends и Module Federation

Виртуализация рендеринга

Виртуализация рендеринга больших данных

Проблема

1000 DOM-узлов = 1000 элементов в памяти браузера. При скролле браузер пересчитывает layout, перерисовывает. На мобильных устройствах список из 5000 элементов может «повесить» страницу.

Пример проблемы:

```
// Плохо: все 10,000 элементов в DOM одновременно
{items.map(item => <Row key={item.id} data={item} />)}
```

Решение: виртуализация (windowing)

Идея: рендерить ТОЛЬКО видимые элементы + небольшой запас. При скролле — переиспользовать DOM-узлы.

Видимые строки (5):	[строка 10]	[строка 11]	[строка 12]	[строка 13]	[строка 14]
Невидимые (9995):	...не рендерятся...				

react-window (react-window)

Самая популярная библиотека. Лёгкая (ЗКВ), фиксированные или динамические размеры.

Фиксированная высота

Как читать `render-prop` `{{ index, style }}` `=> <div style={style}> >...`: `react-window` вызывает твою функцию для каждого видимого элемента и передаёт в неё объект с двумя полями: `index` — номер элемента в массиве, `style` — готовые CSS-координаты (`position: absolute; top: Npx; height: Mpx`). Твоя задача — только подставить `style` в `div` и отрендерить содержимое по индексу. Не ты решаешь, где элемент — библиотека решает за тебя.

```
import { FixedSizeList } from 'react-window';

function VirtualList({ items }) {
  return (
    <FixedSizeList
      height={600}           // высота контейнера
      width="100%"
      itemCount={items.length} // всего элементов
      itemSize={50}          // высота ОДНОГО элемента
    >
      {({ index, style }) => (
        <div style={style}>
          {items[index].name}
        </div>
      )}
    </FixedSizeList>
  );
}
```

Динамическая высота

```
import { VariableSizeList } from 'react-window';

function getItemSize(index: number) {
  return items[index].description.length > 100 ? 100 : 50;
}

<VariableSizeList
  height={600}
  itemCount={items.length}
  itemSize={getItemSize} // функция, возвращающая высоту
  estimatedItemSize={50} // примерная высота для расчёта скролла
>
  {({ index, style }) => <div style={style}>{items[index].description}</div>}
</VariableSizeList>
```

Виртуализация сетки (Grid)

```
import { FixedSizeGrid } from 'react-window';

<FixedSizeGrid
  columnCount={4}
  columnWidth={200}
  height={600}
  rowCount={Math.ceil(items.length / 4)}
  rowHeight={200}
  width={800}
>
  {( { columnIndex, rowIndex, style } ) => {
    const item = items[rowIndex * 4 + columnIndex];
    return <div style={style}>{item?.name}</div>;
  }}
</FixedSizeGrid>
```

TanStack Virtual

Более гибкая альтернатива. Не привязана к React — работает с любым фреймворком (или без него).

Как читать TanStack-позиционирование *transform: translateY(\$ {virtualItem.start}px)* **при** *position: absolute*: виртуализатор НЕ использует поток документа. Вместо этого он создаёт контейнер-растяжку полной высоты (*getTotalSize()* — как если бы все 10 000 элементов были в DOM), а каждый видимый элемент позиционируется абсолютно на своём месте через *translateY*. Так скроллбар показывает реальный размер списка, но в DOM висит только ~20 элементов.


```

import { useVirtualizer } from '@tanstack/react-virtual';

function VirtualList({ items }) {
  const parentRef = useRef<HTMLDivElement>(null);

  const virtualizer = useVirtualizer({
    count: items.length,
    getScrollElement: () => parentRef.current,
    estimateSize: () => 50,           // примерная высота
    overscan: 5,                     // рендерить +5 элементов за
    границами
  });

  return (
    <div ref={parentRef} style={{ height: 600, overflow: 'auto' }}>
      <div style={{ height: virtualizer.getTotalSize() }}>
        {virtualizer.getVirtualItems().map(virtualItem => (
          <div
            key={virtualItem.key}
            style={{
              position: 'absolute',
              top: 0,
              left: 0,
              width: '100%',
              height: virtualItem.size,
              transform: `translateY(${virtualItem.start}px)`
            }}
          >
            {items[virtualItem.index].name}
          </div>
        ))}
      </div>
    </div>
  );
}

```

TanStack Virtual vs react-window

	react-window	TanStack Virtual
Размер	~3 KB	~5 KB
Динамическая высота	Да (VariableSizeList)	Да (измеряет реальную высоту)
Горизонтальный скролл	Да (FixedSizeList с layout="horizontal")	Да
Адаптивная сетка	Нет (фиксированные колонки)	Да (автоопределение кол-ва колонок)
Измерение реального размера	Нет (нужно указывать)	Да (ResizeObserver)
Вне React	Нет	Да (Vanilla, Vue, Solid)

Когда что:

- **react-window** — простые списки с известными размерами, маленький бандл
- **TanStack Virtual** — сложные лейауты, динамические размеры, адаптивные сетки

Бесконечный скролл + виртуализация

Совмещаем useInfiniteQuery (TanStack Query) + виртуализацию:

```
import { useInfiniteQuery } from '@tanstack/react-query';
import { useVirtualizer } from '@tanstack/react-virtual';

function InfiniteVirtualList() {
  const { data, fetchNextPage, hasNextPage } = useInfiniteQuery({
    queryKey: ['items'],
    queryFn: ({ pageParam = 0 }) => fetchPage(pageParam),
    getNextPageParam: (lastPage) => lastPage.nextCursor,
  });

  const allItems = data?.pages.flatMap(p => p.items) ?? [];

  const virtualizer = useVirtualizer({
    count: hasNextPage ? allItems.length + 1 : allItems.length,
    getScrollElement: () => parentRef.current,
    estimateSize: () => 50,
  });

  // Когда юзер докрутил до конца – подгружаем
  const lastItem = virtualizer.getVirtualItems().at(-1);
  if (lastItem && lastItem.index >= allItems.length - 1 && hasNextPage) {
    fetchNextPage();
  }

  return (/* рендер */);
}
```

Что отвечать на собеседовании

«Как рендерить список из 10,000 элементов?»

Виртуализация. Из 10,000 элементов в DOM'е находится только ~20 видимых + overscan. Остальные не рендерятся — вместо них пустое пространство, которое имитирует полную высоту списка. Использую *react-window* для простых случаев и *TanStack Virtual* для сложных (динамические размеры, адаптивные сетки).

«Какие подводные камни?»

1. **Динамическая высота** — если контент меняется, нужно пересчитать размеры. TanStack Virtual измеряет через `ResizeObserver`; `react-window` требует ручного `resetAfterIndex()`.
2. **Поиск внутри списка** — при фильтрации индексы меняются, нужно сбрасывать кэш размеров.
3. **SEO** — виртуализированный контент не в DOM → не индексируется. Для SEO-страниц — SSR с пагинацией вместо виртуализации.
4. **Доступность (a11y)** — скринридеры не видят невидимые элементы. Нужно управлять `aria-*` атрибутами.

«Бесконечный скролл + виртуализация — как?»

`useInfiniteQuery` для пагинации + `useVirtualizer` для рендеринга. Когда последний видимый индекс приближается к концу загруженных данных — вызываю `fetchNextPage()`. Важно: не дожидаться самого конца, а начинать загрузку за 5-10 элементов до конца списка.

Антипаттерны

- Виртуализация для списка из 50 элементов — избыточно
- `itemSize = () => Math.random()` — размер должен быть стабильным
- Менять `items` массив, не сбрасывая кэш размеров → элементы «прыгают»
- Забыть про `overscan` — при быстром скролле видны пустые места

Связанное

- React рендеринг и производительность
- Управление состоянием
- Web API (Intersection Observer)

Интернационализация и локализация i18n

Интернационализация и локализация (i18n)

i18n vs l10n

- **i18n (internationalization)**: подготовка приложения к поддержке нескольких языков
- **l10n (localization)**: перевод на конкретный язык + адаптация форматов

Основные задачи i18n

1. **Перевод строк** — текст в UI
2. **Форматы дат и чисел** — 1,000.00 vs 1 000,00
3. **Плюрализация** — «1 билет» / «2 билета» / «5 билетов»
4. **Направление текста** — LTR vs RTL (арабский, иврит)
5. **Единицы измерения** — км vs miles, °C vs °F

react-intl (FormatJS)

Отраслевой стандарт для React. Базируется на Intl API браузера.

Настройка

```
import { IntlProvider } from 'react-intl';
import English from './locales/en.json';
import Russian from './locales/ru.json';

const messages = { en: English, ru: Russian };

function App({ locale = 'ru' }) {
  return (
    <IntlProvider locale={locale} messages={messages[locale]}>
      <Main />
    </IntlProvider>
  );
}
```

Перевод строк

```
import { FormattedMessage, useIntl } from 'react-intl';

// Вариант 1: компонент
<FormattedMessage
  id="cart.title"
  defaultMessage="Корзина"
/>

// Вариант 2: хук (для атрибутов, aria-label)
function CartButton() {
  const intl = useIntl();
  return (
    <button aria-label={intl.formatMessage({ id: 'cart.open' })}>

    </button>
  );
}
```

Плюрализация

Как читать ICU plural {count, plural, one {# билет} few {# билета} many {# билетов}}: «возьми переменную *count*, примени к ней правило *plural* для текущего языка: если форма *one* — подставь # (это само число) и слово "билет", если *few* — "# билета", если *many* — "# билетов"». # — это автоподстановка значения переменной, не пиши её руками.

```
// locales/ru.json
{
  "cart.items": "{count, plural, one {# билет} few {# билета} many {# билетов}}"
}
```

```
<FormattedMessage id="cart.items" values={{ count: 5 }} />
// → "5 билетов"
```

Форматирование дат и чисел

```
<FormattedNumber value={1500.42} style="currency" currency="RUB" />
// → "1 500,42 ₽"
```

```
<FormattedDate value={Date.now()} year="numeric" month="long" day="numeric" />
// → "16 июля 2026 г."
```

ICU MessageFormat — мощный синтаксис

Как читать ICU select {gender, select, male {Уважаемый} female {Уважаемая} other {Уважаемый(ая)}}: «возьми переменную *gender*, выбери ветку: если *male* — покажи "Уважаемый", если *female* — "Уважаемая", во всех остальных случаях (*other*) — "Уважаемый(ая)"». *select* — это как switch-case для строк, *plural* — для чисел.

```
{cartTotal, number, ::currency/RUB} — форматирование как валюты
{expires, date, short} — короткая дата
{gender, select, male {Уважаемый} female {Уважаемая} other {Уважаемый(ая)}}
```

i18next + react-i18next

Более гибкая альтернатива. Не привязана к React.

Настройка

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';

i18n.use(initReactI18next).init({
  resources: {
    ru: { translation: { welcome: 'Добро пожаловать' } },
    en: { translation: { welcome: 'Welcome' } },
  },
  lng: 'ru',          // язык по умолчанию
  fallbackLng: 'en',  // fallback если перевод не найден
});

function App() {
  const { t } = useTranslation();
  return <h1>{t('welcome')}</h1>;
}
```

Интерполяция переменных

```
{
  "greeting": "Привет, {{name}}! У вас {{count}} новых сообщений."
}
```

```
t('greeting', { name: 'Серегей', count: 3 });
```

Плюрализация в i18next

Как читать i18next interval (1)[1 билет];(2-4)[{{count}} билета];(5+)[{{count}} билетов]: «если *count* равен 1 — покажи "1 билет", если от 2 до 4 — подставь число и "билета", если 5 и больше — подставь число и "билетов"». Круглые скобки — условие (диапазон или + для «и больше»), квадратные — что показать. `{{count}}` — placeholder для подстановки переменной.

```
{
  "ticket": "{{count}} билет",
  "ticket_plural": "{{count}} билетов",
  "ticket_interval":
    "(1)[1 билет];(2-4)[{{count}} билета];(5+)[{{count}} билетов];"
}
```

Пространства имён (namespaces)

```
// locales/ru/checkout.json
{ "title": "Оформление заказа", "pay": "Оплатить" }

// locales/ru/common.json
{ "cancel": "Отмена", "save": "Сохранить" }
```

```
const { t } = useTranslation(['checkout', 'common']);
t('checkout:pay')    // "Оплатить"
t('common:cancel')   // "Отмена"
```

react-intl vs i18next

	react-intl	i18next
Стандарт	ICU MessageFormat	Собственный синтаксис
Вне React	Нет	Да (Vanilla, Vue, Angular)
Загрузка переводов	Ручная через import	Плагины (HTTP, бандл)
Плюрализация	ICU (plural, select)	Свой формат (_plural, _interval)
Размер	~15 KB	~10 KB + плагины

Когда что:

- **react-intl** — React-only проект, нужен стандарт ICU (общий с бэкендом)
- **i18next** — мульти-фреймворк, нужна динамическая загрузка переводов, бэкенд-интеграция

Загрузка переводов: стратегии

1. В бандле (для SPA с 2-3 языками)

```
import { lazy } from 'react';

const locales = {
  ru: () => import('./locales/ru.json'),
  en: () => import('./locales/en.json'),
};
```

2. Ленивая загрузка (много языков)

```
// i18next-http-backend – загружает JSON с сервера при первом обращении
import Backend from 'i18next-http-backend';

i18n.use(Backend).init({
  backend: { loadPath: '/locales/{{lng}}/{{ns}}.json' },
});
```

3. Пространства имён + code splitting

Загружать переводы только для текущей страницы:

```
/locales/ru/checkout.json  ← загружается на /checkout
/locales/ru/profile.json   ← загружается на /profile
```

RTL (Right-to-Left)

Для арабского, иврита и др. нужна зеркальная раскладка:


```
/* layout.css */
html[dir="rtl"] .sidebar {
  left: auto;
  right: 0;
}

html[dir="rtl"] .arrow-left {
  transform: scaleX(-1);
}
```

```
<html dir={locale === 'ar' ? 'rtl' : 'ltr'}>
```

Библиотеки с поддержкой RTL: MUI, Ant Design, Chakra UI — все имеют встроенную поддержку.

Что отвечать на собеседовании

«Как организовать мультиязычность?»

Использую *react-intl* с ICU MessageFormat. Переводы — JSON с ключами. Плюрализация через *plural*, гендерные формы через *select*. Загрузка — ленивая, по 1 языку + namespace. RTL — через атрибут *dir* на *<html>*.

«Подводные камни i18n?»

1. **Конкатенация строк** — *'Всего: ' + count + ' билетов'* не переводится. Только шаблоны с интерполяцией.
2. **Плюрализация** — в русском 3 формы, в арабском 6, а в китайском 1. ICU учитывает CLDR.
3. **Порядок слов** — в японском глагол в конце. Не разбивать предложение на части.
4. **Форматы** — даты хранить в ISO, форматировать на клиенте. Цены — в копейках/центах (integer).
5. **Длина строк** — немецкий текст на 30% длиннее английского. Верстать с запасом.

Антипаттерны

- *'У вас ' + count + ' билетов'* — непереводимая конкатенация
- Хранить переведённый текст в БД → синхронизация между языками
- CSS с фиксированной шириной под один язык — немецкий не влезет

- Загружать переводы для ВСЕХ языков сразу → 100KB+ мёртвого груза

Связанное

- Архитектура React компонентов
- React рендеринг и производительность (code splitting)

Практика написания хуков и usehooks-ts

Практика написания хуков и usehooks-ts

Результат собеседования 16.07.2026: завалил техническую часть. Собеседующий рекомендовал писать код руками, в частности хуки. Рекомендованный ресурс: usehooks-ts.com.

Почему именно хуки

Хук — идеальная единица практики:

- Изолированная логика (не зависит от UI)
- Покрывает замыкания, useEffect, useRef, useCallback, чистку
- Типизируется (TypeScript)
- Компактный размер (20-60 строк)
- Можно написать за 15-30 минут

На собеседовании часто просят: «напишите useDebounce», «напишите useLocalStorage», «напишите usePrevious». Это проверяет понимание жизненного цикла React, замыканий и побочных эффектов.

usehooks-ts: библиотека и учебник

[usehooks-ts](https://usehooks-ts.com) — библиотека из 33 готовых TypeScript-хуков. **Использовать не как npm-пакет, а как учебник:** открыть исходник, понять КАК работает, закрыть, написать САМОСТОЯТЕЛЬНО, сверить.

Полный список хуков библиотеки

1. useBoolean — переключение true/false
2. useClickAnyWhere — отслеживание кликов
3. useCopyToClipboard — копирование текста
4. useCountdown — обратный отсчёт
5. useCounter — счётчик с min/max
6. useDarkMode — тёмная тема
7. useDebounceCallback — debounce для функций
8. useDebounceValue — debounce для значений
9. useDocumentTitle — установка title

10. `useEventCallback` — стабильная ссылка на колбэк
11. `useEventListener` — подписка на события
12. `useHover` — отслеживание наведения
13. `useIntersectionObserver` — видимость элемента
14. `useInterval` — `setInterval` с очисткой
15. `useIsClient` — проверка client-side
16. `useIsMounted` — проверка монтирования
17. `useIsomorphicLayoutEffect` — `useLayoutEffect` без SSR-ошибок
18. `useLocalStorage` — стейт в `localStorage`
19. `useMap` — Map как React-стейт
20. `useMediaQuery` — медиа-запросы
21. `useOnClickOutside` — клик вне элемента
22. `useReadLocalStorage` — чтение из `localStorage`
23. `useResizeObserver` — отслеживание размера
24. `useScreen` — размеры экрана
25. `useScript` — загрузка внешнего скрипта
26. `useScrollLock` — блокировка скrolла
27. `useSessionStorage` — стейт в `sessionStorage`
28. `useStep` — пошаговая навигация
29. `useTernaryDarkMode` — 3-режимная тема
30. `useTimeout` — `setTimeout` с очисткой
31. `useToggle` — переключение с ручным управлением
32. `useUnmount` — колбэк при размонтировании
33. `useWindowSize` — размеры окна

Методика практики

Правило «открыл → понял → закрыл → написал → сверил»

1. Открыть страницу хука на usehooks-ts.com
2. Прочитать описание и сигнатуру (что принимает, что возвращает)
3. Закрыть вкладку с исходником
4. Написать свою реализацию в Codesandbox / локально
5. Открыть исходник и сверить:
6. Правильно ли обработана очистка (cleanup)?
7. Не пропущены ли edge cases?
8. Правильно ли типизировано?

Очерёдность по сложности

Неделя 1 — простые (useState/useEffect):

- useBoolean, useCounter, useToggle
- useDocumentTitle, useIsClient, useIsMounted
- useUnmount, useTimeout, useInterval

Неделя 2 — средние (useRef/useCallback):

- useDebounceValue, useDebounceCallback
- useEventListener, useHover, useOnClickOutside
- useLocalStorage, useSessionStorage, useReadLocalStorage
- useWindowSize, useMediaQuery

Неделя 3 — сложные (DOM API/браузерные API):

- useIntersectionObserver, useResizeObserver
- useCopyToClipboard, useClickAnywhere
- useCountdown, useStep
- useScript, useScrollLock

Неделя 4 — комплексные:

- useDarkMode, useTernaryDarkMode
- useMap, useScreen
- useEventCallback, useIsomorphicLayoutEffect

Формат ежедневной практики

30-40 минут в день:

- └─ 5 мин — повторить вчерашний хук по памяти
- └─ 15 мин — новый хук (открыл → закрыл → написал)
- └─ 5 мин — сверил с оригиналом, записал отличия
- └─ 10 мин — добавил тесты (Vitest) на свой хук

Что именно проверяет написание хуков

1. Понимание cleanup (очистки)

```
// . Частая ошибка: забыли cleanup
function useInterval(callback: () => void, delay: number) {
  useEffect(() => {
    setInterval(callback, delay); // утечка! интервал живёт вечно
  }, [callback, delay]);
}

// Правильно: возвращаем cleanup
function useInterval(callback: () => void, delay: number | null) {
  useEffect(() => {
    if (delay === null) return;
    const id = setInterval(callback, delay);
    return () => clearInterval(id); // ← очистка
  }, [callback, delay]);
}
```

2. Проблема устаревших замыканий (stale closure)

```
// callback захвачен один раз и никогда не обновляется
function setTimeout(callback: () => void, delay: number) {
  useEffect(() => {
    const id = setTimeout(callback, delay);
    return () => clearTimeout(id);
  }, [delay]); // callback не в зависимостях → stale closure
}

// useRef для хранения актуального callback
function setTimeout(callback: () => void, delay: number | null) {
  const callbackRef = useRef(callback);
  callbackRef.current = callback; // всегда актуальный

  useEffect(() => {
    if (delay === null) return;
    const id = setTimeout(() => callbackRef.current(), delay);
    return () => clearTimeout(id);
  }, [delay]);
}
```

3. SSR-безопасность

```
// Упадёт в Next.js при SSR
function useLocalStorage<T>(key: string, initialValue: T) {
  const [value, setValue] = useState<T>(() => {
    const stored = localStorage.getItem(key); // ReferenceError на
сервере!
    return stored ? JSON.parse(stored) : initialValue;
  });
}

// Проверка window
function useLocalStorage<T>(key: string, initialValue: T) {
  const [value, setValue] = useState<T>(() => {
    if (typeof window === 'undefined') return initialValue;
    try {
      const stored = localStorage.getItem(key);
      return stored ? JSON.parse(stored) : initialValue;
    } catch {
      return initialValue;
    }
  });
}
```

4. Типизация дженериками

```
// Без дженерика – не знаем тип значения
function useLocalStorage(key: string, initialValue: any) {
  // ...
}

// Дженерик
function useLocalStorage<T>(key: string, initialValue: T): [T, (value:
T) => void] {
  // ...
}
```

Связанное

- React рендеринг и производительность
- Управление состоянием
- TypeScript продвинутый
- Алгоритмические задачи
- Event Loop макротаски микротаски

Soft skills собеседование

Технических навыков недостаточно. На Middle+ собеседованиях ~30% времени — поведенческие вопросы. Проверяют: адекватность, умение решать проблемы, работу в команде.

1. Расскажи про конфликт в команде

О чём спрашивают на самом деле:

- Умеешь ли ты решать проблемы, а не создавать их
- Можешь ли посмотреть на ситуацию глазами другого
- Готов ли признавать свои ошибки

Формат STAR (обязательно!):

	Что значит	Пример
Situation	Контекст	«В проекте event-платформы за месяц до релиза...»
Task	Твоя задача	«Нужно было выбрать между TanStack Query и Redux Toolkit Query»
Action	Что ты СДЕЛАЛ	«Я собрал таблицу сравнения, показал на примере двух страниц...»
Result	Результат (цифры!)	«Выбрали TanStack Query, скорость разработки выросла на 40%»

Пример хорошего ответа

S: На проекте был спор: тимлид настаивал на SSR для всех страниц через Next.js, а я видел, что админке SSR не нужен и замедляет сборку.

T: Нужно было принять решение, не затягивая спринт.

A: Я не пошёл спорить на словах. За выходные сделал PoC: замерил время сборки с SSR и без на админке. Разница — 3.5 минуты против 22 секунд. Принёс цифры, а не мнение. Предложил компромисс: публичные страницы — SSR, админка — SPA.

R: Сократили время CI с 8 до 4 минут, тимлид согласился, подход закрепили в архитектурном решении.

Антипаттерны

- «Конфликтов не было, мы все друзья» — либо врёшь, либо не замечаешь проблем

- «Тимлид был дурак, я ему объяснил» — не льёт воду на мельницу твоей адекватности
 - Длинная преамбула без Action/Result
-

2. Самый сложный баг

О чём спрашивают: методология поиска, системное мышление, настойчивость.

Пример структуры ответа

Баг: Пользователи жаловались, что корзина билетов «сама очищается» через 5-10 минут после добавления.

Диагностика (шаг за шагом):

1. Попытался воспроизвести — не смог. Понял: гонка состояний.
2. Добавил логирование в критический путь: когда добавляется билет, когда очищается корзина
3. Нашёл закономерность: очистка происходила после фонового `refetch'a`
4. Увидел в логах: `TanStack Query` при ревалидации возвращал `{ cart: null }`, `Zustand` перезаписывал корзину

Причина: Бэкенд возвращал `null` вместо пустого массива при отсутствии корзины. `onSuccess` в `useQuery` делал `setCart(data.cart) → null`.

Исправление: На бэке — отдавать `[]` вместо `null`. На фронте — добавил `guard: setCart(data.cart ?? [])` как защиту на будущее.

Предотвращение: Добавил тест на `null`-ответ API, добавил правило в линтер: не использовать `data?.cart` без fallback.

Что хотят услышать

- Ты НЕ гадаешь — ты выдвигаешь гипотезы и проверяешь
 - Ты идёшь от симптомов к корневой причине
 - Ты фиксишь системно (тест, линтер), а не точно
-

3. Почему уходишь / ищешь работу

Золотое правило: не поливать грязью текущего работодателя. Даже если там ад.

Плохие ответы

- «Там бардак и технологии древние» — будешь жаловаться и на нас
- «Мало платят» — уйдёшь, как только предложат больше

- «Токсичный коллектив» — может, токсичен ты

Хорошие ответы

- «Хочу расти в Middle+/Senior, а на текущем месте достиг потолка — все процессы отлажены, вызовов нет. Ищу проект, где можно влиять на архитектуру»
- «Интересен ваш стек и продукт. На текущем месте пилим внутреннюю CRM, а хочется работать над продуктом с реальными пользователями»
- «Компания меняет фокус на аутсорс, а я хочу развиваться в продуктовой разработке»

Формула: акцент на РОСТ + совпадение с компанией, а не на проблемы прошлого места.

4. Как оцениваешь задачу по времени

О чём спрашивают: умеешь ли декомпозировать, реалистичен ли, не завалишь ли спринт.

Пример структуры

1. **Декомпозирую:** читаю задачу → разбиваю на подзадачи. Если требований не хватает — иду к РМ/аналитику
2. **Оцениваю риски:** неизвестные мне части (новая библиотека, незнакомый API) — ставлю коэффициент 2х
3. **Оставляю буфер:** 20% времени на тесты и ревью, 20% на «неизвестное неизвестное»
4. **Если не укладываюсь:** сообщаю ЗАРАНЕЕ, а не в день дедлайна. Предлагаю план Б (что можно сократить/отложить)

Что хотят услышать

- Ты не гадаешь «3 дня» с потолка
 - Ты закладываешь риски и тестирование
 - Ты коммуницируешь проблемы ДО того, как стало поздно
-

5. Кем ты видишь себя через 3 года

Идеальный ответ: технический рост в рамках компании.

- «Хочу стать тимлидом» — компании может быть не нужен ещё один менеджер
- «Открыть свою компанию» — зачем нам тебя нанимать на 1-2 года

- «Middle+ → Senior → Tech Lead в рамках вашего продукта. Интересна глубокая экспертиза в React и архитектуре, менторство джуниоров, влияние на технические решения»

Формула: тех-рост (не менеджерский) + привязка к продукту/компании.

6. Расскажи про проект, которым гордишься

- Выбери проект, где ты ПРИНИМАЛ РЕШЕНИЯ, а не просто выполнял задачи
 - STAR-формат обязателен
 - Акцент на ТВОЙ вклад: «я выбрал», «я предложил», «я оптимизировал» — не «мы сделали»
-

7. Вопросы, которые Ты задаёшь компании

Показывают твой уровень. Хорошие вопросы:

- «Как устроен процесс код-ревью? Кто и как ревьюит?»
- «Какая архитектура фронтенда? Почему выбрали именно её?»
- «Как измеряете качество кода? Есть ли тесты, покрытие, линтинг в CI?»
- «Как происходит онбординг? Есть ли документация?»
- «Какие самые большие технические вызовы у команды сейчас?» — этот вопрос показывает, что ты хочешь решать проблемы

Плохие вопросы:

- «Сколько платят?» (это знают до собеседования)
 - «Можно ли удалённо?» (если в вакансии написано)
 - «Какой стек?» (ты должен знать ДО собеседования)
-

Связанное

- Вопросы на собеседовании FullStack
- Вопросы на собеседовании FullStack

Вопросы на собеседовании FullStack

Вопросы на собеседовании FullStack

Развёрнутые ответы на ключевые вопросы, проверяющие архитектурное мышление Middle+/Senior FullStack-разработчика.

1. Самый сложный фуллстэк-проект: технологии и почему

Структура ответа (STAR: Situation → Task → Action → Result)

О чём спрашивают на самом деле:

- Умеешь ли ты принимать архитектурные решения, а не просто «сказали — сделал»
- Понимаешь ли компромиссы (trade-offs) между технологиями
- Видишь ли проект целиком, а не только свой кусок

Пример развёрнутого ответа

Situation: Event-платформа с продажей билетов. 50K+ пользователей, пиковые нагрузки во время старта продаж (10K RPS в первые 5 минут).

Task: Сделать систему, которая не ляжет при наплыве, с实时-обновлением остатков билетов.

Action — выбор технологий и обоснование:

Технология	Почему
Next.js 14 (App Router)	SSR для SEO страниц событий, ISR для каталога, Server Components для снижения JS-бандла
TypeScript (strict)	Без вариантов: типизация API-контрактов между фронтом и бэком
NestJS (бэкэнд)	Модульная архитектура, DI из коробки, декораторы = читаемый код. Альтернатива: Fastify + самописная структура
PostgreSQL	ACID для билетов (деньги!), JSONB для гибких настроек событий
Redis	Кэш + очередь + блокировки билетов при бронировании. Без него — гонка за билетами
BullMQ (очереди на Redis)	Асинхронная отправка email/SMS, генерация PDF-билетов
WebSocket (Socket.IO)	Живое обновление остатков билетов на странице события
Docker + k8s	Горизонтальное масштабирование: больше подов → больше RPS
tRPC или GraphQL	Типобезопасный контракт между фронтом и бэком. tRPC если один клиент, GraphQL если мобилка + веб

Result: Выдержали 12K RPS, p99 latency < 200ms, ноль овербукингов.

Ключевое в ответе: не список, а ОБОСНОВАНИЕ каждого выбора

- Плохо: «Мы использовали Next.js, NestJS, PostgreSQL и Redis.»
- Хорошо: «Redis взяли для атомарного резервирования билетов через

INCR, потому что PostgreSQL при 10K конкурентных UPDATE на одну строку даёт локи.»

2. Оптимизация при наплыве пользователей

С чего начинать: измерять, а не гадать

Порядок действий:

1. Снимаем метрики (Grafana, DataDog, NewRelic) — CPU, memory, latency, RPS
2. Находим узкое место: база? бэкенд? фронтенд? сеть?
3. Чиним по приоритету «бутылочного горлышка»

Типичные проблемы и решения

База данных — узкое место

Симптом: медленные запросы, таймауты, очередь коннектов.

Решения:

- **Индексы:** EXPLAIN ANALYZE → добавить недостающие. Для event-платформы: индекс на event_id + status в таблице билетов
- **Партиционирование:** разбить таблицу билетов по event_id — старые события не мешают новым
- **Репликация:** primary — запись, replicas — чтение. Список событий читаем с реплики
- **Пул соединений:** PgBouncer/Pgpool — 1000 запросов ≠ 1000 коннектов к PG
- **Кэш:** Redis. Сессии, остатки билетов, конфиг событий — всё в кэш

```
// Паттерн: проверка остатка через Redis, затем списание через БД
async reserveTicket(userId: string, eventId: string) {
  // 1. Атомарно проверяем и резервируем в Redis
  const remaining = await redis.decr(`event:${eventId}:tickets`);
  if (remaining < 0) {
    await redis.incr(`event:${eventId}:tickets`); // откат
    throw new Error('Sold out');
  }
  // 2. Асинхронно списываем в БД (не блокируем пользователя!)
  await queue.add('create-ticket', { userId, eventId });
  return { success: true };
}
```

Бэкенд — узкое место

Решения:

- **Горизонтальное масштабирование:** больше инстансов за балансировщиком
- **Rate limiting:** ограничить 1 IP = N запросов/сек. Защита от ботов и

перекупов

- **Async first:** email, PDF, уведомления — только через очередь. HTTP-запрос не должен ждать генерации PDF
- **HTTP/2 + gzip/brotli:** уменьшить размер ответов

Фронтенд — узкое место

- **Code splitting + lazy loading:** админка и страница события — разные бандлы
- **Виртуализация списков:** 1000 событий в каталоге $\neq$ 1000 DOM-узлов (react-window, TanStack Virtual)
- **Debounce/throttle:** строка поиска, скролл-ивенты
- **Optimistic UI:** показали «Билет забронирован» сразу, подтверждение — фоном

Что точно не надо делать

- Сразу менять стек («давайте перепишем на Go»)
- Добавлять кэш везде без инвалидации («у пользователя билет есть, а мы говорим sold out»)
- Игнорировать холодный старт serverless (если Lambda — держите warm)

3. Архитектура БД: пользователи и события со множеством связей

Требования event-платформы

- Пользователи: регистрация, роли (организатор/участник), профиль
- События: название, описание, дата, место, типы билетов, цены
- Связи: пользователь покупает билет → бронирование. Организатор создаёт событие → владение. Участник сохраняет в избранное.
- Плюс: отзывы, рейтинги, теги, категории

Модель данных (упрощённая)

users

id: UUID PK
email: string UNIQUE
name: string
role: 'user' | 'organizer' | 'admin'
created_at: timestamp

events

id: UUID PK
organizer_id: UUID FK → users.id
title: string
description: text
venue: string
start_date: timestamp
end_date: timestamp
status: 'draft' | 'published' | 'cancelled'
config: JSONB -- гибкие настройки (кастомные поля, галочки)
created_at: timestamp

ticket_types

id: UUID PK
event_id: UUID FK → events.id
name: string -- 'VIP', 'Standard', 'Early Bird'
price: decimal
quantity: integer -- всего мест
available: integer -- осталось (денормализация для скорости!)
sale_start: timestamp
sale_end: timestamp

bookings

id: UUID PK
user_id: UUID FK → users.id
event_id: UUID FK → events.id
ticket_type_id: UUID FK → ticket_types.id
quantity: integer
total_price: decimal
status: 'pending' | 'confirmed' | 'cancelled' | 'refunded'
created_at: timestamp
-- Составной индекс для частых запросов:
INDEX idx_user_event (user_id, event_id)
INDEX idx_event_status (event_id, status)

favorites

user_id: UUID FK → users.id
event_id: UUID FK → events.id
PRIMARY KEY (user_id, event_id) -- составной ключ вместо отдельного id

reviews

id: UUID PK
user_id: UUID FK → users.id

```
event_id: UUID FK → events.id
rating: integer CHECK (rating >= 1 AND rating <= 5)
comment: text
created_at: timestamp
UNIQUE (user_id, event_id) -- один отзыв на событие
```

```
event_tags
event_id: UUID FK → events.id
tag_id: UUID FK → tags.id
PRIMARY KEY (event_id, tag_id)
```

Ключевые решения

1. JSONB для config — почему не EAV

EAV (Entity-Attribute-Value) даёт гибкость, но запросы — боль:

```
-- EAV: найти события с «бесплатной парковкой»
SELECT event_id FROM event_attributes
WHERE key = 'parking' AND value = 'free';
-- против
SELECT * FROM events WHERE config->>'parking' = 'free'; -- JSONB + GIN
индекс
```

2. Денормализация ticket_types.available

Храним остаток в таблице типов билетов, а не считаем SUM(quantity) из bookings. Потому что при 10К бронирований в минуту SELECT SUM() — бутылочное горлышко.

3. Составной PRIMARY KEY вместо автоинкремента

favorites(user_id, event_id) — не нужен отдельный id, если связь уникальна. Экономия места + индекс.

4. Мягкое удаление

```
ALTER TABLE events ADD COLUMN deleted_at timestamp NULL;
-- Все SELECT-запросы добавляют WHERE deleted_at IS NULL
```

Пользователи хотят видеть историю купленных билетов, даже если событие отменили.

Антипаттерны

- MongoDB для билетов: нет ACID-транзакций → двойная продажа
- Хранить цену только в ticket_types: при изменении цены ломается история покупок. Цену при покупке копируем в bookings.total_price
- status как свободная строка. Только enum/CHECK constraint

4. Безопасность фронтенд ↔ бэкенд

Уровни защиты (defence in depth)

Уровень 1: Аутентификация и авторизация

Браузер —[JWT/сессия]—> Бэкенд —[RBAC]—> Доступ

- **JWT:** Access Token (15 мин) + Refresh Token (7 дней, httpOnly cookie)
- **Refresh Token в httpOnly Secure SameSite Cookie** — не доступен из JS (защита от XSS)
- **Access Token в памяти (переменная JS)** — не в localStorage!
- **RBAC:** role-based access. Организатор ≠ админ ≠ пользователь

```
// Middleware на бэке
@Injectable()
export class RolesGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.get<string[]>('roles', context.getHandler());
    const { user } = context.switchToHttp().getRequest();
    return requiredRoles.includes(user.role);
  }
}

// Контроллер
@Roles('organizer', 'admin')
@Post('events')
createEvent() { ... }
```

Уровень 2: Валидация ВСЕХ входящих данных

- **Никогда не доверять фронтенду.** Фронт может быть подменён (MitM, расширения, консоль)
- Валидация на бэке (class-validator/Zod) — обязательна
- Санитизация от XSS: DOMPurify на фронте, экранирование на бэке

```
// Zod — валидация и тип в одном
import { z } from 'zod';

const CreateEventSchema = z.object({
  title: z.string().min(3).max(200),
  startDate: z.string().datetime(),
  price: z.number().positive().max(1000000),
  description: z.string().max(10000).transform(s => sanitizeHtml(s)),
});

// Тип автоматически выводится из схемы
type CreateEvent = z.infer<typeof CreateEventSchema>;
```

Уровень 3: CORS, CSP, HTTPS

```
// CORS: только наш домен
app.enableCors({
  origin: ['https://myeventplatform.com'],
  methods: ['GET', 'POST', 'PATCH', 'DELETE'],
  credentials: true,
});
```

```
<!-- CSP: запрещаем inline-скрипты -->
<meta http-equiv="Content-Security-Policy"
  content="default-src 'self'; script-src 'self'; style-src 'self'
  'unsafe-inline';">
```

- **HTTPS always** — HSTS заголовок
- **SameSite=Strict** для всех кук

Уровень 4: Защита от специфичных атак

CSRF:

```
// Если API не pure-REST (есть куки): CSRF-токен
// Если SPA с JWT в заголовке Authorization: CSRF не нужен
// (браузер не прикрепит заголовок Authorization автоматически)
```

Rate Limiting:

```
// 100 запросов в 15 минут с одного IP
app.use(rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
}));
```

SQL Injection → не актуально при ORM:

```
// template string
db.query(`SELECT * FROM users WHERE id = ${userId}`);
// параметризованный запрос (ORM делает автоматически)
db.user.findUnique({ where: { id: userId } });
```

IDOR (Insecure Direct Object Reference):

```
// Не проверяем принадлежность
@Get('bookings/:id')
async getBooking(@Param('id') id: string) {
  return this.db.booking.findUnique({ where: { id } });
}

// Проверяем, что бронирование принадлежит текущему пользователю
@Get('bookings/:id')
async getBooking(@Param('id') id: string, @Req() req) {
  return this.db.booking.findFirst({
    where: { id, userId: req.user.id }
  });
}
```

Чек-лист безопасности

- [] JWT Refresh в httpOnly cookie, не в localStorage
- [] Все входные данные валидируются на бэке (Zod/class-validator)
- [] CORS разрешает только свой домен
- [] CSP-заголовки
- [] Rate limiting на критические эндпоинты
- [] Все SELECT-запросы проверяют принадлежность (userId)
- [] Пароли: bcrypt/argon2, никогда не plain text
- [] HTTPS enforced (HSTS)
- [] Зависимости проверены (npm audit, Snyk)

5. Отладка багов на стороне клиента

Методология поиска

Шаг 1: Воспроизвести

- Баги «только у клиента» — почти всегда связаны с:
- Конкретным окружением (браузер, ОС, расширения)
 - Конкретными данными (у этого пользователя особый набор)
 - Таймингами (гонка состояний, slow network)

Способы воспроизвести:

- Попросить **точные шаги** (не «у меня не работает», а «захожу на / events/123 → жму Купить → белый экран»)
- Собрать **окружение**: браузер + версия, ОС, расширения (попросить попробовать в инкогнито без расширений!)
- Воссоздать **состояние**: экспортировать данные пользователя в тестовое окружение

Шаг 2: Собрать информацию

Инструменты в порядке эффективности:

Инструмент	Что даёт
Sentry / Rollbar / LogRocket	Автоматический сбор ошибок + replay сессии
Кастомный логгер	logger.error('checkout failed', { userId, cart, step })
Chrome DevTools → Network	Какие запросы ушли, с какими ответами
Performance API	performance.getEntriesByType('navigation')
User Report	Форма «Что случилось?» с авто-прикреплением скриншота и логов

Практический флоу отладки

```
// 1. Оборачиваем критический код в try/catch с контекстом
async function checkout(cart: Cart) {
  try {
    logger.info('checkout.start', { cartId: cart.id, items: cart.items.length });
    const order = await api.createOrder(cart);
    logger.info('checkout.order_created', { orderId: order.id });
    await redirectToPayment(order.paymentUrl);
  } catch (error) {
    logger.error('checkout.failed', {
      cartId: cart.id,
      step: error.step || 'unknown',
      error: error.message,
      userAgent: navigator.userAgent, // ← контекст!
    });
    throw error;
  }
}

// 2. Sentry – автоматический сбор
Sentry.init({
  dsn: '...',
  environment: process.env.NODE_ENV,
  beforeSend(event) {
    // Добавляем версию приложения, ID пользователя
    event.user = { id: currentUser.id };
    event.tags = { app_version: APP_VERSION };
    return event;
  },
});
```

Типичные баги «только у клиента» и их причины

Симптом	Вероятная причина	Решение
Работает в Chrome, не работает в Safari	Нет полифила (ResizeObserver, Intl, CSS-свойство)	browserslist + @babel/preset-env + проверять caniuse
Пустой экран у 1% пользователей	AdBlock ломает код (React DevTools, Sentry-лайблери запрещены)	ErrorBoundary, загружать критическое синхронно
Заказ подвис на оплате	Пользователь закрыл вкладку, WebSocket разорвался	Idempotency key: повторный запрос не создаст дубликат заказа
«У меня всё лагает»	Расширения (Grammarly, LastPass), медленный ПК, 100+ вкладок	Проверить performance.memory, предложить инкогнито
Форма не отправляется	Автозаполнение браузера вставило пробел в конец email	.trim() на бэке + debounce на фронте

Продвинутые инструменты

```
// 1. Feature flags: быстро отключить проблемный функционал для
конкретного пользователя
if (featureFlags.isEnabled('new-checkout', user.id)) {
  return <NewCheckout />;
}
return <OldCheckout />;

// 2. Session Replay: LogRocket/FullStory записывают всё, что делал
пользователь
// Видим: пользователь нажал кнопку → ушёл запрос → ответ 500 → ошибка

// 3. Кастомный ErrorBoundary в React
class CheckoutErrorBoundary extends React.Component {
  componentDidCatch(error, errorInfo) {
    logger.error('checkout.crash', {
      error: error.message,
      componentStack: errorInfo.componentStack,
      url: window.location.href,
      cart: cartStorage.get(), // сохраняем корзину
    });
    cartStorage.saveForRecovery(this.props.cart); // чтобы не
потерять
  }
  render() {
    if (this.state.hasError) {
      return <CheckoutFallback onRetry={() => this.setState({ hasError: false })} />;
    }
    return this.props.children;
  }
}
```

Что точно не надо делать

- «У меня работает» — и закрыть тикет
- Просить не-технического пользователя открыть DevTools
- Править наугад без понимания причины
- Игнорировать баги с частотой <1% (это 500 пользователей при 50К аудитории)

Связанное

- Event Loop макротаски микротаски — Event Loop глубоко
- Интернационализация и локализация i18n — локализация приложений
- Soft skills собеседование — поведенческие вопросы и подготовка

- TypeScript продвинутый
- Архитектура React компонентов
- Управление состоянием
- Тестирование React